

## 瑞萨 RA 系列

# 使用 RA8 双核 MCU 进行开发

## 引言

本应用项目重点展示了 RA8 双核 MCU 的性能优势，该芯片包含运行在 1 GHz 的 Cortex-M85（CM85）内核和运行在 250 MHz 的 Cortex-M33（CM33）内核。本文讨论了双核架构，并提供了用例来说明如何有效地将工作负载分配到两个内核上，以提升系统的整体性能。

本文为创建 RA8 双核 MCU 应用程序提供了必要的步骤指导，包括：

- 应用亮点
- 用例框图
- 工具配置
- 示例项目确认
- 将应用项目迁移到新的 RA8 双核 MCU 的步骤

## 所需资源

本应用笔记中引用了以下资源。

### 开发工具与软件

- e<sup>2</sup> studio 版本：2025-10（25.10.0）
- LLVM Embedded Toolchain for Arm v18.1.3
- 瑞萨 Flexible Software Package（FSP）v6.2.0 或更高版本。

### 目标器件

以下是本文档信息所适用的瑞萨 MCU 产品：

- RA8P1。
- RA8D2。
- RA8M2。
- RA8T2。硬件
- EK-RA8P1，RA8P1 MCU 群组评估套件（<http://www.renesas.com/ek-ra8p1>）。
- EK-RA8D2（可选），RA8D2 MCU 群组评估套件（<http://www.renesas.com/ek-ra8d2>）。
- 本应用笔记中的描述以运行 Windows® 10 操作系统的 PC 为例。关于开发工具和软件所支持操作系统的完整列表，请参阅相应的用户手册。
- 本应用笔记中的描述以运行 Windows® 10 操作系统的 PC 为例。关于开发工具和软件所支持操作系统的完整列表，请参阅相应的用户手册。

使用一根 USB 设备线缆（Type-C）连接评估套件和 PC。

## Contents

|                                                 |    |
|-------------------------------------------------|----|
| 1. 示例应用概述 .....                                 | 4  |
| 2. RA8 双核 MCU .....                             | 4  |
| 3. 使用瑞萨 e <sup>2</sup> studio 创建 RA8 双核应用 ..... | 4  |
| 3.1 为 RA8P1 双核 MCU 创建 Solution 项目 .....         | 4  |
| 3.2 调试和运行多核项目 .....                             | 8  |
| 4. 使用 RA8 双核 MCU 开发应用程序 .....                   | 13 |
| 4.1 划分系统并最大化性能 .....                            | 13 |
| 4.2 在应用中使用核间通信 .....                            | 13 |
| 4.2.1 使用核间中断 .....                              | 14 |
| 4.2.2 使用 IPC 消息 FIFO .....                      | 14 |
| 4.3 在 RA8 双核 MCU 中使用共享内存和共享资源 .....             | 15 |
| 4.3.1 在 FSP Flat 项目中使用共享内存和共享资源 .....           | 15 |
| 4.3.2 在基于 RTOS 的项目中使用共享内存和共享资源 .....            | 18 |
| 4.4 在 RA8 双核应用中使用缓存和 TCM .....                  | 18 |
| 4.4.1 紧耦合内存（TCM） .....                          | 18 |
| 4.4.1 使用 ITCM 提升性能 .....                        | 19 |
| 4.4.2 使用 DTCM 提升性能 .....                        | 22 |
| 4.4.3 使用 CTCM 提升性能 .....                        | 23 |
| 4.4.4 利用 Cortex®-CM85 内核数据缓存提升性能 .....          | 25 |
| 4.4.5 使用神经网络处理单元（NPU） .....                     | 25 |
| 5. 应用项目 .....                                   | 27 |
| 5.1 IPC - 共享内存项目 .....                          | 27 |
| 5.1.1 在应用中实现核间通信。 .....                         | 29 |
| 5.1.2 在应用中实现两个内核之间的共享内存。 .....                  | 30 |
| 5.2 RTOS/IPC/共享内存/TCM 项目 .....                  | 34 |
| 6. 验证多核 Flat Bare-Metal 项目 .....                | 34 |
| 6.1 导入项目 .....                                  | 34 |
| 6.2 编译项目 .....                                  | 35 |
| 6.2.1 编译在 CM85 内核上开发的项目 .....                   | 35 |
| 6.2.2 编译在 CM33 内核上开发的项目 .....                   | 37 |
| 6.3 下载和运行项目 .....                               | 38 |
| 7. 验证基于 FreeRTOS 的项目 .....                      | 43 |
| 7.1 导入项目 .....                                  | 43 |
| 7.2 编译项目 .....                                  | 43 |
| 7.3 下载和运行项目 .....                               | 44 |
| 8. 将双核示例项目迁移到新的双核 MCU .....                     | 47 |

---

|               |    |
|---------------|----|
| 9. 参考资料 ..... | 55 |
|---------------|----|

## 1. 示例应用概述

本文档包含的示例应用项目概述了使用瑞萨 FSP 在 RA8 双核 MCU 上开发应用程序的基本流程，特别以 RA8P1 MCU 为重点。这些流程同样适用于其他 RA8 双核 MCU。示例项目演示了在裸机实现中创建多核应用的方法，包括任务划分、利用共享内存和硬件信号量（Semaphore）进行核间通信（IPC），以及使用 TCM 和缓存（Cache）来优化性能。

## 2. RA8 双核 MCU

RA8 双核采用非对称架构，包含一个高性能 1 GHz Arm® Cortex®-M85 内核和一个 250 MHz Arm® Cortex®-M33 内核，通过利用两个内核并发执行多个任务，具备以下关键特性。

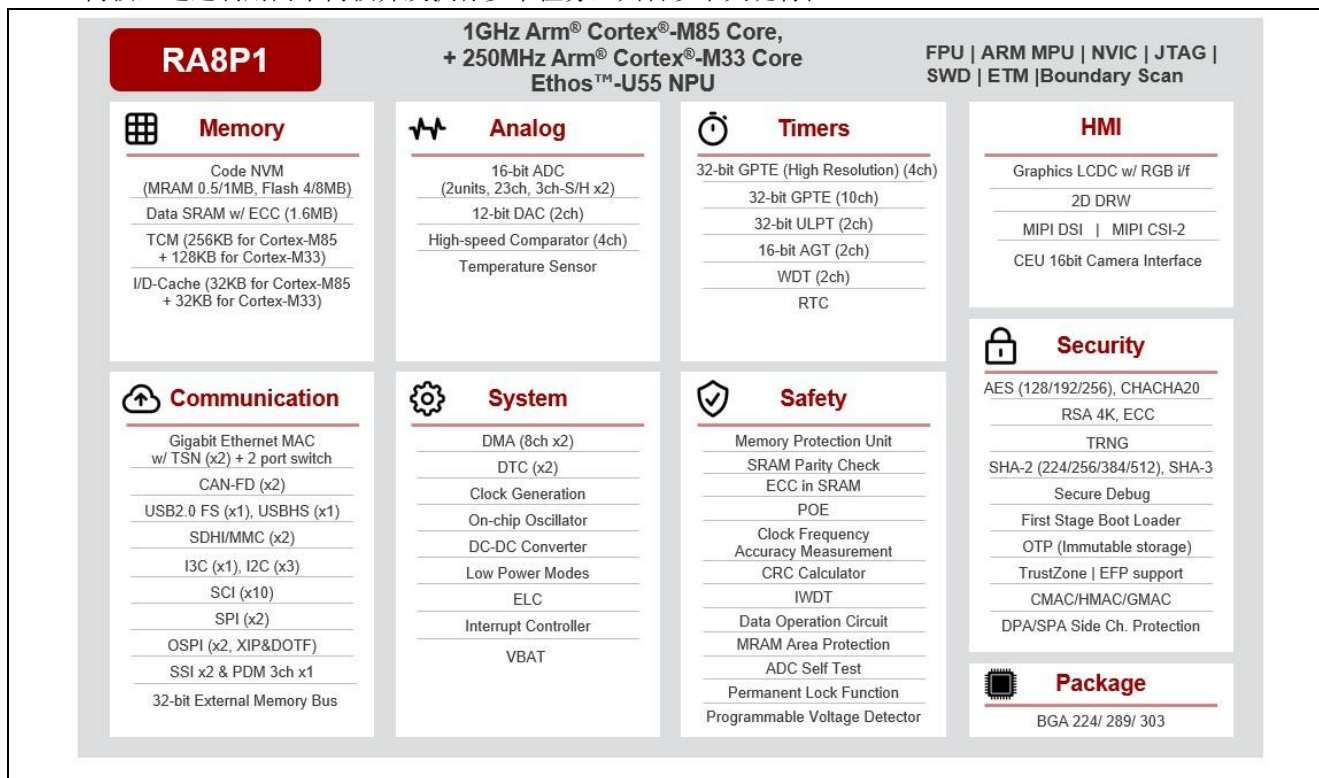


Figure 1. Example of Key Features in RA8P1 MCU

## 3. 使用瑞萨 e² studio 创建 RA8 双核应用

创建项目时，需要选择项目类型、提供名称和位置，并配置项目设置。主要设置包括 FSP 版本、目标板、工具链版本和调试器，如图 3 所示。本节提供了在 e² studio 上创建、编译和调试多核项目的逐步操作指南，以 Blade 开发板为例。

### 3.1 为 RA8P1 双核 MCU 创建 Solution 项目

对于 RA8P1 MCU 应用，请按照以下步骤在 e² studio 上创建多核项目。本节基于 Bare Metal-Blinky 项目模板。创建项目时的另一个可选模板是 Bare Metal-Minimal。

启动 e² studio，点击 File > New > Renesas C/C++ Project > Renesas RA，然后选择 Renesas RA FSP Solution > Next。

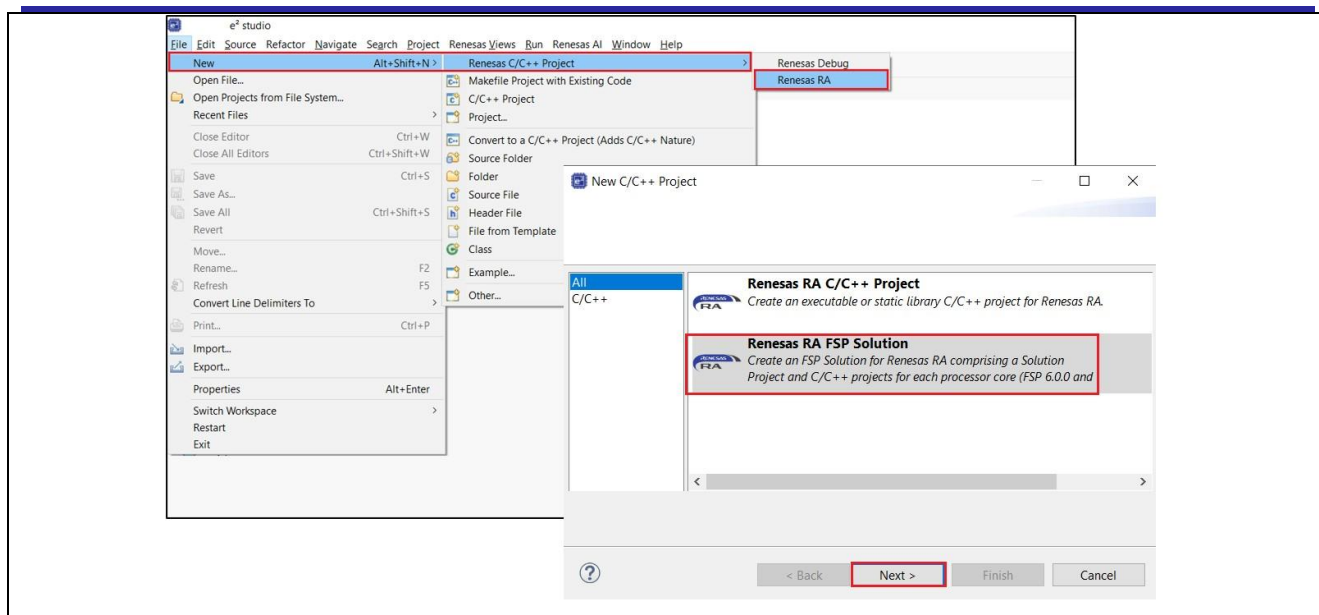


图 2. 使用 RA FSP Solution 创建双核项目示例。为新项目分配一个名称，例如

**ek\_ra8p1\_blinky**，然后点击 **Next**。

在器件选择（Device Selection）中，选择任意支持的双核 RA 套件的板卡类型（例如 EK-RA8P1 或 EK-RA8D2）。

在 Solution 模板选择（Solution Template Selection）框中，从 Multicore > Flat > Bare Metal 中选择 Blinky 项目模板，设置工具链（Toolchains）为 LLVM Embedded Toolchain for Arm，调试器（Debugger）为 J-Link ARM，然后点击 Finish。

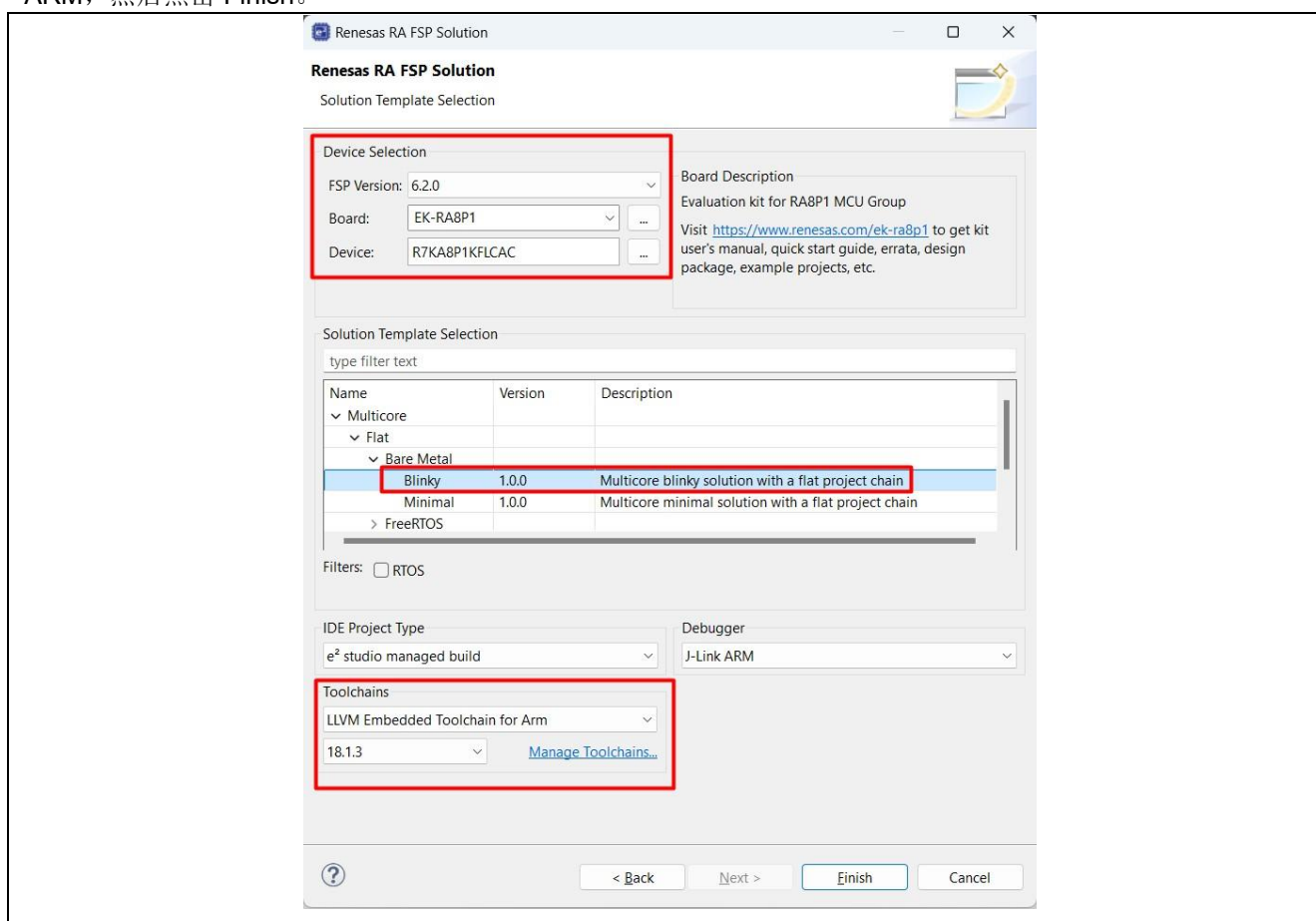


图 3. 项目选择示例

请注意，此步骤完成后，双核项目应自动成功创建并编译。它由三个项目文件夹组成，如图 4 所示。

- 名为 `ek_ra8p1_blinky` 的 Solution 项目为 CPU0 和 CPU1 提供了一组关联项目。它管理两个内核的时钟配置和存储器设置。
- `ek_ra8p1_blinky_CPU0` 项目对应 CPU0 的应用程序。
- `ek_ra8p1_blinky_CPU1` 项目对应 CPU1 的应用程序。

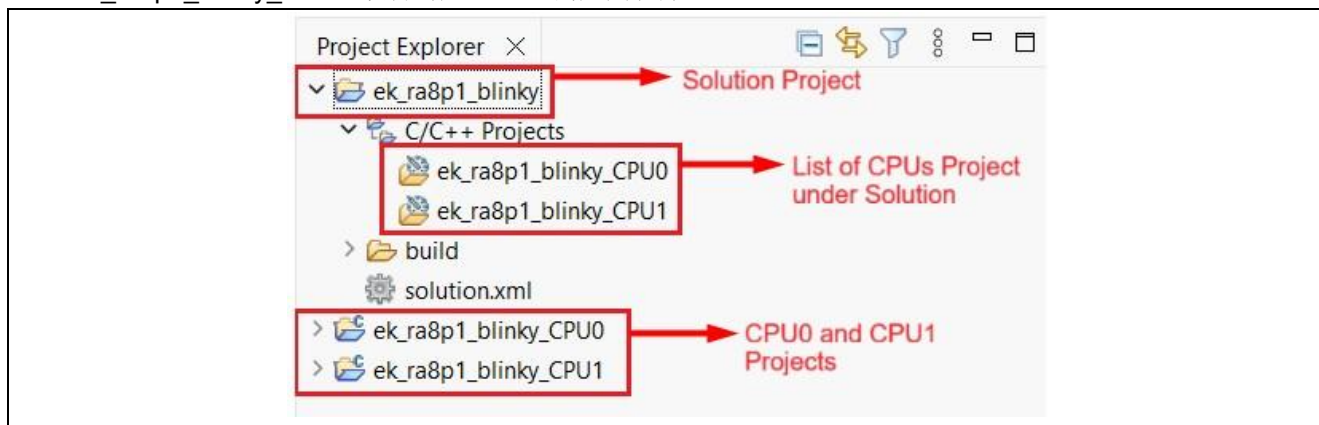
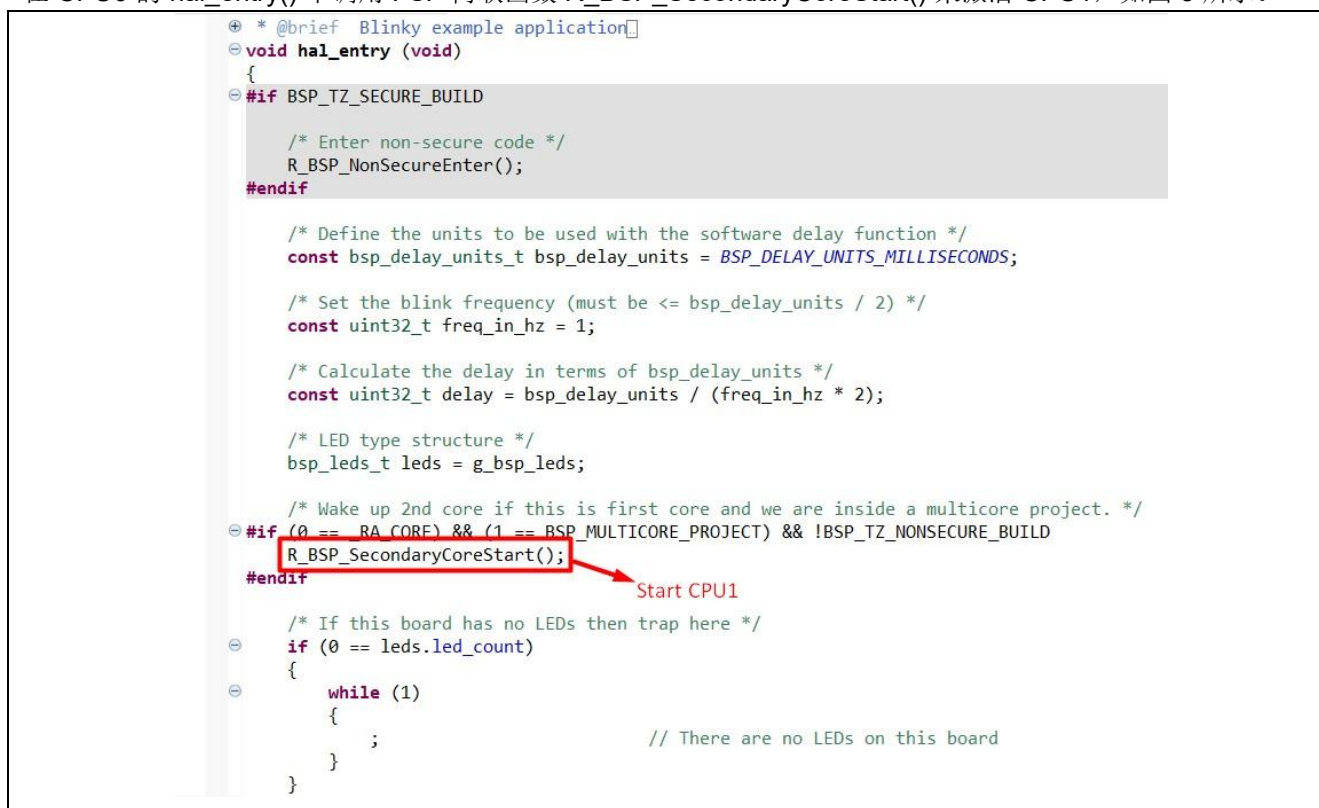


图 4. 双核项目创建成功示例

在 CPU0 的 `hal_entry()` 中调用 FSP 内联函数 `R_BSP_SecondaryCoreStart()` 来激活 CPU1，如图 5 所示。

图 5. CPU0 项目的 `hal_entry.c` 中启动 CPU1 的示例。

在此 `blinky` 示例中，CPU0 闪烁板上的 LED1，CPU1 闪烁 LED2。此功能允许用户轻松识别当前是哪个内核在翻转 LED，如图 6 所示。

```

while (1)
{
    /* Enable access to the PFS registers. If using r_ioport module then register
    * handled. This code uses BSP IO functions to show how it is used.
    */
    R_BSP_PinAccessEnable();

    #if BSP_NUMBER_OF_CORES == 1

        /* Update all board LEDs */
        for (uint32_t i = 0; i < leds.led_count; i++)
        {
            /* Get pin to toggle */
            uint32_t pin = leds.p_leds[i];

            /* Write to this pin */
            R_BSP_PinWrite((bsp_io_port_pin_t) pin, pin_level);
        }

    #else

        /* Update LED that is at the index of this core. */
        R_BSP_PinWrite((bsp_io_port_pin_t) leds.p_leds[_RA_CORE], pin_level);

    #endif

    /* Protect PFS registers */
    R_BSP_PinAccessDisable();
}

```

Toggle a single LED based on the CPU number

图 6. hal\_entry.c 中对应的 Blinky LED 实现。

重新编译项目时，请按照以下顺序进行：

- 首先编译 CPU0 项目。
- 然后编译 CPU1 项目。

如果在 **Solution** 项目中修改了任何设置，则 **Solution** 项目也必须重新编译，以将这些更改传递到 CPU0 和 CPU1 项目。

或者，您可以通过右键单击双核 **Solution** 项目并选择 **Build Project** 来简化操作。自动编译过程将按以下顺序进行：首先编译 CPU0 项目，然后编译 CPU1 项目。

对于基于 **Solution** 或多核的项目，建议从 **Solution** 项目进行编译，以确保两个 CPU 之间的配置一致性和依赖关系处理。

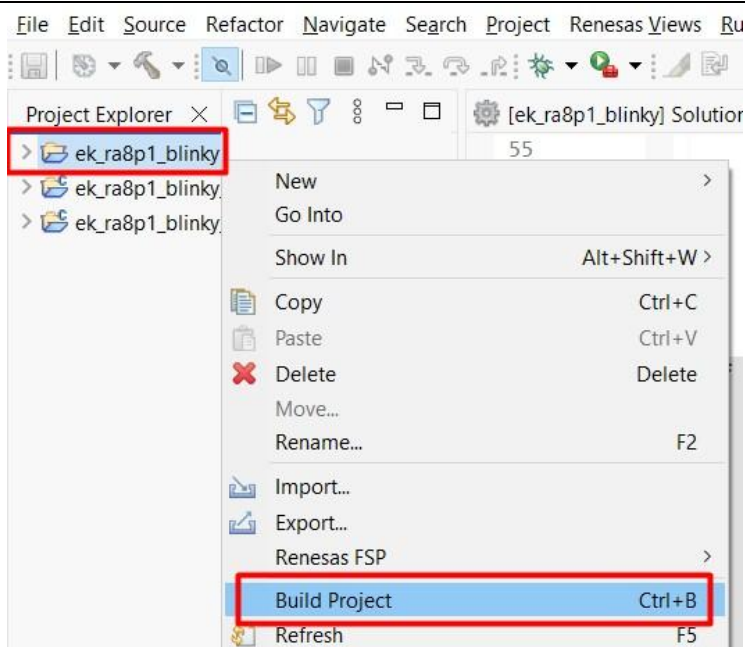


图 7. 编译双核 **Solution** 项目的示例。

确保两个 CPU 的项目都成功编译，并且在 **Debug/** 目录中生成了相应的应用程序映像 (\*.elf 文件)（例如 **Debug/\*.elf**）。

### 3.2 调试和运行多核项目

要同时调试双核，首先使用瑞萨 Flash Programmer 或瑞萨 Device Partition Manager 对 MCU 执行初始化操作，以确保其设置为保护等级 2，并且尚未设置 TrustZone 边界。如果已设置 TrustZone 边界，则需要将其重置以建立合适的调试环境。初始化完成后，即可同时调试两个内核。

使用 Device Partition Manager 进行应用程序下载

按照以下步骤使用瑞萨 Device Partition Manager 初始化设备：

1. 从 Run > Renesas Debug Tools 打开瑞萨 Device Partition Manager。
2. 在 Action 框中启用 Initialize device。
3. 选择目标 MCU 连接（Target MCU Connection）和连接类型（Connection type）。
4. 点击 Run 以初始化 MCU。

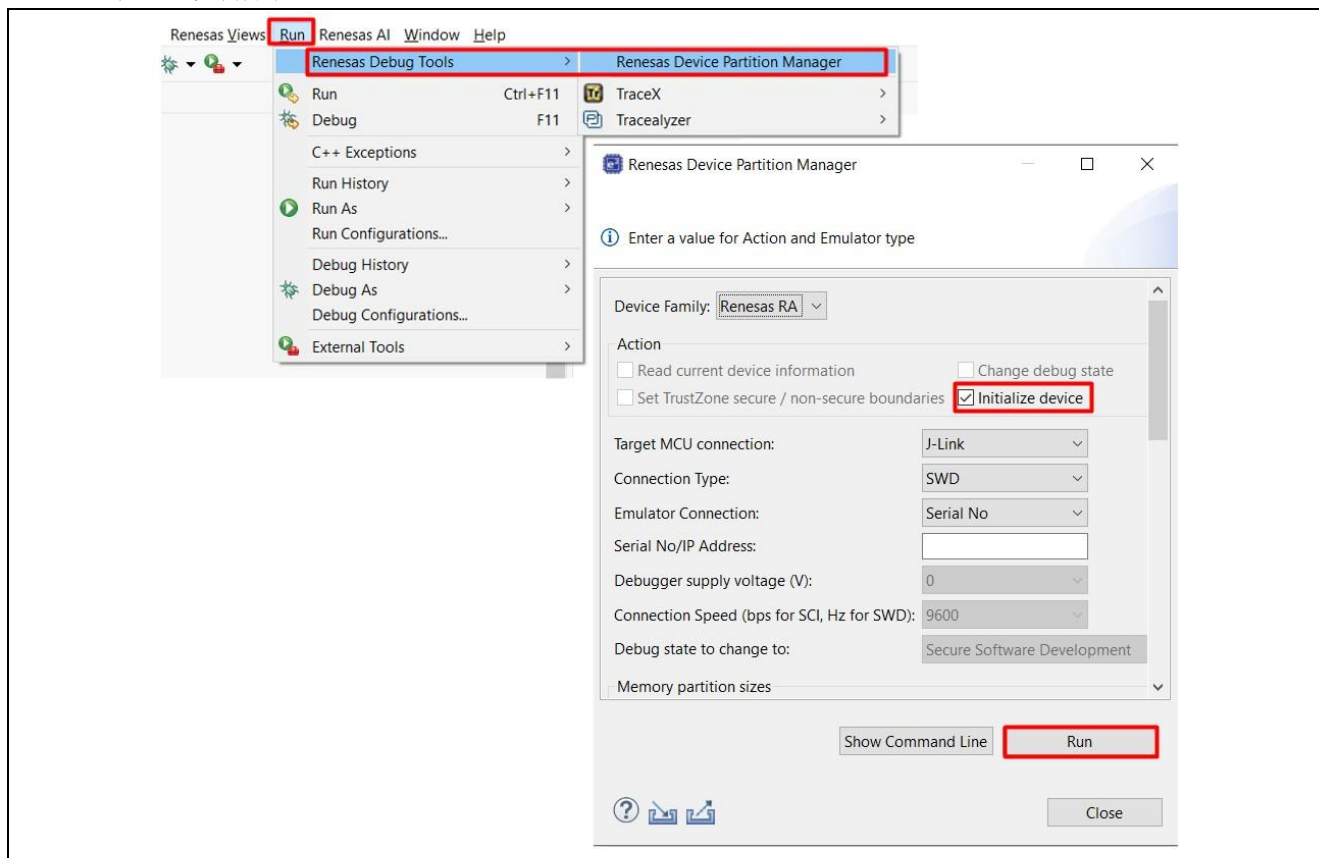


图 8. 使用 RDPM 初始化 MCU。初始化

成功后，将显示如图 9 所示的消息。

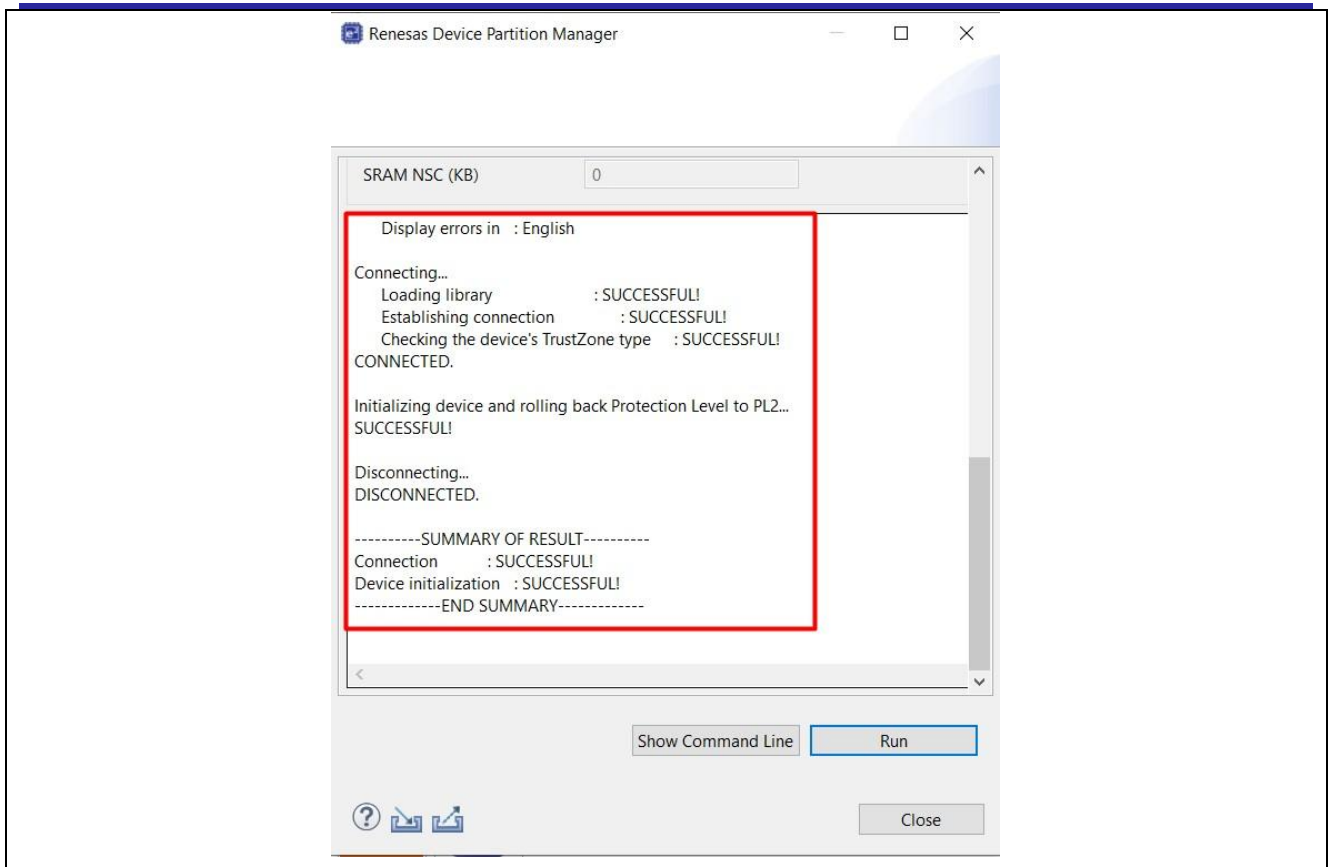


图 9. RDPM 上设备初始化成功的消息。使用瑞萨 Flash

#### Programmer 初始化设备:

1. 打开瑞萨 Flash Programmer 软件。
2. 创建一个新项目并建立与目标 MCU 的连接。
3. 导航到 Target Device 选项卡。
4. 点击 Initialize Device 执行初始化操作。

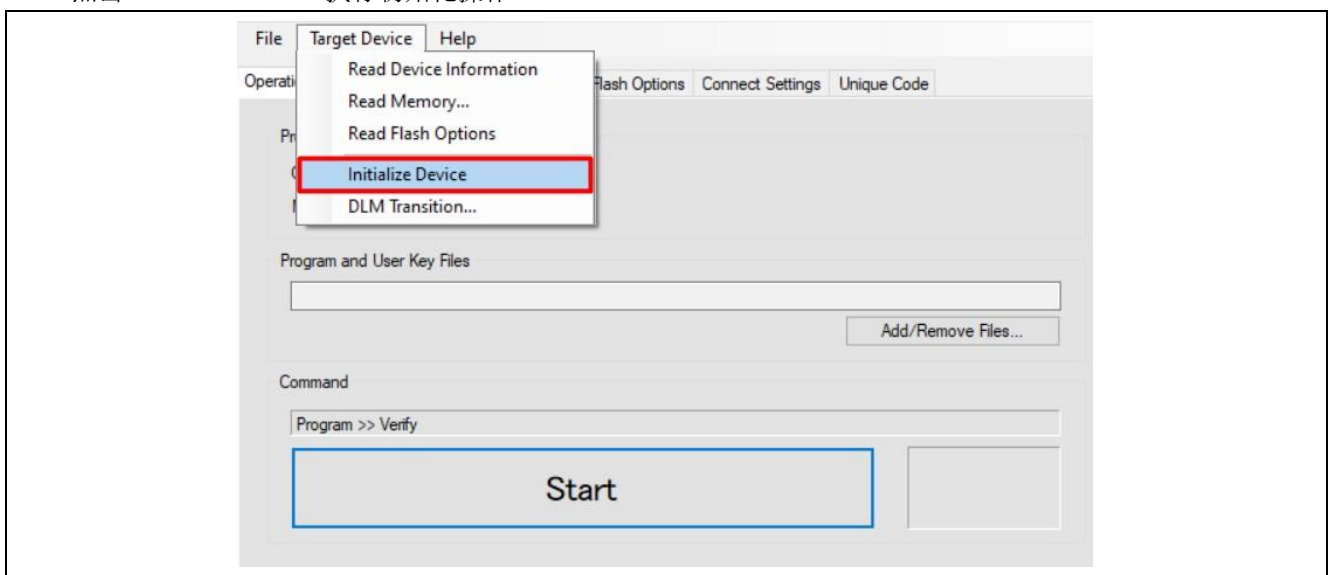


图 10. 使用 RFP 初始化 MCU。状态消息

将显示在控制台上，如图 11 所示。

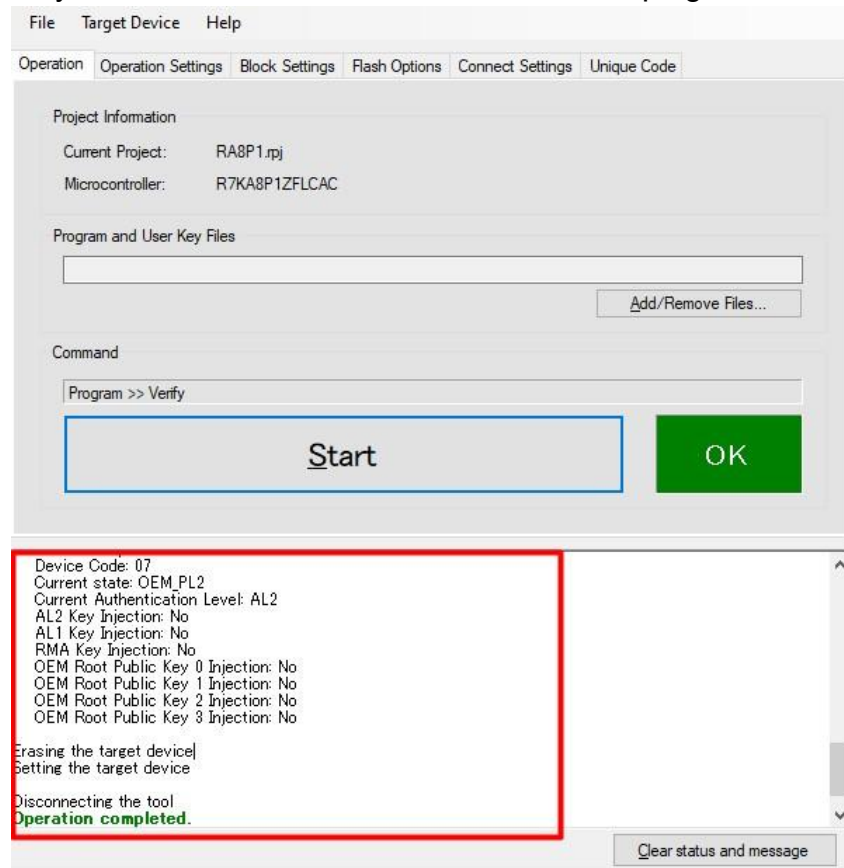


图 11. RFP 上设备初始化成功的消息。

使用 RDPM 或 RFP 初始化设备后，访问 Debug Configuration。选择 ek\_ra8p1\_blinky\_CPU1 Debug\_Multicore，然后导航到 Debugger 选项卡并点击 Connection Settings。确保 TrustZone 边界设置已禁用。

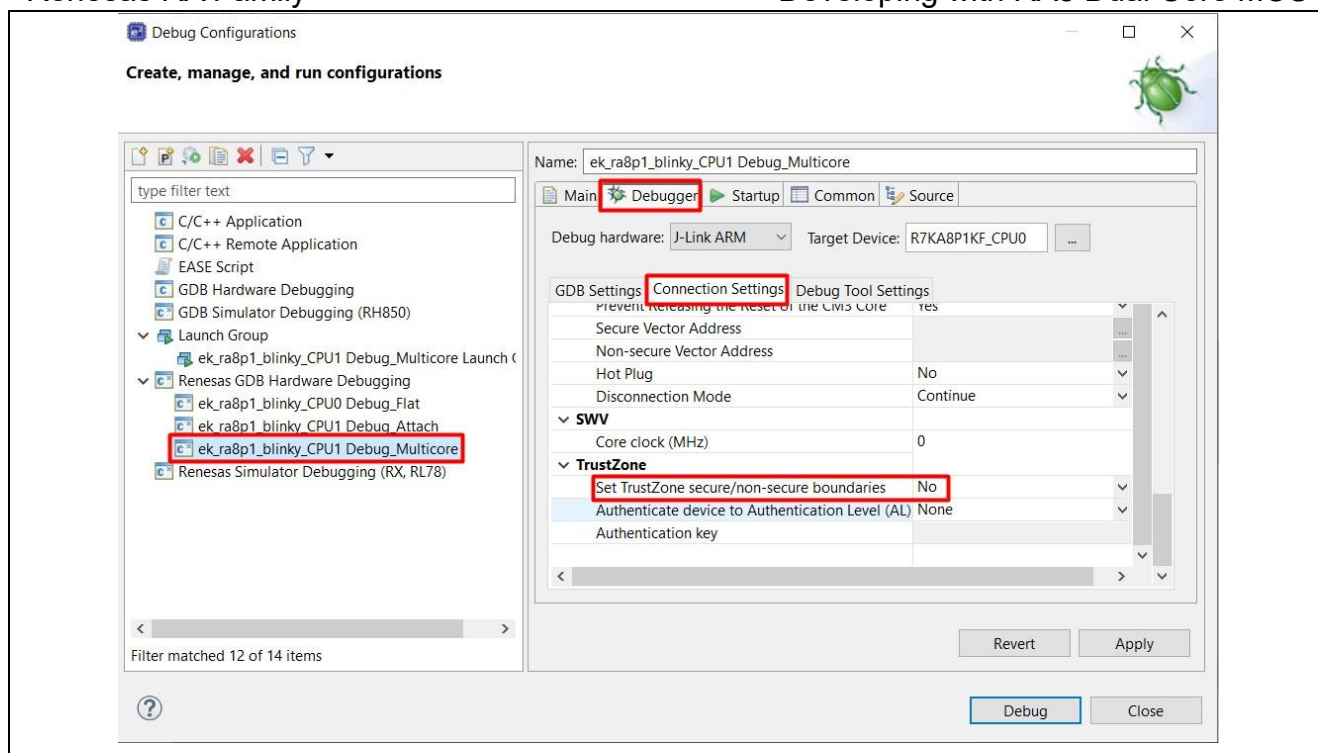


图 12. 禁用 TrustZone 边界设置的示例

e<sup>2</sup> studio 提供了高效的调试能力，允许同时调试两个内核项目。您可以通过使用一个将两个独立启动配置组合在一起的"Launch Group"来实现此目的。当 e<sup>2</sup> studio 生成 CPU1 项目时，它会自动创建此 Launch Group。

打开 Debug Configurations 对话框，选择已创建的 Debug Multicore Launch Group，然后点击 Debug 开始调试会话。

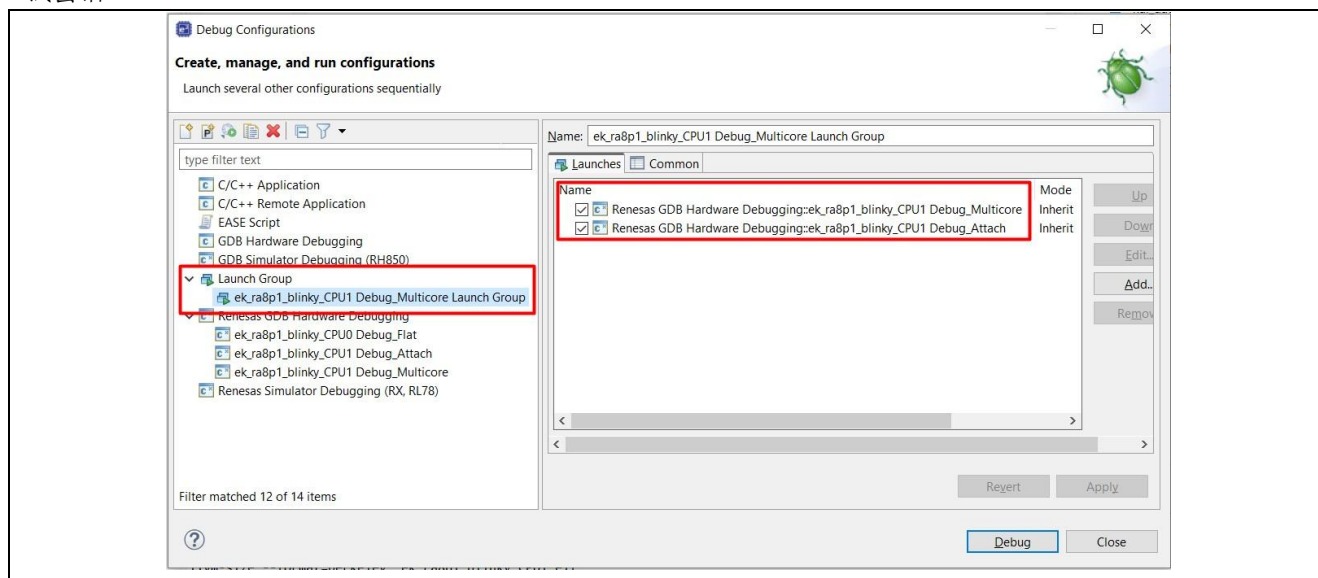


图 13. Debug Multicore Launch Group 示例

在 Debug 信息选项卡中，CPU0 项目的调试会话首先启动并连接，然后是 CPU1 项目。调试器将停止在 CPU0 项目的 Reset\_Handler() 处。当您点击两次 Resume 按钮后，CPU0 将开始执行。通过 main() 函数后，它将调用 R\_BSP\_SecondaryCoreStart() 启动 CPU1。

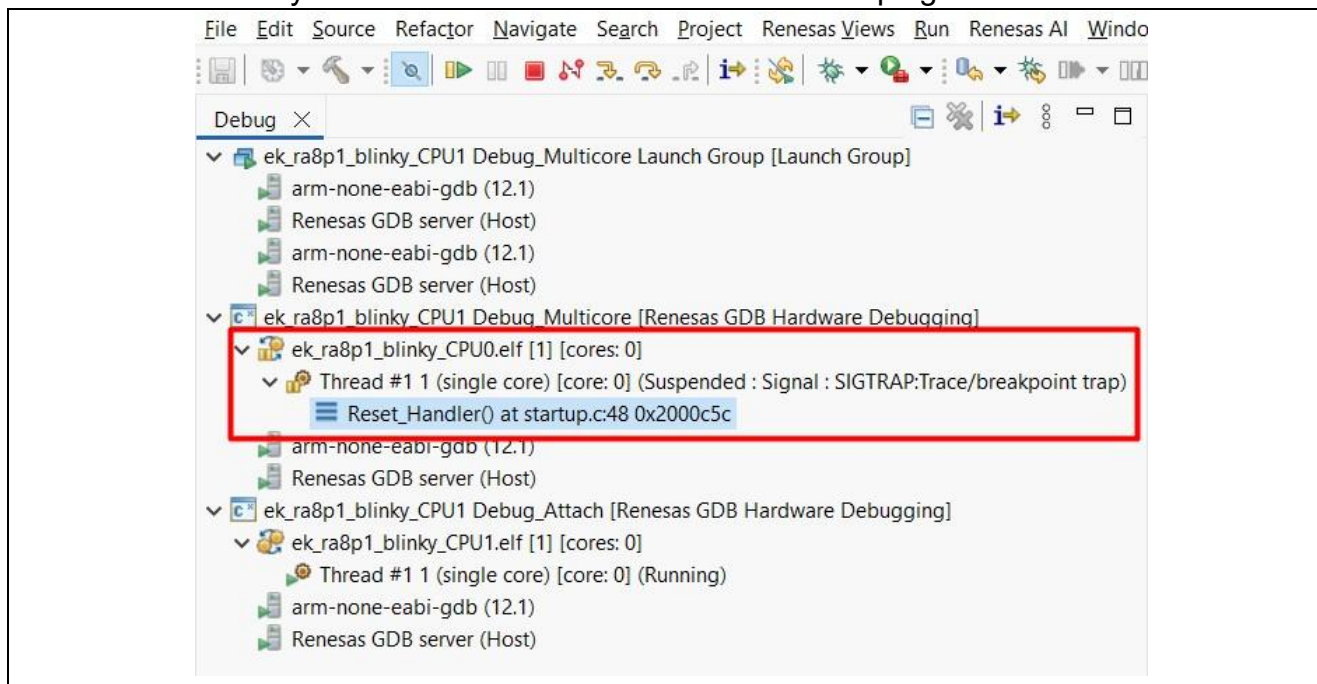
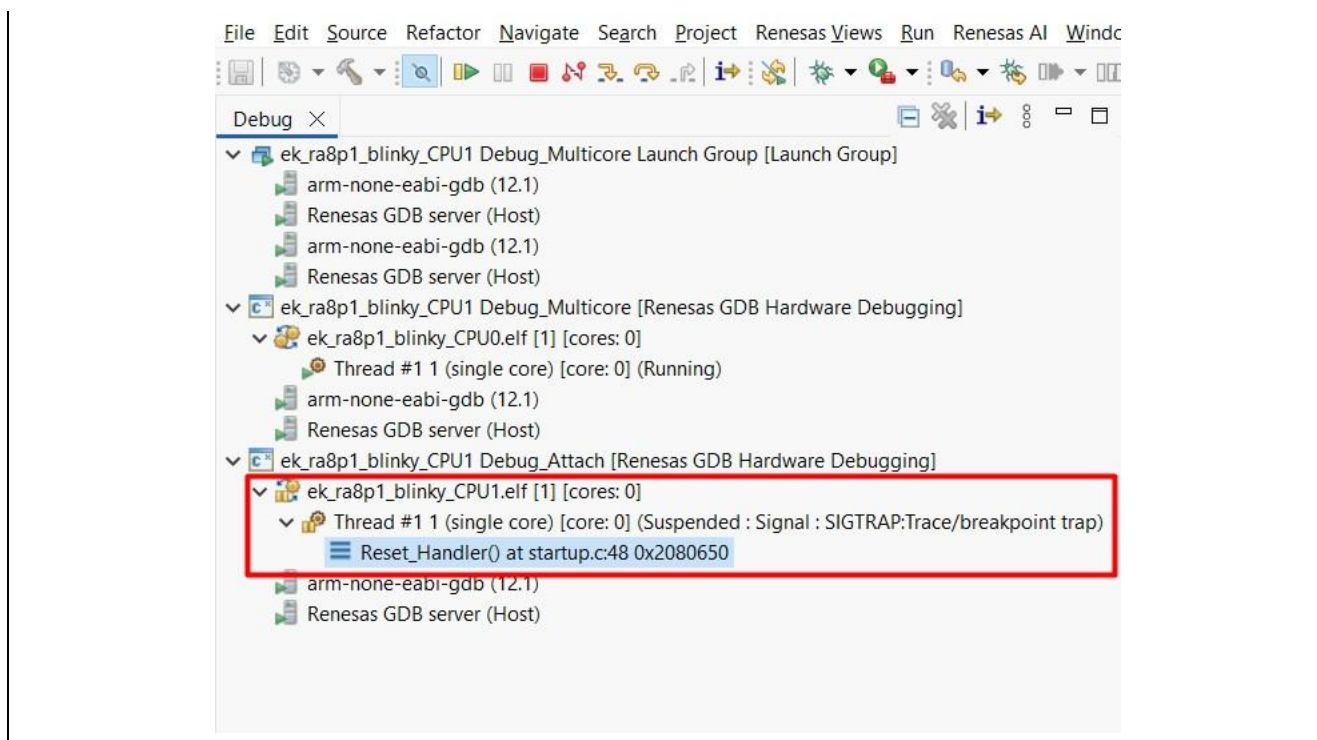


图 14. 双核执行启动调试状态示例。

如图 15 所示，一旦 CPU0 调用 `R_BSP_SecondaryCoreStart()`，CPU1 的调试会话将完全激活，从而可以进行标准的调试控制。

按下 **Resume** 按钮继续执行；LED2 将开始闪烁，表示 CPU1 已成功启动。

图 15. `R_BSP_SecondaryCoreStart()` 之后的调试状态示例

完成项目对的首次多核调试会话后，您可以通过从下拉菜单中选择相应的 **Launch Group** 来快速启动调试会话。

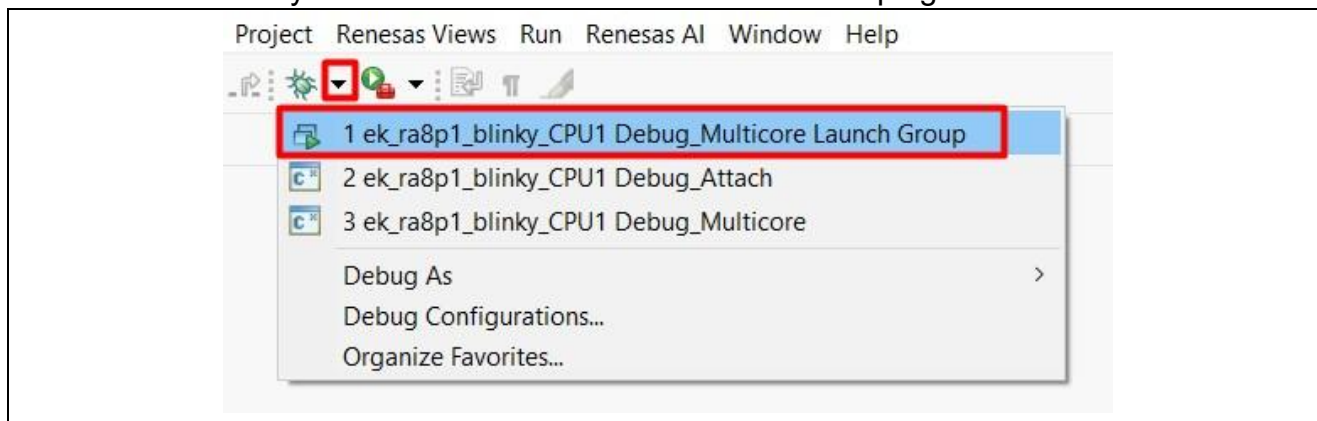


图 16. 使用调试下拉菜单快速访问的示例

## 4. 使用 RA8 双核 MCU 开发应用程序

在规划和设计利用 RA8 双核 MCU 的应用程序时，必须考虑多个因素以最大化性能。

- 将应用程序划分为多个任务，使每个 CPU 能够独立处理。
- 使用 IPC（核间通信）模块来促进运行在不同内核上的任务之间的通信。此外，需谨慎管理共享资源（Shared Resources）以防止冲突、保持数据完整性，并优化应用程序的整体效率。
- 利用 ITCM、DTCM、CTCM 和 STCM 来提升性能。
- 利用指令缓存和数据缓存来进一步提升性能。
- 将较重的任务，例如图形相关操作、数字信号处理（DSP）和人工智能/机器学习（AI/ML），分配给 CM85 内核。该内核可利用其更高的时钟频率和 M-Profile 向量扩展（MVE，即 Helium™ 技术）。相比之下，较轻的任务，包括通信栈、I/O 管理和用户界面，则适合分配给 CM33 内核。

### 4.1 划分系统并最大化性能

我们将实时控制、AI/ML 和图形任务分配到各个 CPU 内核上，以充分利用双核架构。例如，我们可以按以下方式在两个 CPU 之间划分一个典型的图形应用。

- CPU0 管理图形模块、JPEG 解码器和 SDRAM 访问。
- CPU1 处理输入/输出和用户界面，包括数据采集、触摸控制器和输出控制。

这种职责划分减少了资源争用（特别是在系统总线和共享内存方面），并使每个 CPU 能够以最小的干扰运行。它还为额外的工作负载（如实时控制环路、传感器融合或 AI/ML 推理）创建了可扩展的基础，这些工作负载可以分配给最合适的内核。

明确定义的任务划分，结合核间通信（例如 IPC 中断、共享内存和消息队列），使双核系统能够高效运行，同时确保高优先级任务可预测地执行。

### 4.2 在应用中使用核间通信

核间通信（IPC）允许 MCU 内部的两个处理器之间共享硬件资源和交换数据。IPC 还通过生成中断事件来支持通信，这些事件有助于同步和协调处理器之间的操作。

本节重点介绍 IPC 的关键组件，包括：

- 数据交换
- 任务同步
- 高效的资源共享

### 4.2.1 使用核间中断

核间中断（IPI）提供了一种低延迟机制，用于在共享数据或系统事件需要立即关注时通知另一个 CPU。RA 双核架构支持可屏蔽和不可屏蔽的核间中断，可根据应用的紧迫性需求实现灵活的优先级处理。

在图 17 所示的典型工作流程中，一个内核执行事件监控（例如外设活动、缓冲区可用性或处理任务完成），然后向对等内核发出核间中断。该中断充当轻量级通知机制，允许对等内核快速响应并在需要时执行回调。

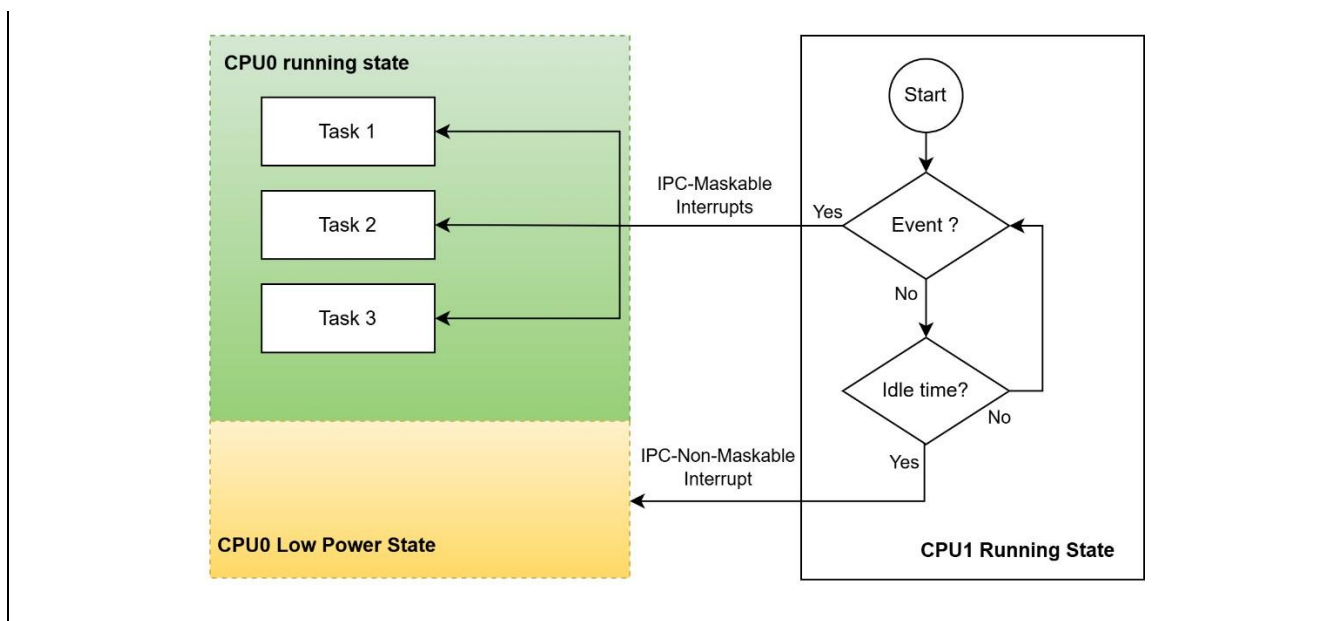


图 17. CPU0 使用 IPC 中断进行通知的示例

为避免在共享内存或状态标志时出现竞争条件（Race Condition），每个中断通知通常与一个明确定义的通信协议配合使用。这可能包括写入预留的共享内存结构、更新状态标志或通过 IPC 消息队列发送数据。接收方内核在处理中断信号后，会从共享区域中读取数据或服务事件。

当与信号量（Semaphore）、消息队列和共享内存缓冲区结合使用时，核间中断构成了实时或图形密集型双核系统中可靠核间协调的基础。它们确保确定性的唤醒、最小化延迟，并通过避免基于总线的轮询来减少系统总线争用。

关于在应用中实现核间中断的详细信息，请参考第 5.1.1 节。

### 4.2.2 使用 IPC 消息 FIFO

IPC 模块提供四个硬件 FIFO，可实现两个 CPU 内核之间高效且确定性的消息传递。每个 FIFO 是单向的，允许软件为不同的消息类型或优先级级别建立独立的通信通道。

- **IPC00 和 IPC01：**用于从 CPU1 → CPU0 发送消息的 FIFO。
- **IPC10 和 IPC11：**用于从 CPU0 → CPU1 发送消息的 FIFO。

每个 FIFO 包含四级深度并支持 32 位数据宽度，能够快速传输小型命令结构、标志或消息令牌，而无需涉及共享内存或复杂的同步操作。由于 FIFO 是在硬件中实现的，数据传输以最小延迟进行，并且无需软件仲裁，这使得 IPC 消息 FIFO 成为低开销、确定性核间通信的理想选择。

此机制非常适用于：

- 命令和事件通知。
- 轻量级同步。
- 发送小型数据令牌。
- 触发对等 CPU 上的状态更改或调度任务。

图 18 展示了使用 IPC 消息 FIFO 的数据交换机制。

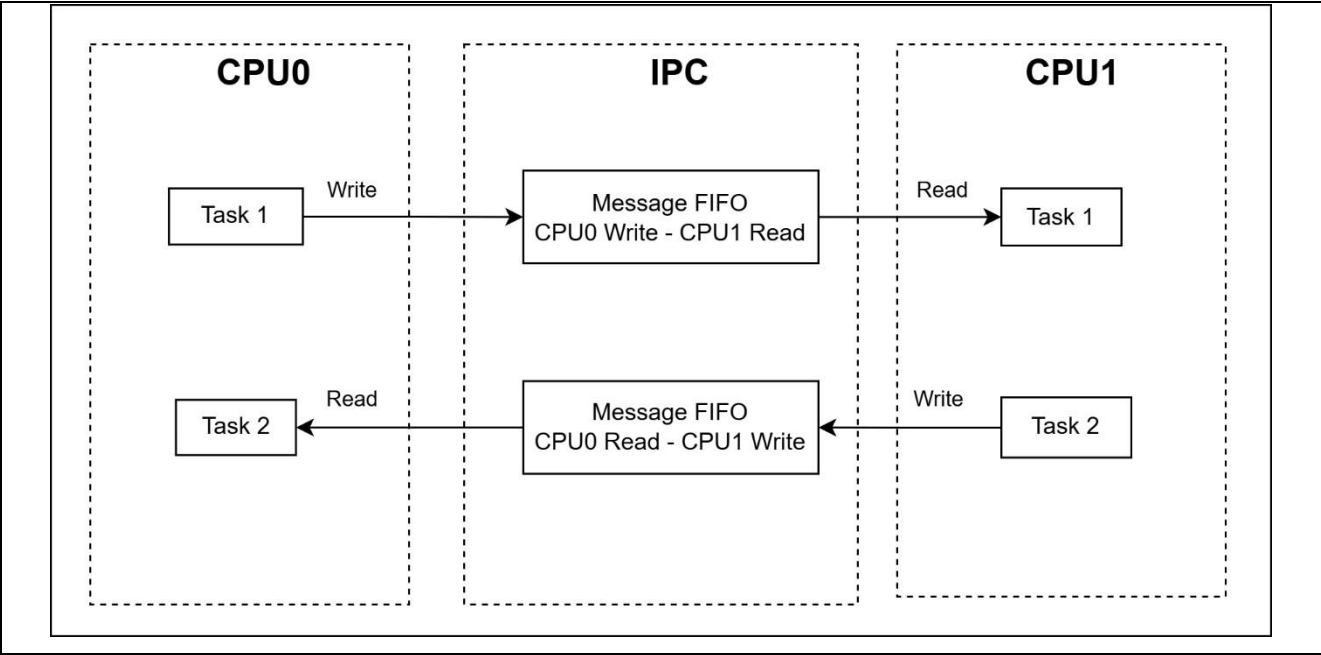


图 18. 使用 IPC 消息 FIFO 进行数据交换的示例

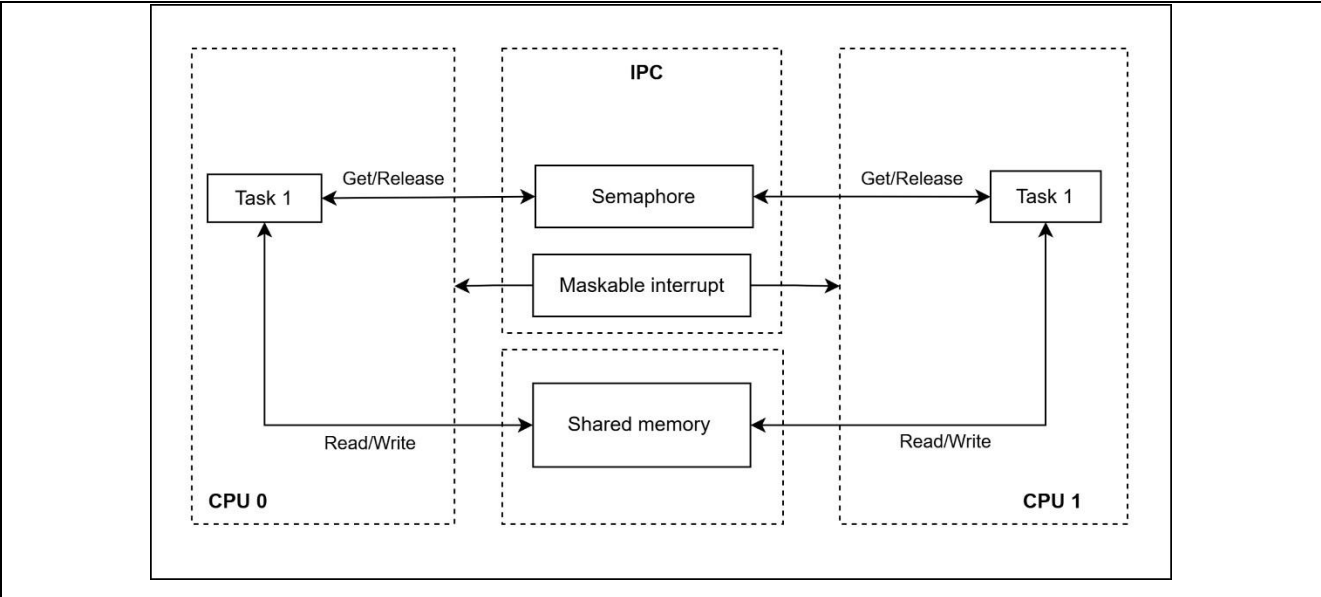
有关更高级的共享内存通信或大数据交换模式，请参考第 5.1 节，其中 IPC FIFO 与共享内存结构和 RTOS 队列结合使用，用于核间数据交换。

### 4.3 在 RA8 双核 MCU 中使用共享内存和共享资源

共享内存是双核系统中两个内核之间交换大量数据最高效的方法之一。该机制允许两个内核直接访问一个公共内存区域来读/写数据，但需要进行并发控制以避免竞争条件和数据损坏。

#### 4.3.1 在 FSP Flat 项目中使用共享内存和共享资源

当两个内核需要交换高速、大量数据（例如视频流）时，共享内存是理想的选择，因为其速度高于其他 IPC 方法。然而，需要适当的同步机制来避免竞争条件。图 19 展示了一种在双核裸机系统中使用事件标志和中断进行数据交换和同步的方法。



**图 19. 共享内存数据交换和同步示例**

在双核系统中实现共享内存时，必须考虑几个关键因素。

- 内存分配：应预留一个专用内存区域，并将其配置为两个内核均可访问的共享内存。
- 同步机制：需要适当的同步方法，如互斥锁（**Mutex**）、信号量（**Semaphore**）或硬件标志，以防止数据损坏并确保正确的访问顺序。
- 数据交换：一个内核将数据写入共享内存区域，而另一个内核根据需要读取数据，从而实现内核之间的协调通信。

图 20 展示了当两个 CPU 执行信号量并通过核间可屏蔽中断发送内核通知时发生的独占控制流程。

关于在应用中实现共享内存的说明，请参考第 5.1 节。

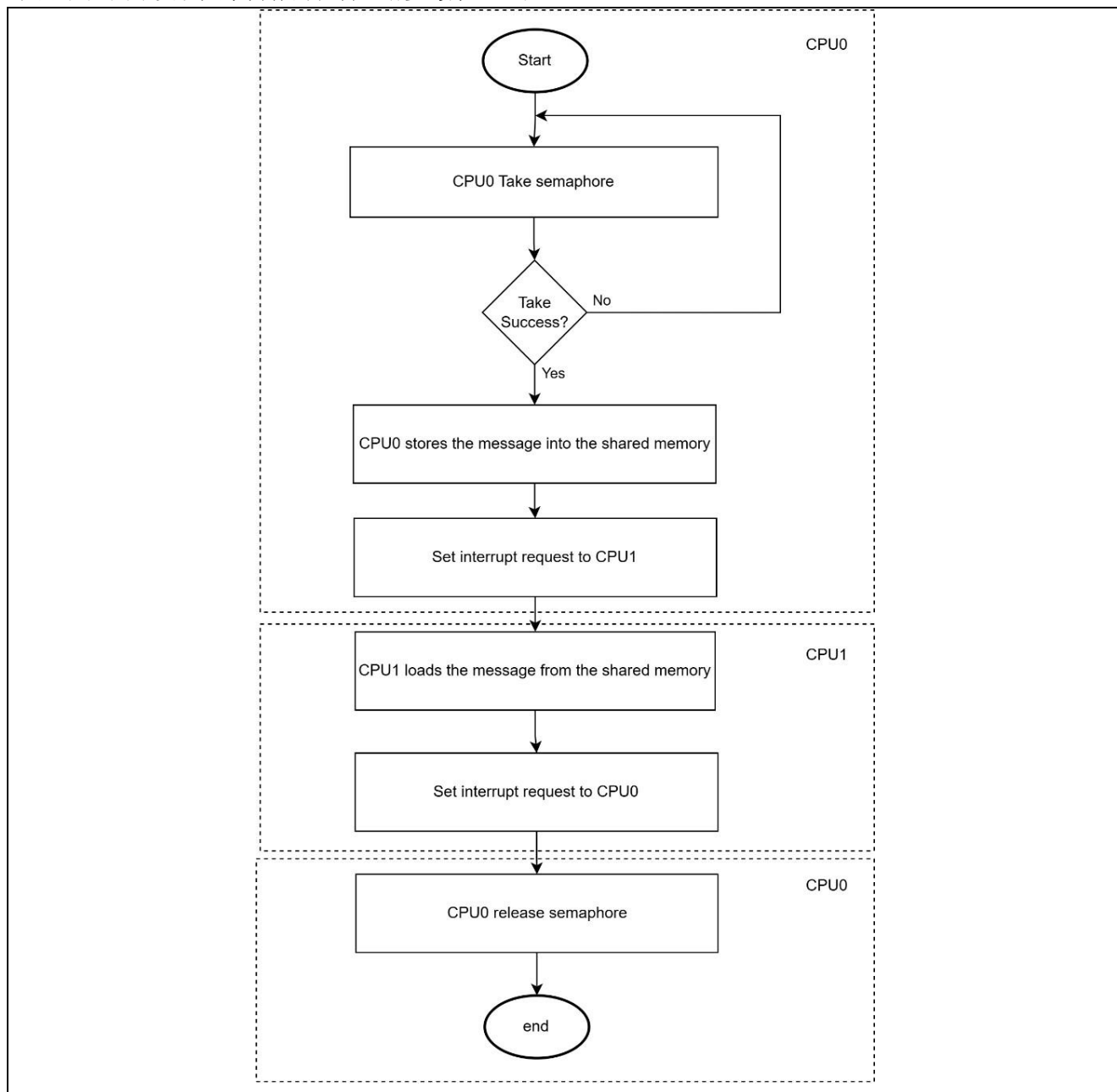


图 20. 应用独占控制流程示例

在使用双核系统时，将某些服务专用于单个内核可以优化性能并避免资源重复。此策略确保每个内核可以专注于其指定的任务而不受干扰，从而提高效率。通过有效地管理工作负载，双核架构可以支持更多具有不同需求的并行任务。

某些类型的服务在两个内核之间共享，但仅在一个内核上处理，例如外设管理（UART/SPI/I2C）、文件系统管理和网络栈管理。图 21 展示了共享 UART 外设，该外设由 CPU1 管理，而 CPU0 通过消息通道访问 UART 服务，而不是直接控制外设。

此架构也在随附的示例应用中得到了演示。

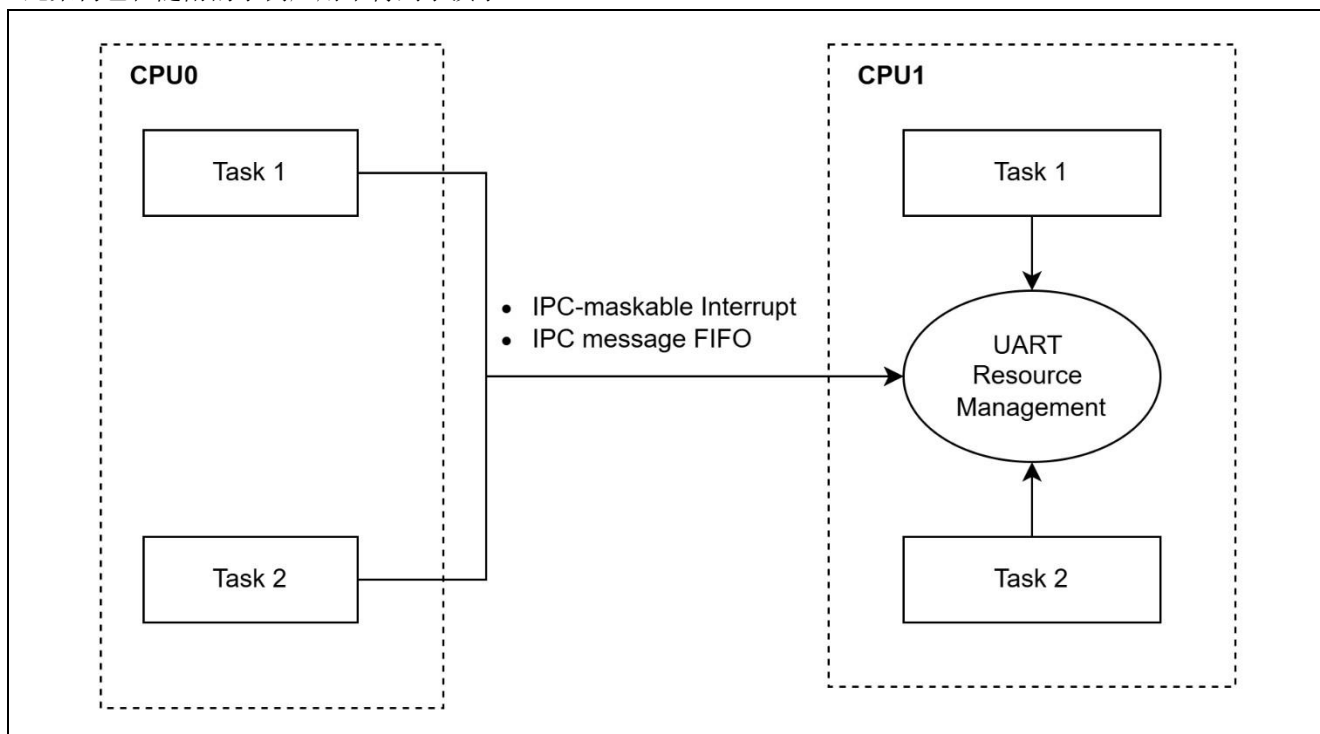


图 21. 共享资源 - UART 示意图示例

#### 4.3.2 在基于 RTOS 的项目中使用共享内存和共享资源

在基于 FreeRTOS 的多核应用中，共享内存和共享资源的使用在概念上与裸机实现相同。当两个 CPU 访问公共内存区域或外设时，用户仍需应用适当的技术来避免竞争条件和总线争用。

然而，FreeRTOS 提供了额外的同步机制，如信号量、互斥锁、队列、流缓冲区和消息缓冲区，简化了复杂多任务、多核应用的开发。在本项目中，在瑞萨 FSP IPC 模块之上构建了一个轻量级消息队列封装，用于高效管理两个内核之间的数据和控制事务。

有关 FreeRTOS 下共享内存使用和 IPC 机制的更多示例，请参考第 5.2 节。

#### 4.4 在 RA8 双核应用中使用缓存和 TCM

为了获得最佳性能，可以结合 Helium™ 技术使用紧耦合内存（TCM）和缓存。

TCM 通常提供确定性的、单周期的访问，最大限度地减少时间关键操作的延迟。将性能关键的代码和频繁访问的数据放置在 TCM 中可确保更快、更可预测的执行。

##### 4.4.1 紧耦合内存（TCM）

在 RA8 双核 MCU 中，每个 CPU 都有其自己专用的 TCM 资源。

- CPU0: 指令紧耦合内存（ITCM）；数据紧耦合内存（DTCM）。
- CPU1: C-AHB 紧耦合内存（CTCM）；S-AHB 紧耦合内存（STCM）。

CPU0 中的 256 KB TCM 内存由 128 KB ITCM 和 128 KB DTTCM 组成。

CPU1 中的 128 KB TCM 内存由 64 KB CTCM 和 64 KB STCM 组成。

注意：在 CPU 深度睡眠模式、软件待机模式和深度软件待机模式下，无法访问 TCM。

Figure 22 shows TCM in the local MCU subsystem.

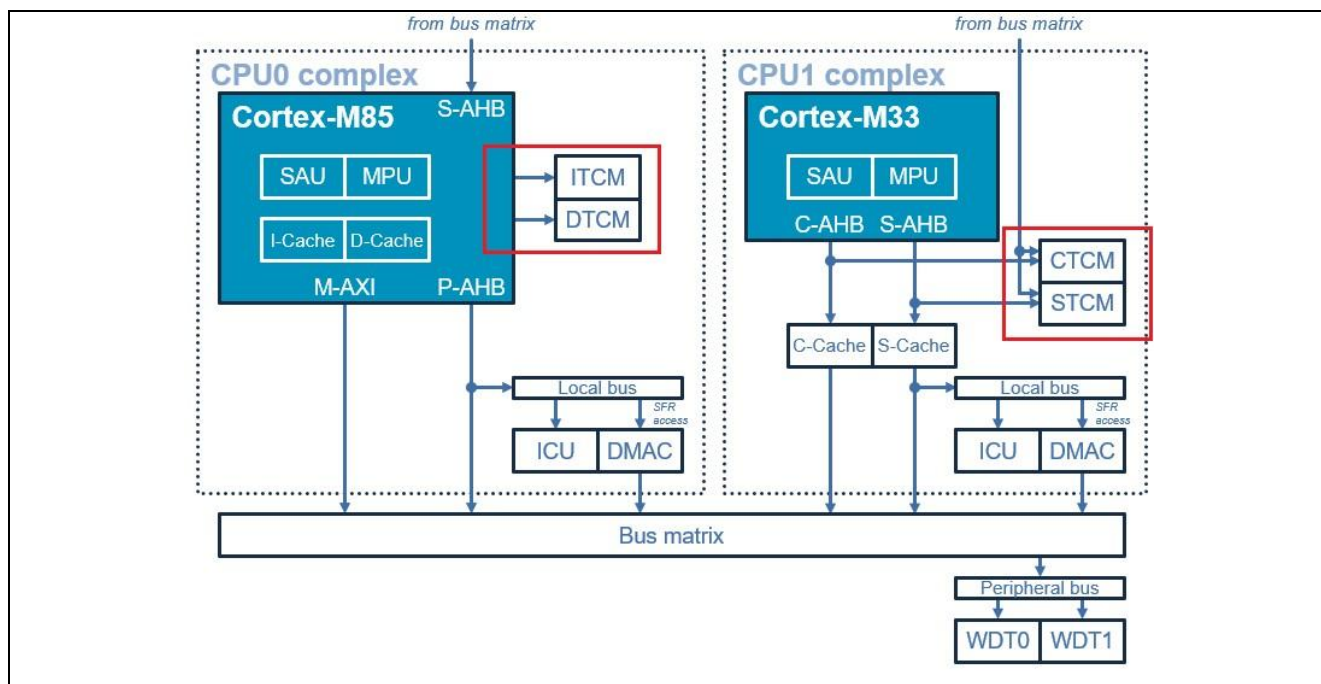


图 22. 本地 MCU 子系统系统中的 TCM 内存示例

#### 4.4.1 使用 ITCM 提升性能

为了为时间关键函数获得最佳性能，可以将特定的程序指令放置在指令紧耦合内存（ITCM）中。此配置可以通过 e<sup>2</sup> studio 中 C/C++ 项目 FSP 配置器（FSP Configurator）的 Linker Sections 设置进行调整。

以下示例说明了如何使用 Linker Sections Configuration 界面将 arm\_cmplx\_mag\_f32 函数分配到 RA8P1\_DSP\_example 项目的 ITCM 内存中。

1. 在 e<sup>2</sup> studio 中打开项目。
2. 在 Project Explorer 中，双击 configuration.xml。
3. 导航到 Linker Sections 选项卡（参见图 23）。
4. 将函数 arm\_cmplx\_mag\_f32 分配到指定的 ITCM 段（图 24 至图 26）。

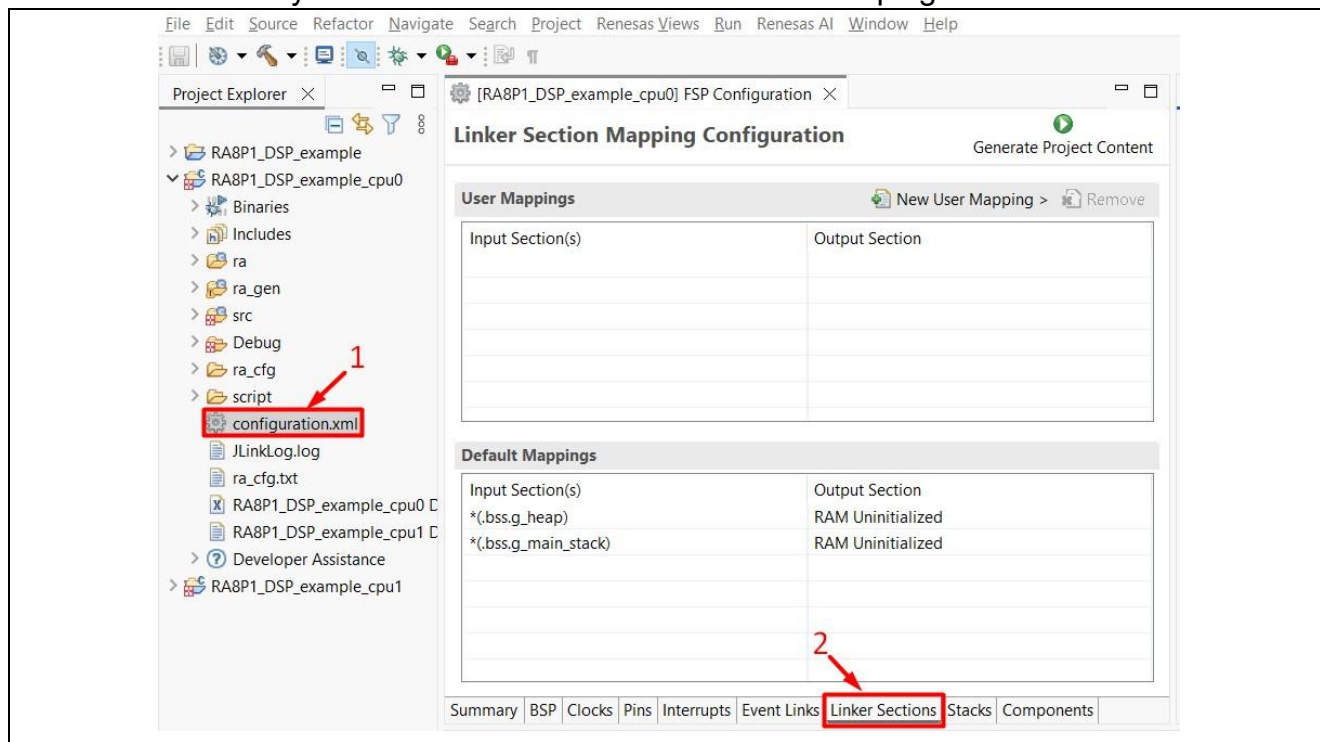


图 23. 打开 Linker Sections 选项卡

点击 New User Mapping > ITCM > Code initialized from > ITCM code from FLASH.

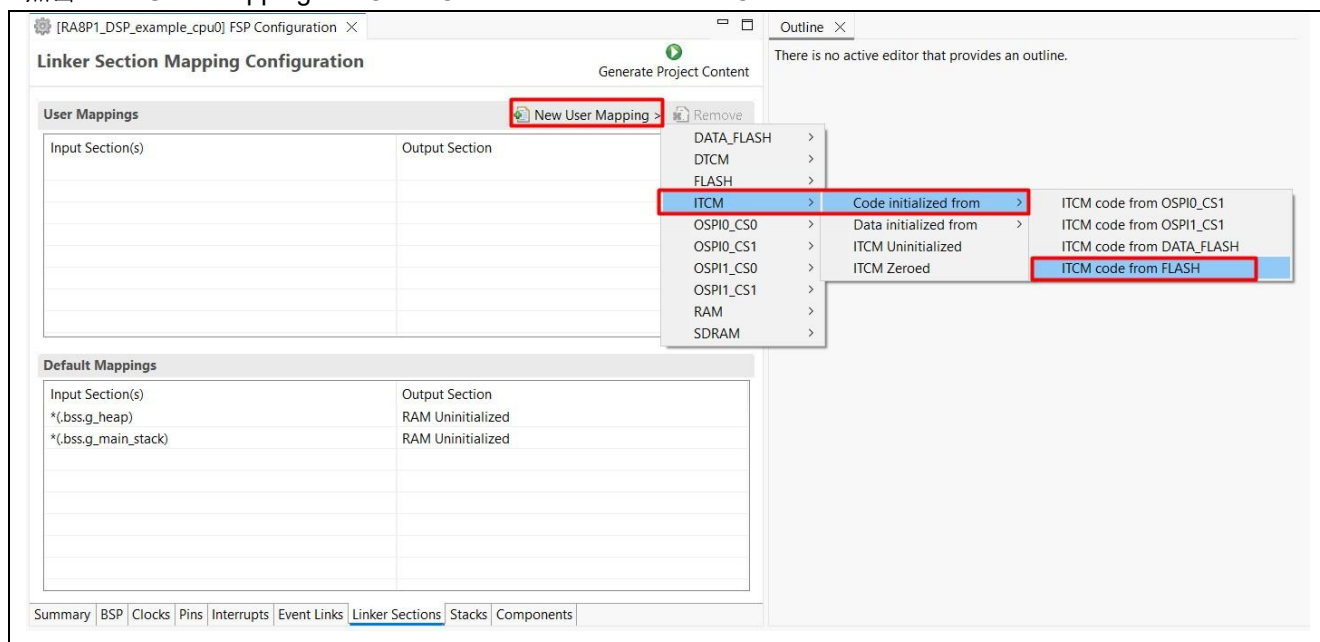


图 24. 为 ITCM 定义新段映射的示例

将出现一个新的用户映射窗口。输入段名称必须遵循特定的格式。对于包含指令代码的段，请确保输入段名称遵循所需的命名约定：`.text.<函数名称>`。图 25 展示了一个将 `arm_cmplx_mag_f32` 函数分配到 `.text.arm_cmplx_mag_f32` 的示例。

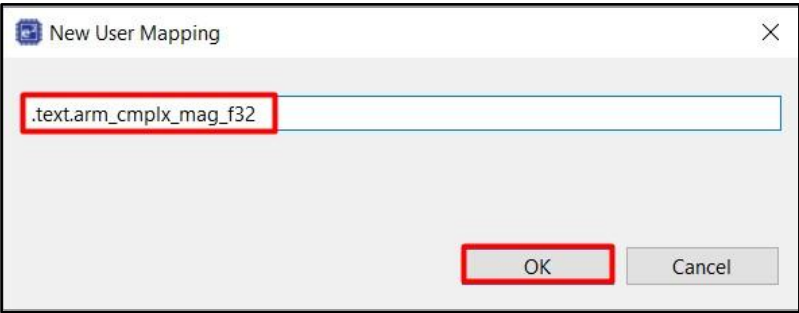


图 25. 为指令代码定义输入段名称的示例

完成图 26 所示的配置后，按照以下步骤应用更改：点击"Generate Project Content"，然后选择"Build Project"来生成代码并编译更新后的项目配置。

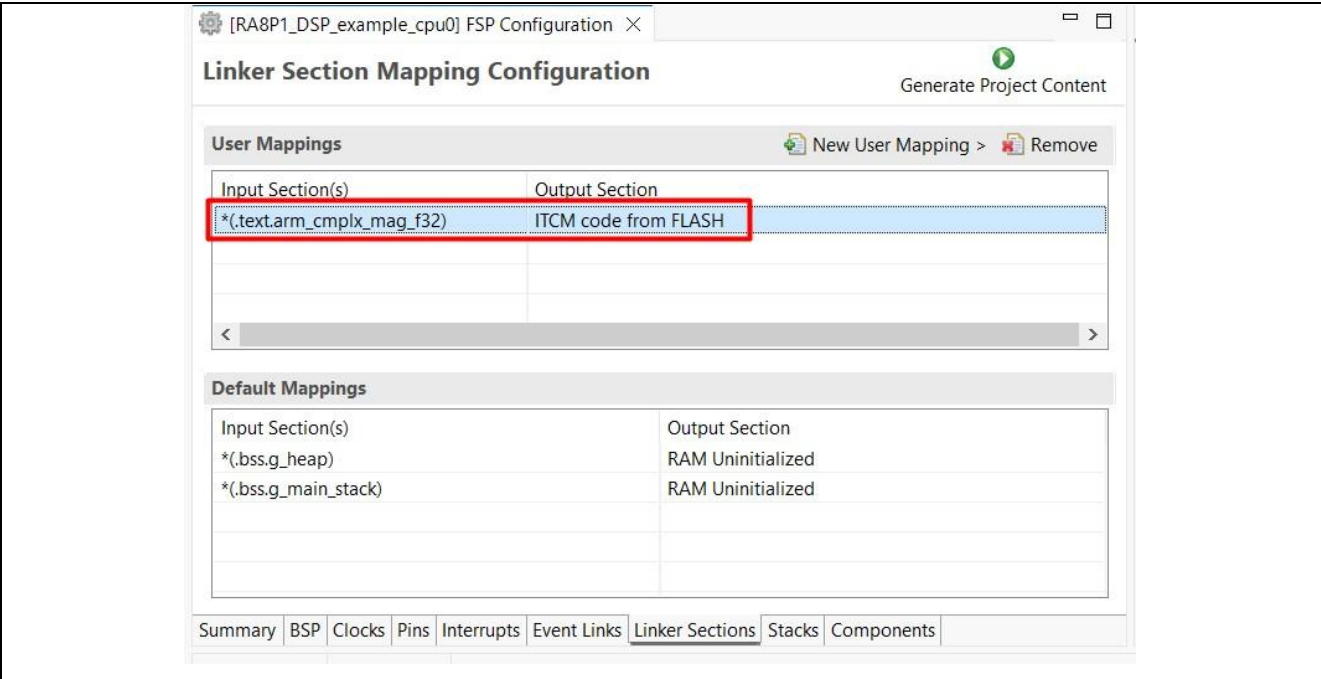


图 26. ITCM 段中函数设置成功示例。

成功编译后，可以通过检查位于 Debug/\*.map 的映射文件来验证代码或数据在指定内存段中的放置情况。该文件提供了详细的内存布局，包括段分配。参考图 27 的示例，它展示了内容映射到 ITCM 段的正确情况。

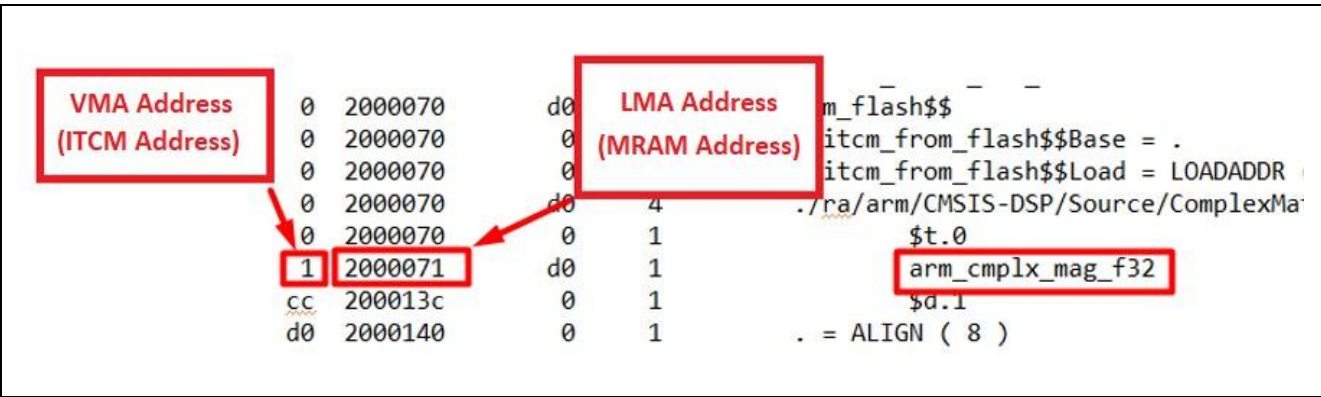


图 27. 验证代码放置在 ITCM 中的示例

#### 4.4.2 使用 DTCM 提升性能

为了提高运行性能，可以将数据缓冲区分配在数据紧耦合内存（DTCM）区域中。这些缓冲区的管理可以通过 e² studio 项目配置中的 Linker Sections 选项卡来执行。

以下流程演示了如何通过 Linker Sections Configuration 界面将 RA8P1\_DSP\_example 项目中使用的 testInput\_f32\_10khz 缓冲区（如图 28 所示）定位到 DTCM 内存中：

点击 Linker Sections 选项卡 > New User Mapping > DTCM > Data initialized from > DTCM data from FLASH，如图 29 所示。

此配置确保缓冲区在启动时从 MRAM 复制到 DTCM，并在执行期间实现更快的数据访问。

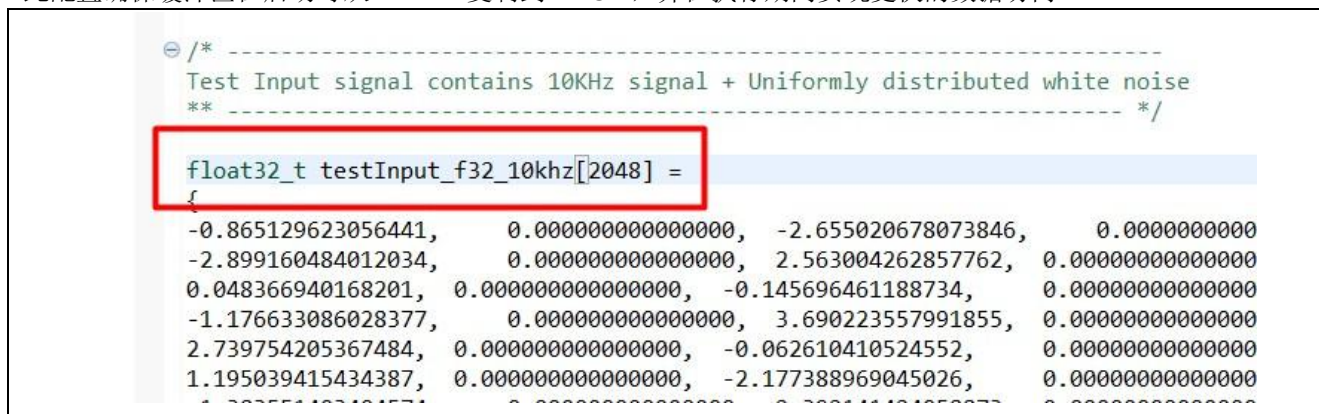


Figure 28. Example of Placing testInput\_f32\_10khz Buffer to DTCM

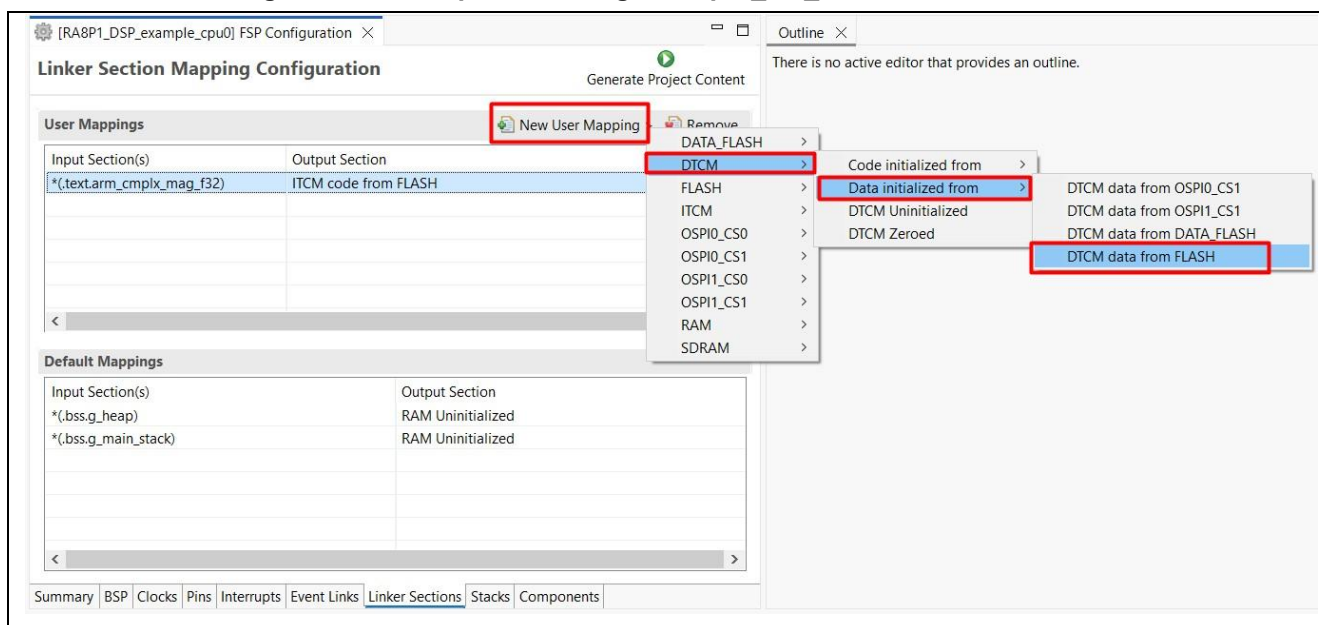


图 29. 为 DTCM 定义新段映射的示例

接下来，分配与初始化数据对应的 "Input section name"。段命名请使用以下格式：.data.<缓冲区名称>

例如，要映射初始化缓冲区 testInput\_f32\_10khz，请将输入段指定为：.data.testInput\_f32\_10khz。

参考图 30 查看此配置的图示示例。

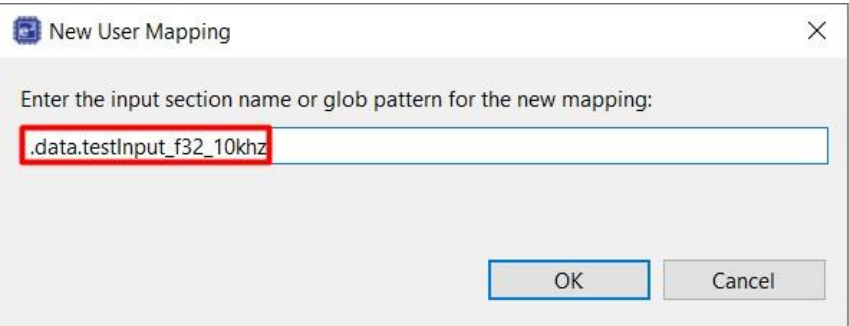


图 30. 为初始化数据定义输入段名称的示例

完成图 31 所示的配置后，点击 **Generate Project Content**，然后选择 **Build Project** 来应用更改并编译更新后的项目。

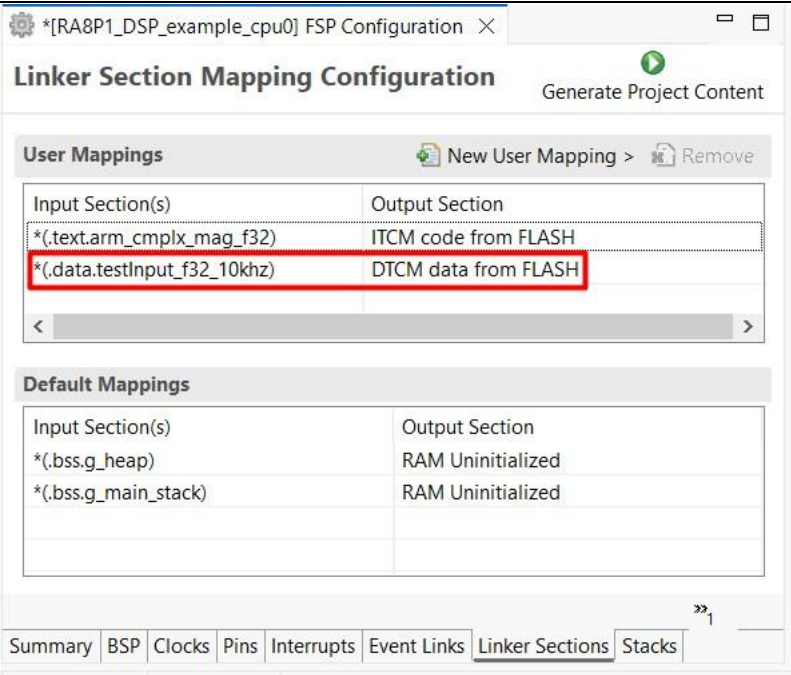


图 31. DTCM 段中数据设置成功

成功编译后，可以通过检查位于 `Debug/*.map` 的映射文件来验证代码或数据在指定内存段中的放置情况。该文件提供了详细的内存布局，包括段分配。参考图 32 的示例，它展示了内容映射到 DTCM 段的正确情况。

| VMA address<br>(DTCM address) | LMA address<br>(MRAM address) |      |   |                                                       |
|-------------------------------|-------------------------------|------|---|-------------------------------------------------------|
| 22000000                      | 2000140                       |      |   | am_from_flash\$\$Base = .                             |
| 22000000                      | 2000140                       | 2000 | 4 | am_from_flash\$\$Load = LOADADDR ( __ram_from_flash\$ |
| 22000000                      | 2000140                       | 2000 | 1 | ./src/arm_fft_bin_data.o:(.data.testInput_f32_10khz)  |
| 22002000                      | 2002140                       | 4    | 4 | testInput_f32_10khz                                   |
|                               |                               |      |   | ./src/arm_fft_bin_example_f32.o:(.data.fftSize)       |

图 32. 验证测试输入数据放置在 DTCM 中的示例

#### 4.4.3 使用 CTCM 提升性能

为了在 CPU1 上为时间关键函数获得最佳性能，可以将特定的程序指令放置在 CTCM 中。此配置可以通过 e<sup>2</sup> studio 中 CPU1 项目的 `configuration.xml` 中提供的 **Linker Sections** 设置来执行。通过将选定的函数显式映射

到 CTCM 内存区域，代码将以单周期延迟执行，且没有缓存未命中的情况，非常适合在从核（CPU1）上运行的中断服务例程或周期性高优先级任务。

以下流程演示了如何使用 e<sup>2</sup> studio 中的 Linker Sections 配置界面将 ipc\_callback() 函数放置到 CPU1 的 CTCM 中。此方法确保回调例程高速执行并具有确定性的内存访问，从而在核间通信工作负载期间减少响应时间延迟。

1. 在 e<sup>2</sup> studio 中打开项目。
2. 在 Project Explorer 中，双击 configuration.xml。
3. 导航到 Linker Sections 选项卡。
4. 将函数 ipc\_callback() 分配到指定的 CTCM 段（图 33 和图 34）。

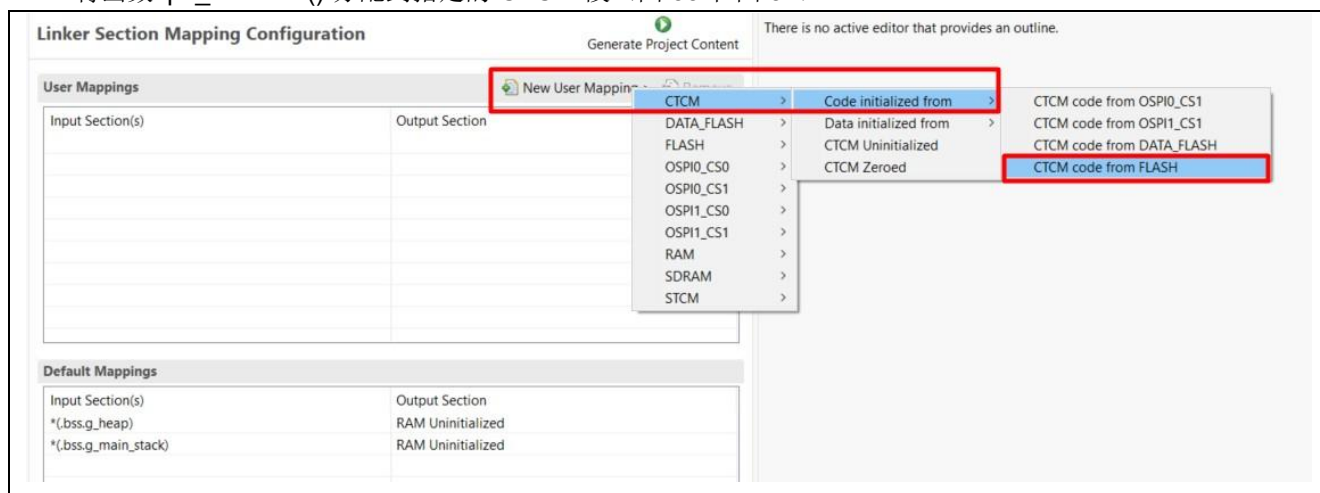


Figure 33. Example of Defining a New Section Mapping for CTCM

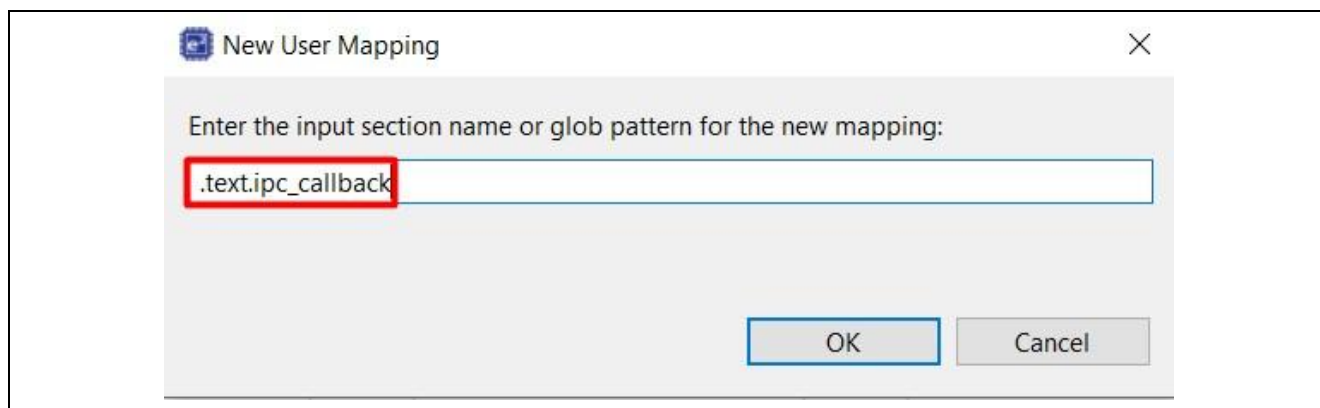


图 34. 为 CPU1 上的指令代码定义输入段名称的示例

完成配置后，点击 Generate Project Content，然后选择 Build Project 来生成代码并编译更新后的项目配置。成功编译后，您可以通过检查位于 Debug/\*.map 的映射文件来验证代码或数据在指定内存段中的放置情况。请确保 ipc\_callback 函数被放置在 CTCM 内存的地址范围内，如图 35 所示。

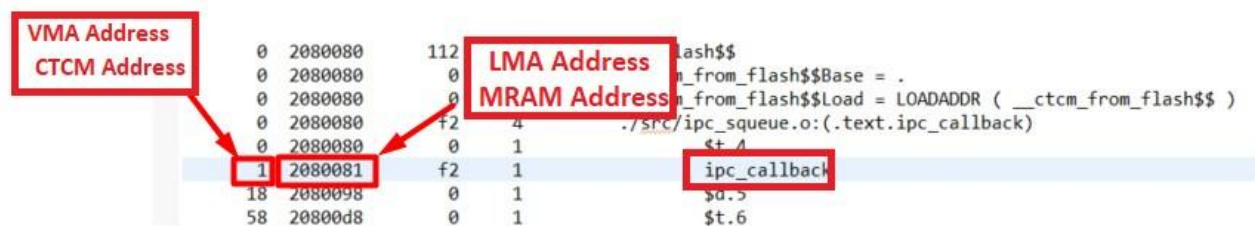


图 35. 成功将 ipc\_callback 分配到 CTCM 的示例

#### 4.4.4 利用 Cortex®-CM85 内核数据缓存提升性能

在 RA8 器件的默认配置中，FSP 始终启用 CM85 指令缓存（I-Cache）并根据需要管理其一致性。FSP 还允许在 BSP 配置设置中选择性启用 CM85 数据缓存（D-Cache），如图 36 所示，但默认情况下保持禁用状态。当应用程序从外部存储器（例如外部 Octal SPI RAM）访问数据时，启用 D-Cache 可以显著提升性能。

| EK-RA8P1 |                                 |             |
|----------|---------------------------------|-------------|
| Settings | Property                        | Value       |
|          | ▼ RA8P1 Family                  |             |
|          | > SDRAM                         |             |
|          | > OSPI_B                        |             |
|          | > Security                      |             |
|          | > Clocks                        |             |
|          | ▼ Cache settings                |             |
|          | Data cache                      | Enabled     |
|          | Data cache forced write-through | Enabled     |
|          | > I/O Ports                     |             |
|          | Enable inline BSP IRQ functions | Enabled     |
|          | Main Oscillator Wait Time       | 8163 cycles |

图 36. CM85 数据缓存（D-Cache）启用的示例

#### 4.4.5 使用神经网络处理单元（NPU）

RA8P1 双核 MCU 集成了 Ethos-U55，它支持边缘端的基于 Transformer 的模型（这是新语言和视觉模型的基础），并可从 32 扩展到 256 个 MAC 单元，以可持续的方式实现更高性能的边缘 AI 用例。Ethos-U55 提供与先前 Ethos-U 处理器相同的工具链，并支持 AI 框架（如 TensorFlow 和 PyTorch）的广泛选择。

通过将 ML 计算从 CPU 卸载到 Ethos NPU，系统实现了更高的性能和能效，从而在边缘端实现 AI 而不会产生显著的功耗开销。图 37 和图 38 分别是 NPU 框图和在瑞萨 FSP 中配置 Ethos 的示例。

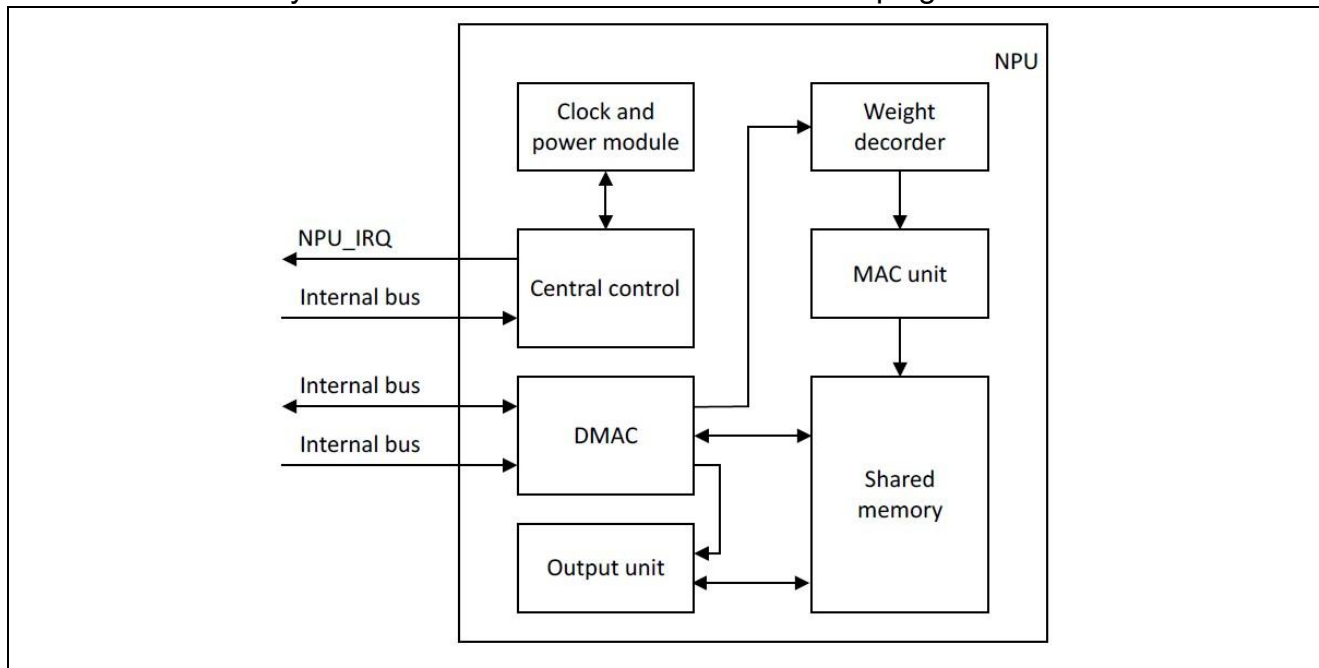


Figure 37. NPU Block Diagram

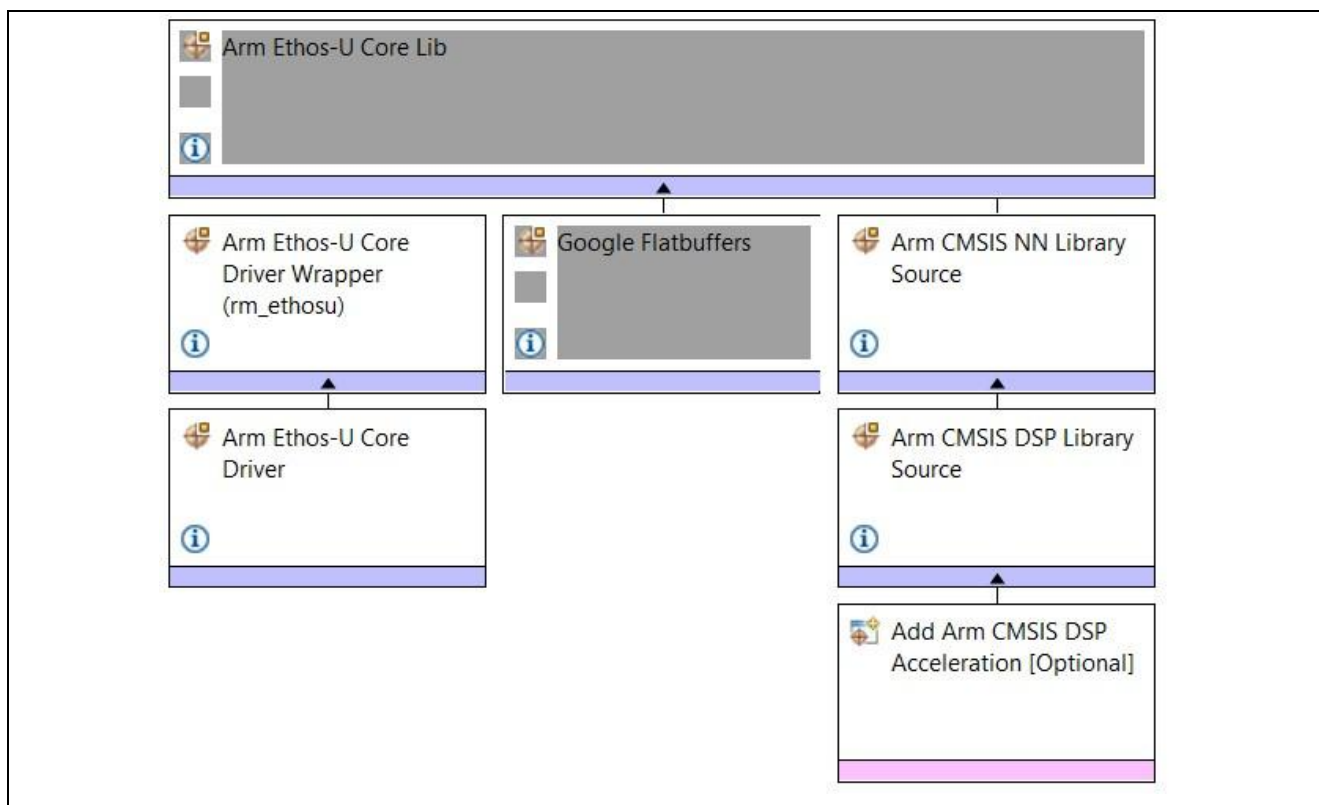


图 38. 瑞萨 FSP 中 Ethos 支持的示例

关于在 RA8 双核 MCU 上开发 AI 应用，请参考以下应用笔记：“Reference System Design for Vision AI Design using Ethos-U NPU”（文档编号 R11AN0995）和“Using the Ethos-U NPU with RA8 MCUs”（文档编号 R01AN7712）。

5. 应用项目

5.1 IPC - 共享内存项目

本应用项目的实现遵循以下规范。

CPU1 负责管理用户界面并将数据记录到终端。CPU0 将处理实时控制和传感器数据处理。

两个内核上的任务分配如图 39 所示。

本应用项目中使用的硬件资源列于下方的表 1 中。

表 1. 项目中使用的资源列表

| CPU0                      | CPU1                      |
|---------------------------|---------------------------|
| 核间通信（IPC0 和 IPC1）         | 核间通信（IPC0 和 IPC1）         |
| 硬件信号量（Hardware Semaphore） | 硬件信号量（Hardware Semaphore） |
| 实时时钟控制                    | 串行通信接口 UART               |
| 12 位 A/D 转换器 - 温度传感器      | I/O 端口控制                  |
| I/O 端口控制 - 用户 LED         |                           |

在此示例应用中，所有运行时信息通过 SCI-UART 接口传输到 Tera Term 终端控制台，该接口由 CPU1 管理。

同时，CPU0 负责处理实时时钟（RTC）、传感器数据采集和用户 LED 控制。

Figure 40 shows the functionality of this application.

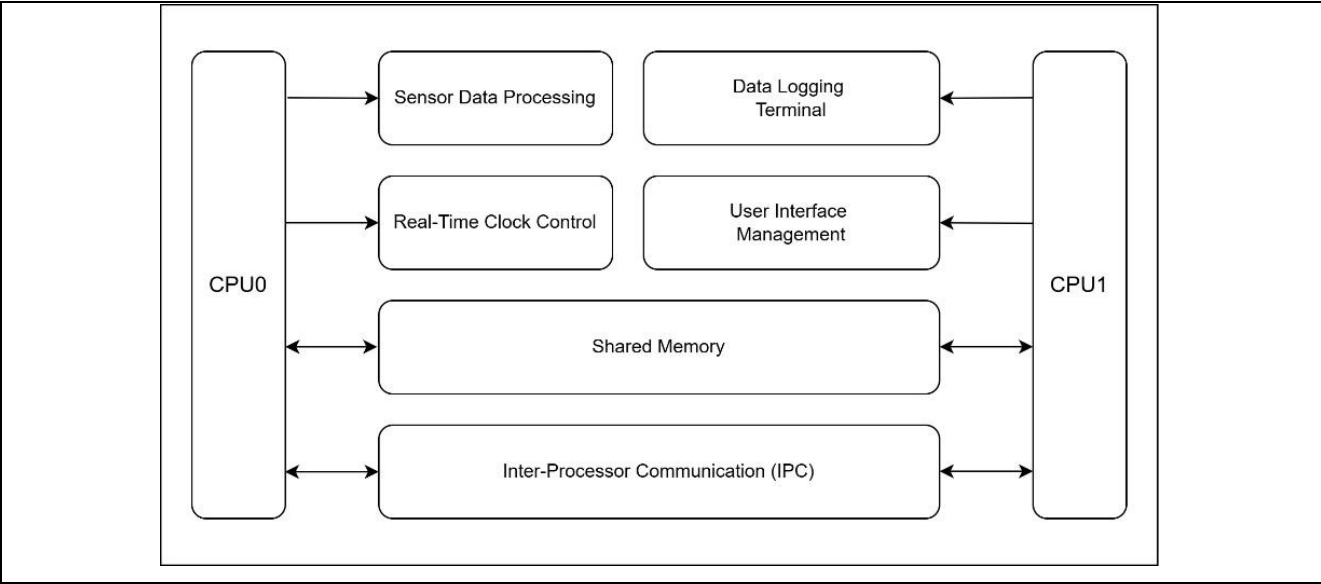


图 39. 双核裸机示例上的任务划分

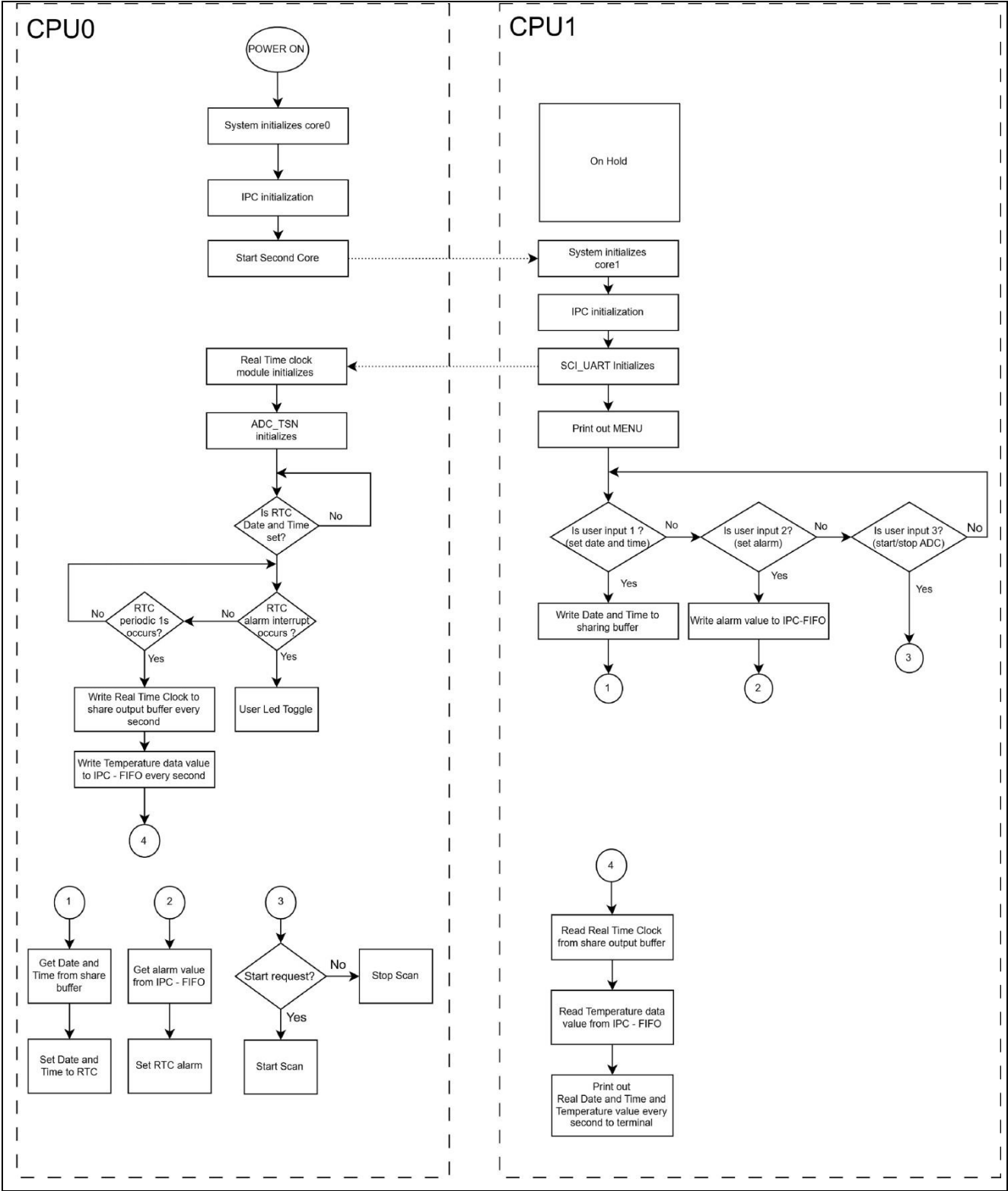


图 40. 使用 IPC/信号量和共享内存功能的应用。

表 2 总结了本应用中使用的 IPC 和硬件信号量 API。这些 API 有助于双核系统中处理器之间的核间通信和同步。

表 2. 核间通信 API

| 函数                     | 描述                 |
|------------------------|--------------------|
| R_IPC_Open             | 配置 IPC 实例          |
| R_BSP_IpcNmiEnable     | 为 IPC NMI 分配用户回调函数 |
| R_IPC_MessageSend      | 向 IPC 消息 FIFO 发送消息 |
| R_IPC_EventGenerate    | 向另一个内核生成 IPC 可屏蔽中断 |
| R_BSP_IpcSemaphoreTake | 获取硬件信号量            |
| R_BSP_IpcSemaphoreGive | 释放硬件信号量            |
| R_BSP_IpcNmiRequestSet | 向另一个内核触发不可屏蔽中断     |

### 5.1.1 在应用中实现核间通信。

核间通信（IPC）有助于同一 CPU 系统内的两个处理器之间进行硬件资源共享和数据交换。

IPC 模块最多支持 16 个硬件信号量，能够实现内核之间的高效同步。

此外，IPC 能够生成中断驱动事件以支持核间信号传递，包括可屏蔽中断和不可屏蔽中断（NMI）。

图 41 和图 42 展示了在应用中实现 IPC 可屏蔽中断和 IPC 不可屏蔽中断的示例。

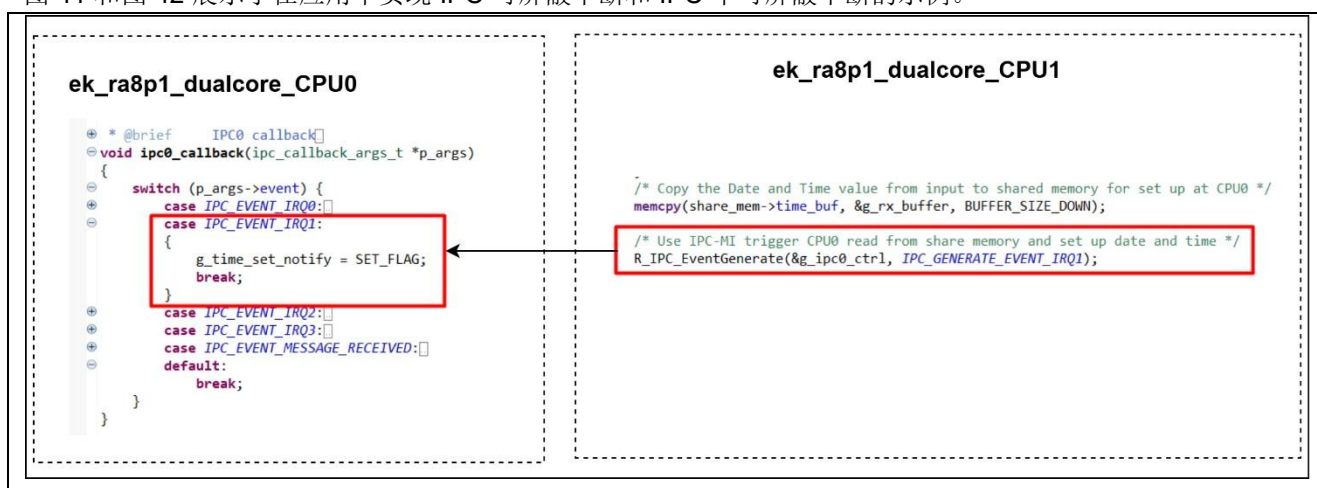


Figure 41. Example of IPC Maskable Interrupt Sample Application

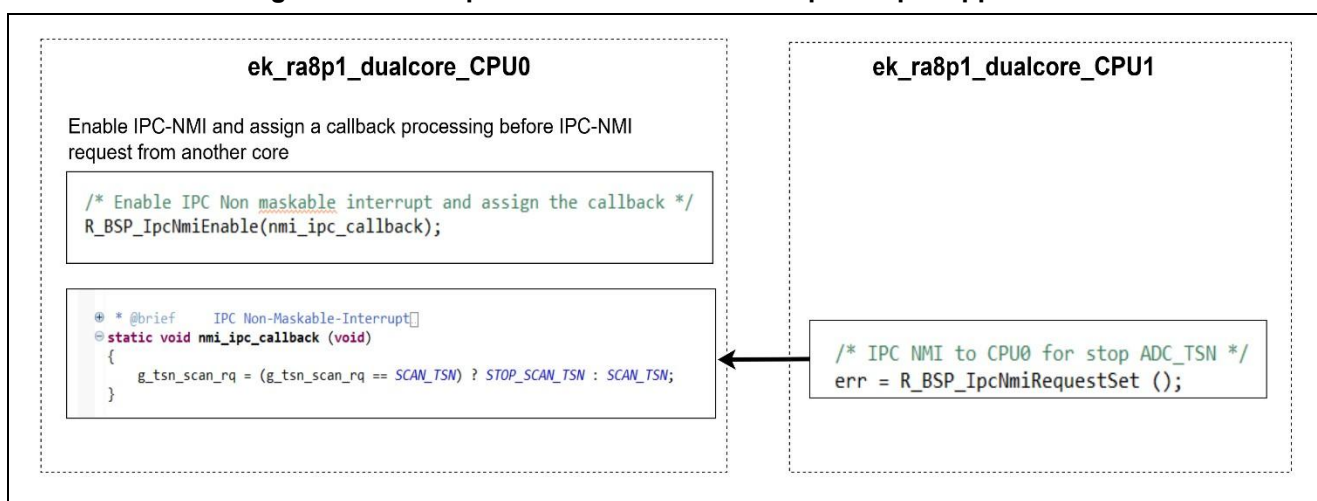


图 42. IPC 不可屏蔽中断示例应用。

在此应用中，单向 FIFO 也用于在两个 CPU 之间交换简单数据。具体来说，温度数据从 CPU0 发送到 CPU1，而用户闹钟设置值从 CPU1 发送到 CPU0，如图 43 所示。

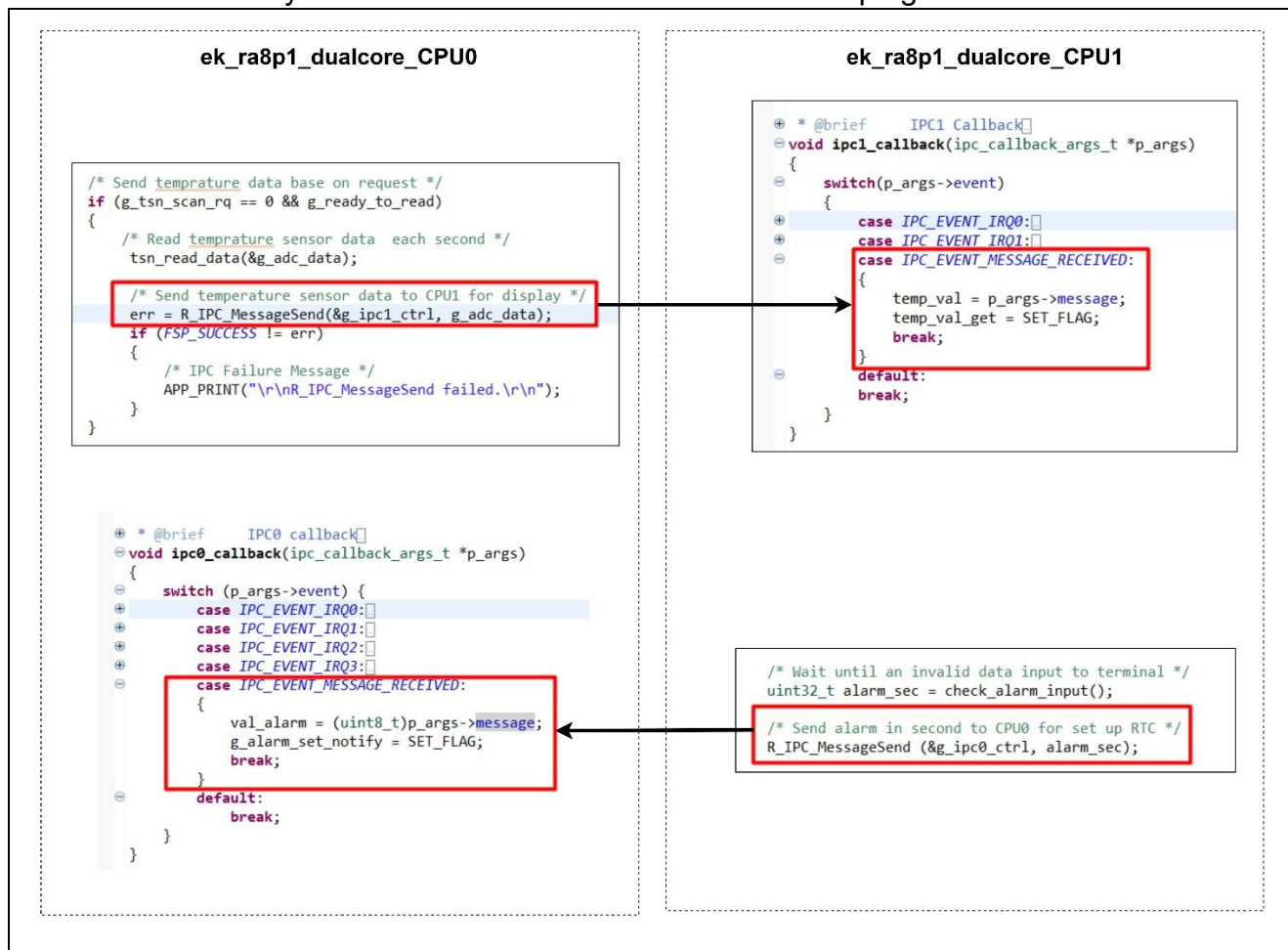


图 43. IPC 消息 FIFO 示例应用

### 5.1.2 在应用中实现两个内核之间的共享内存。

为了实现两个内核之间的共享内存机制，必须在 Solution 项目中显式定义一个专用内存区域。在此应用中，共享内存分配到 CPU0 RAM 的最后 32 KB。定义此内存区域的配置流程如下所述。

将 RAM\_CPU0\_S 的 RAM 大小更改为 0xE2000。

The screenshot shows the 'Memories' tab in the development tool. The 'RAM' partition is selected, and its size is being modified from 0x1D4000 to 0xE2000. A red arrow points to the 'Add Partition' button.

| Name          | Start      | Size       | Core | Security            |
|---------------|------------|------------|------|---------------------|
| RAM_NS        | 0x32000000 | 0x1D4000   |      | Non-secure          |
| RAM           | 0x22000000 | 0x1D4000   |      | Secure              |
| RAM_CPU0_S    | 0x22000000 | 0xEA000    | CPU0 | Secure              |
| RAM_CPU0_C    | 0x220EA000 | 0x0        | CPU0 | Non-secure Callable |
| RAM_CPU1_S    | 0x220EA000 | 0xEA000    | CPU1 | Secure              |
| RAM_CPU1_C    | 0x221D4000 | 0x0        | CPU1 | Non-secure Callable |
| FLASH_NS      | 0x12000000 | 0x100000   |      | Non-secure          |
| FLASH         | 0x02000000 | 0x100000   |      | Secure              |
| DATA_FLASH_NS | 0x37000000 | 0x0        |      | Non-secure          |
| DATA_FLASH    | 0x27000000 | 0x0        |      | Secure              |
| SDRAM         | 0x68000000 | 0x8000000  |      |                     |
| OSPI0_CS0     | 0x80000000 | 0x10000000 |      |                     |
| OSPI0_CS1     | 0x90000000 | 0x10000000 |      |                     |
| OSPI1_CS0     | 0x70000000 | 0x8000000  |      |                     |
| OSPI1_CS1     | 0x78000000 | 0x8000000  |      |                     |

图 44. RAM\_CPU0\_S 大小修改示例。点击 RAM &gt; Add

Partition。

The screenshot shows the 'Memories' tab in the development tool. The 'RAM' partition is selected, and the 'Add Partition' button is highlighted. A red arrow points to the 'Add Partition' button.

| Name                | Start      | Size       | Core | Security            |
|---------------------|------------|------------|------|---------------------|
| RAM_NS              | 0x32000000 | 0x1D4000   |      | Non-secure          |
| RAM                 | 0x22000000 | 0x1D4000   |      | Secure              |
| RAM_CPU0_S          | 0x22000000 | 0xE2000    | CPU0 | Secure              |
| RAM_CPU0_C          | 0x220EA000 | 0x0        | CPU0 | Non-secure Callable |
| RAM_CPU1_S          | 0x220EA000 | 0xEA000    | CPU1 | Secure              |
| RAM_CPU1_C          | 0x221D4000 | 0x0        | CPU1 | Non-secure Callable |
| FLASH_NS            | 0x12000000 | 0x100000   |      | Non-secure          |
| FLASH               | 0x02000000 | 0x100000   |      | Secure              |
| DATA_FLASH_NS       | 0x37000000 | 0x0        |      | Non-secure          |
| DATA_FLASH          | 0x27000000 | 0x0        |      | Secure              |
| SDRAM               | 0x68000000 | 0x8000000  |      |                     |
| OSPI0_CS0           | 0x80000000 | 0x10000000 |      |                     |
| OSPI0_CS1           | 0x90000000 | 0x10000000 |      |                     |
| OSPI1_CS0           | 0x70000000 | 0x8000000  |      |                     |
| OSPI1_CS1           | 0x78000000 | 0x8000000  |      |                     |
| OPTION_SETTING_OFS0 | 0x02C9F040 | 0x4        |      | Secure              |

图 45. 为共享内存区域添加分区的示例。在 New Partition 弹出窗口中，如

图 46 所示，填写所有必填字段并点击 OK。

对于 Multicore Flat Bare-Metal 项目，按以下方式配置字段：

- **Name:** SHARED\_MEM

- **Start:** 0x220E2000
- **Size:** 0x8000
- **Core:** CPU0
- **Security:** Secure

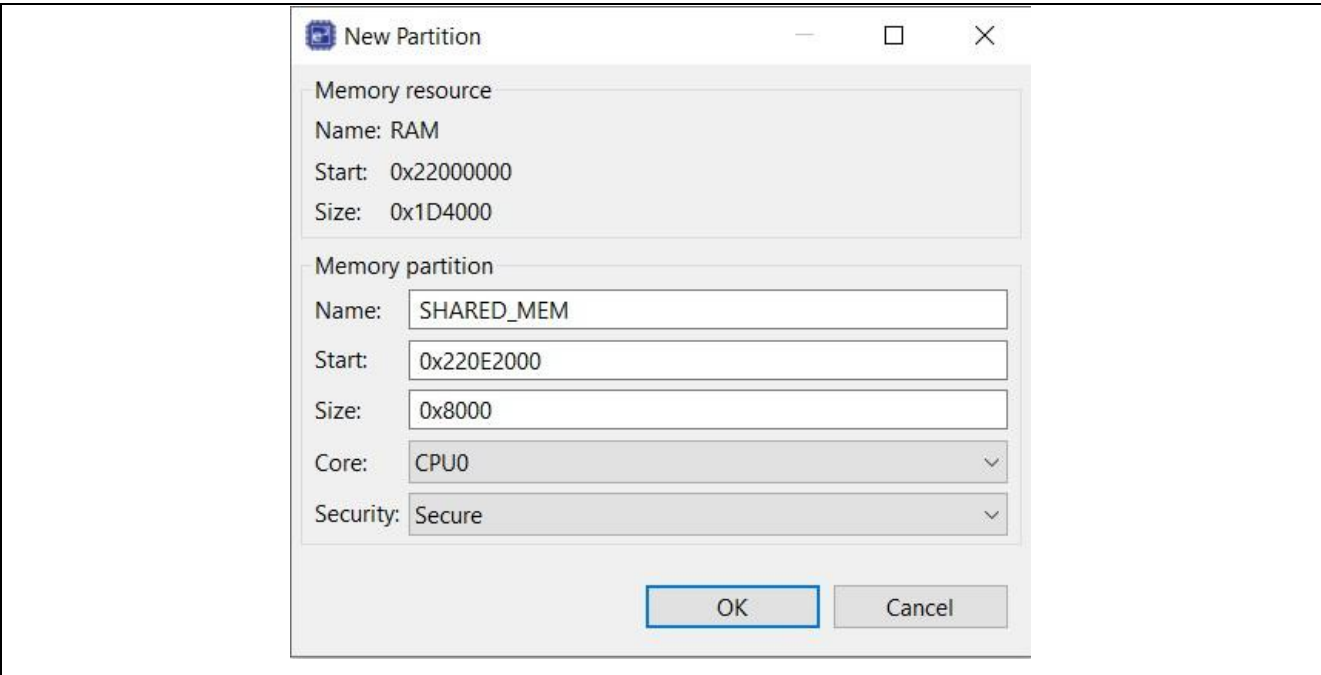


图 46. 定义共享内存分区的示例

如果出现错误消息，提示"Partition does not follow address order"，这意味着内存分区未按严格递增的地址顺序排列。

要解决此问题，请点击 **Sort Partitions** 重新正确排序，如图 47 所示。这确保了正确的内存映射，并防止编译过程中出现地址冲突。

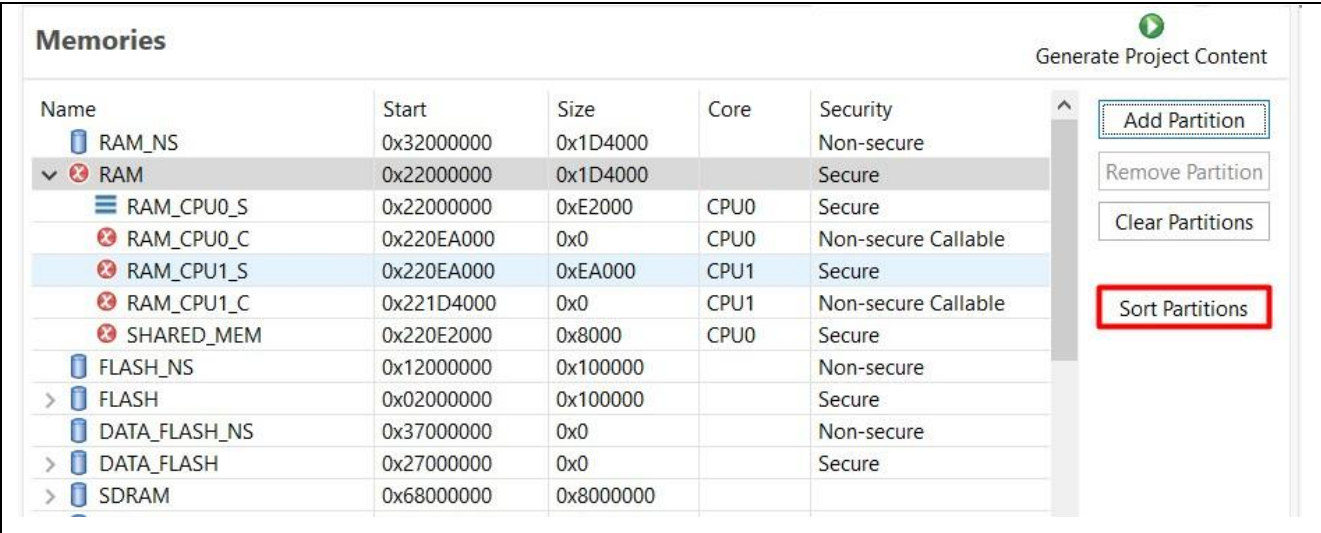


图 47. RAM 中分区排序的示例。成功设置的分区如下方的图

48 所示。

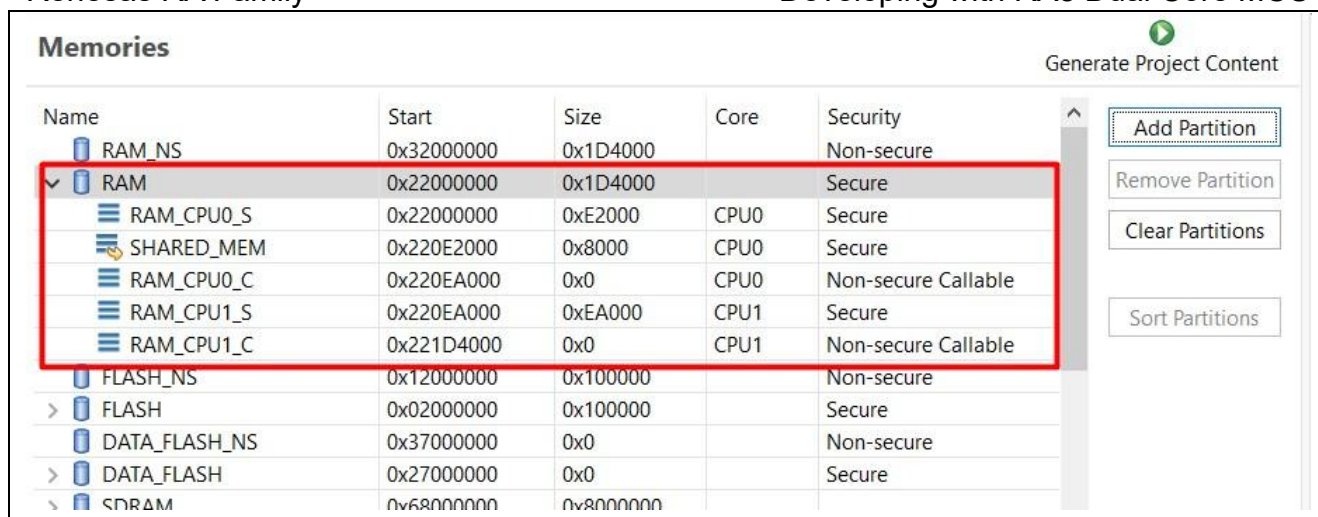


图 48. 成功设置共享内存分区的示例

完成配置后，右键单击 **Solution** 项目并选择 **Build Project** 以编译更改。

成功编译后，通过双击位于 `<solution_project>/build/*.sbd` 的 `.sbd` 文件来验证共享内存分区的创建。该文件确认了共享内存区域的正确分区。

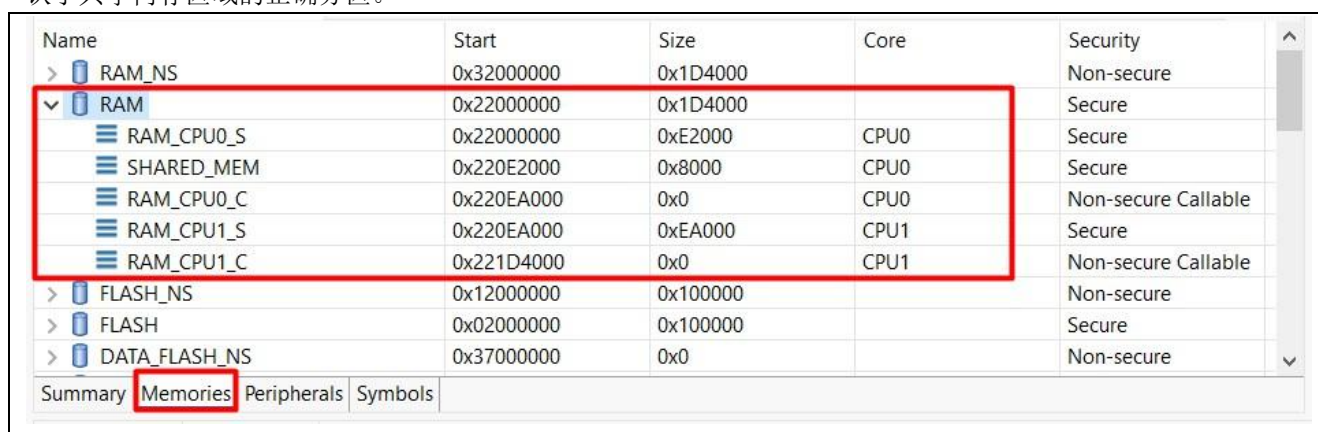


图 49. 成功创建共享内存分区的示例

此后，应用程序可以在共享内存区域内分配缓冲区。实现过程如图 50 所示。此配置保证缓冲区驻留在共享内存空间中，从而实现两个内核之间的无缝数据交换。



图 50. 将共享缓冲区放置在共享内存中的示例

共享内存管理的基本原则是保证在任何给定时间只有一个 CPU 进行独占访问。在此应用中，硬件信号量和核间可屏蔽中断（MI）一起使用，以协调和仲裁两个内核之间的访问。此机制允许每个 CPU 在拥有信号量时安全地访问共享缓冲区，从而降低数据损坏的风险。

## 5.2 RTOS/IPC/共享内存/TCM 项目

这些示例项目利用 RTOS、IPC 模块、IO 接口和 TCM 内存（启用数据缓存）来促进两个 CPU 内核之间的通信。

该应用改编自官方的 Arm CMSIS-DSP FFT 示例，在 CPU0（CM85 内核）上运行，计算任务在 TCM 内执行以最大化性能。FFT 结果通过基于片上 IPC 模块的 RTOS 队列传输到第二个内核。然后，第二个内核通过 UART 串口终端输出计算结果。

在 RA FSP 上移植 Arm 的 CMSIS-DSP 示例时，请参考应用笔记 R01AN5865 Arm® DSP Examples 获取详细的集成指南。

在此实现中，基于 FreeRTOS 的队列与硬件 IPC 消息 FIFO 集成，以管理两个内核之间的消息事务，如图 51 所示。

或者，根据用例和数据流需求，也可以使用 FreeRTOS 消息缓冲区（Message Buffers）或流缓冲区（Stream Buffers）进行核间通信。

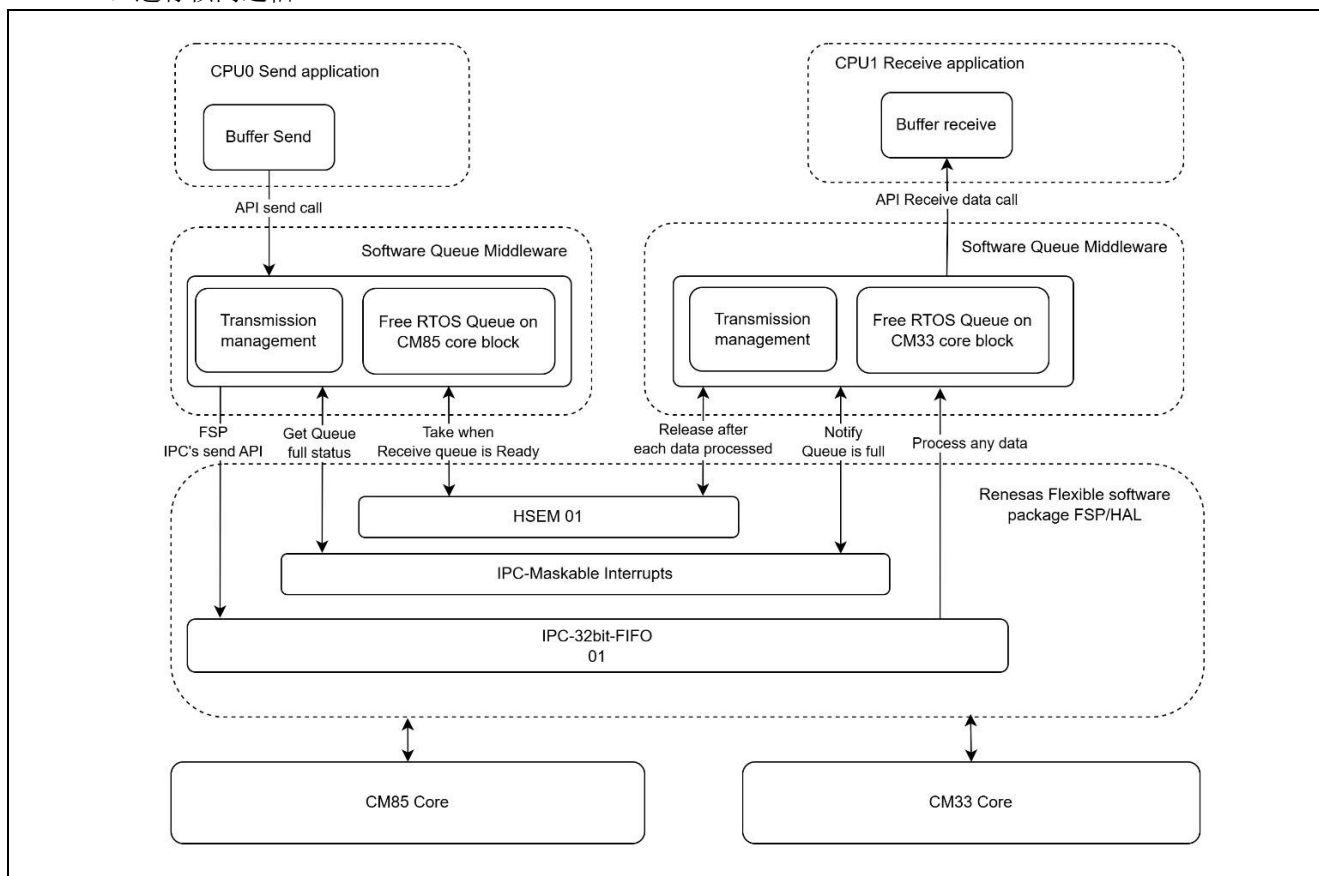


图 51. 基于 FreeRTOS 的通过 IPC 硬件的核间消息传递

## 6. 验证多核 Flat Bare-Metal 项目

e<sup>2</sup> studio 项目采用分离式项目开发模型来支持双核应用开发。每个内核项目作为独立的项目创建，利用核间通信（IPC）来实现两个内核之间的高效通信和共享内存管理。

要在 e<sup>2</sup> studio 中编译和运行示例应用项目 ek\_ra8p1\_dualcore，请按照以下流程操作。

### 6.1 导入项目

1. 启动 e<sup>2</sup> studio IDE。
2. 在 Workspace launcher 中选择任意工作空间。
3. 关闭 Welcome 窗口。

4. 选择 File > Import。
5. 从 Import 对话框中选择 Existing Projects into Workspace。
6. 选择名为"Developing\_with\_RA8\_Dual\_Core\_MCU.zip"的文件中的压缩文件"ek\_ra8p1\_dualcore.zip"。
7. 选择 Solution 项目和每个内核上的已开发项目示例，如下所示，点击 Finish。

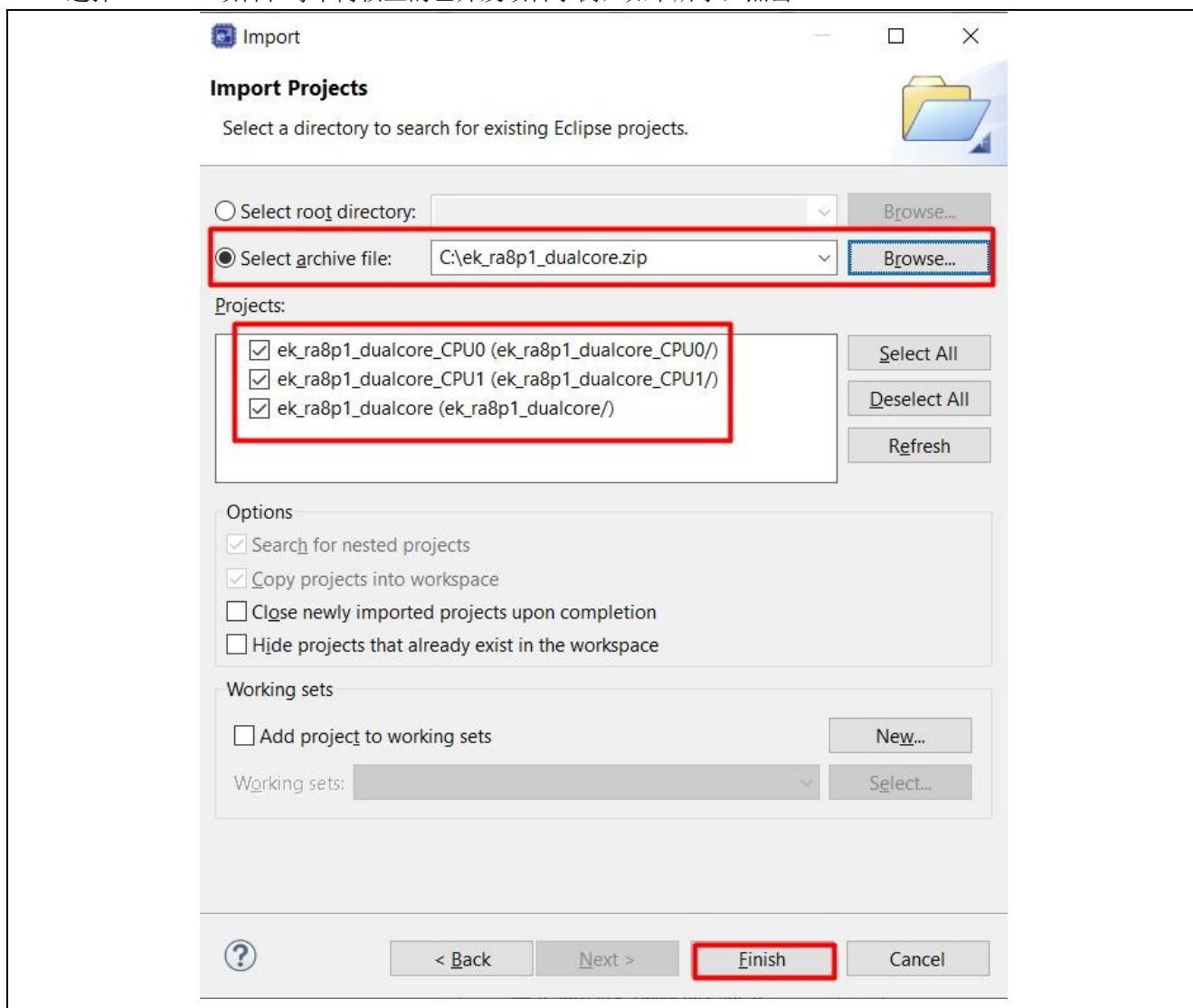


图 52. 将项目导入工作空间的示例。

## 6.2 编译项目

最简单的方法是右键单击 Solution 项目 ek\_ra8p1\_dualcore 并选择 Build Project。编译 Solution 项目时，两个 CPU 项目将自动按顺序编译：先编译 CPU0，然后编译 CPU1。

如果单独编译项目，请确保先编译 CPU0 项目，然后再编译 CPU1 项目。

### 6.2.1 编译在 CM85 内核上开发的项目

双击位于 ek\_ra8p1\_dualcore\_CPU0 项目中的 configuration.xml > 点击"Generate Project Content"。

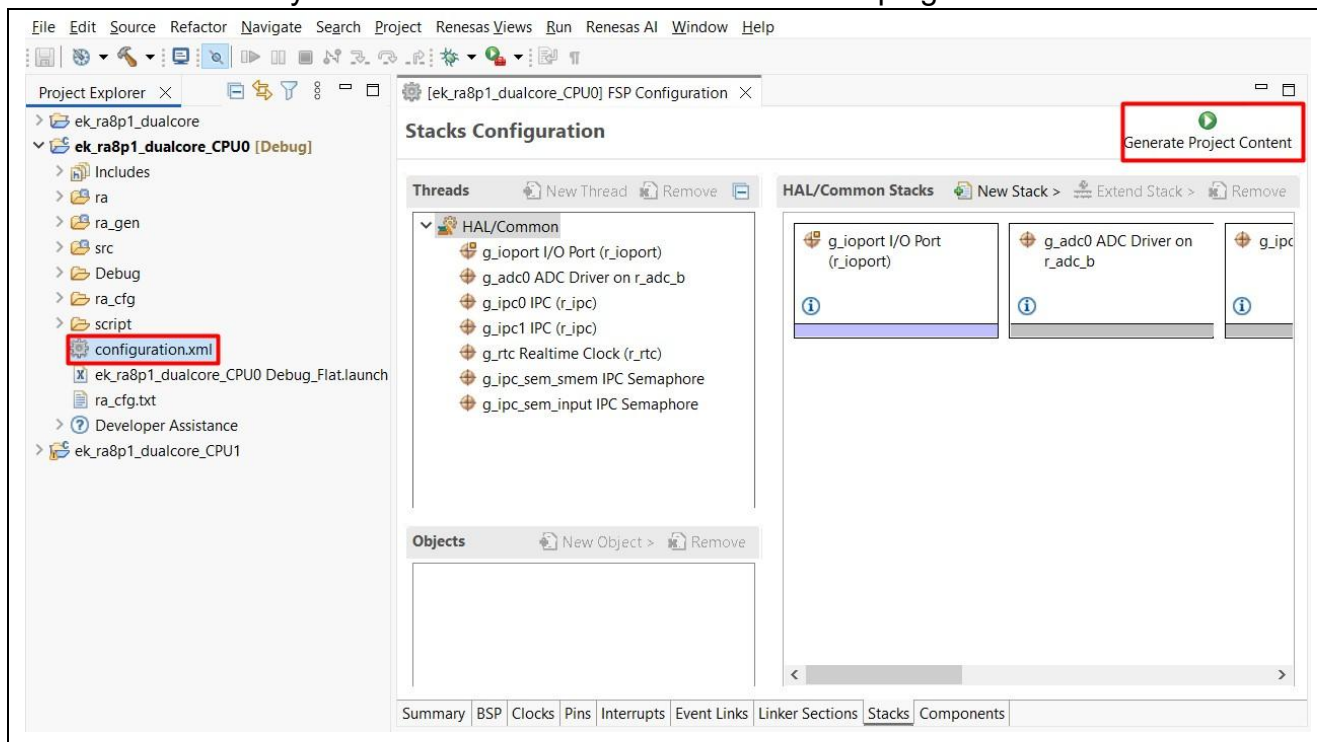


图 53. 在 CPU0 项目上生成项目内容的示例

生成项目内容后，右键单击 `ek_ra8p1_dualcore_CPU0`，然后选择 **Build Project** 以编译 CPU0 内核应用程序。

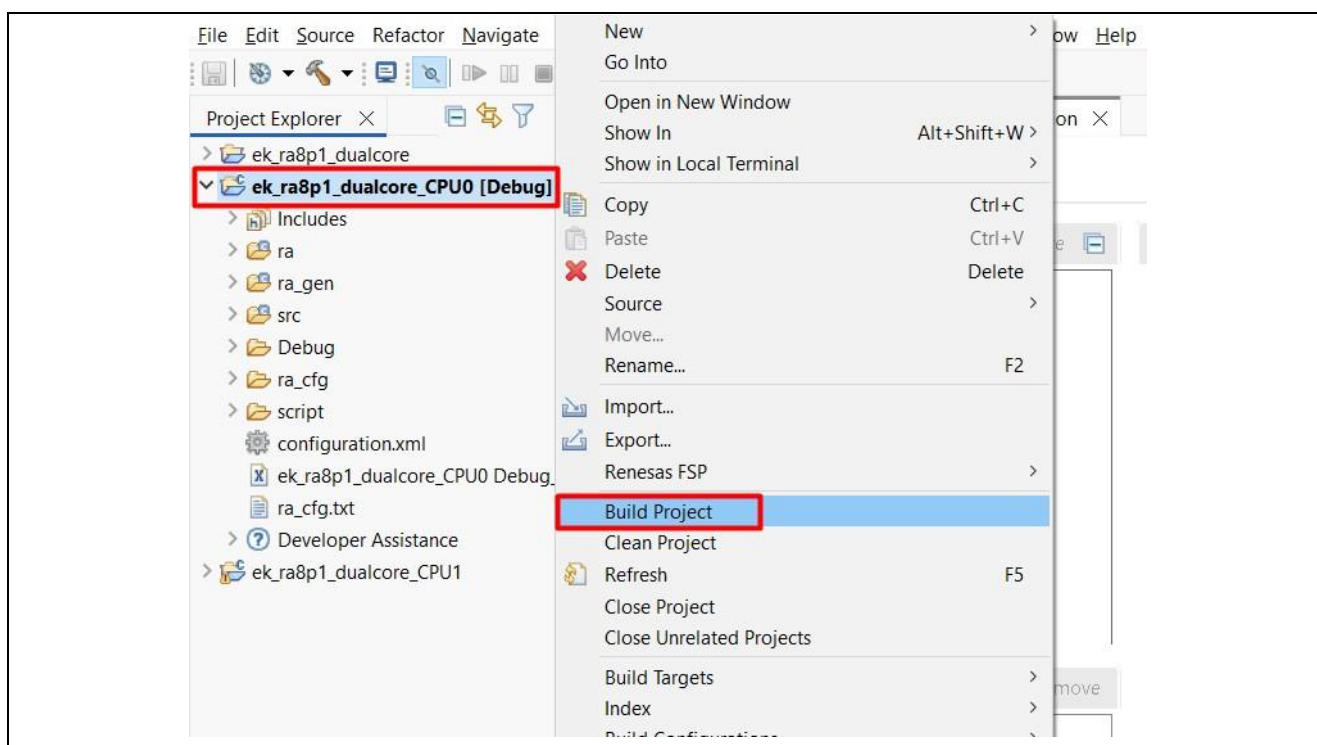


图 54. 编译 CPU0 双核项目的示例

通过检查 **Build Log** 控制台中的输出来验证编译是否成功完成。

### 6.2.2 编译在 CM33 内核上开发的项目

在 ek\_ra8p1\_dualcore\_CPU1 项目中，双击 configuration.xml，然后单击 Generate Project Content 以应用 CPU1 内核的配置设置。

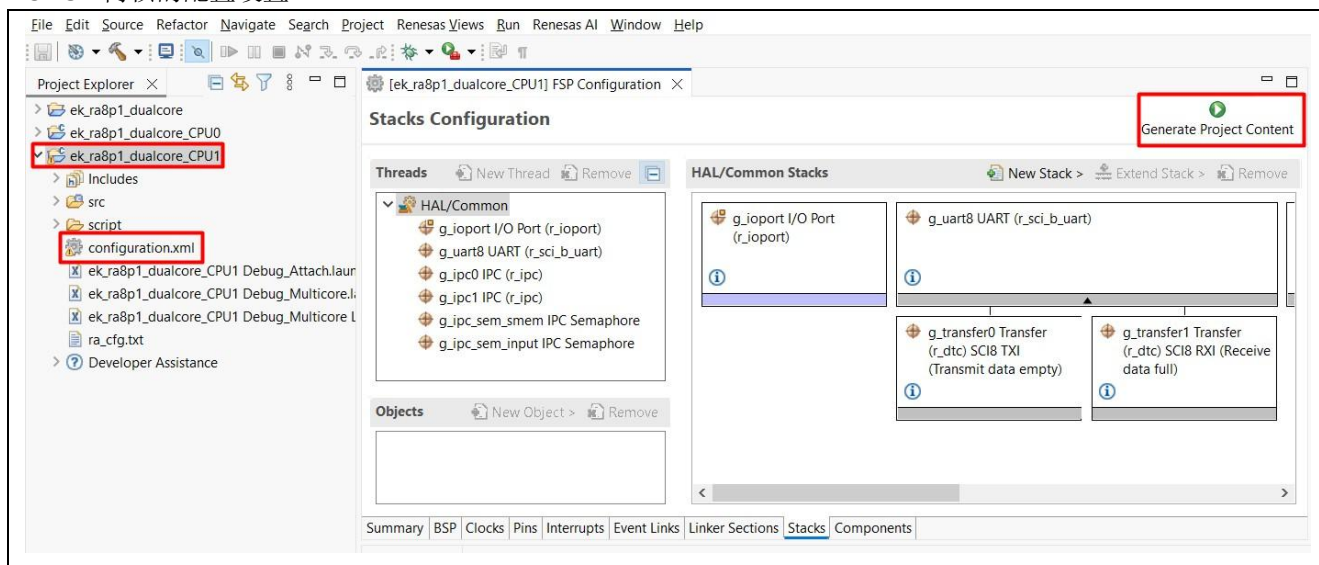


图 55. 在 CPU1 项目上生成项目内容的示例

内容生成后，右键单击 ek\_ra8p1\_dualcore\_CPU1，然后选择 Build Project 以编译 CPU1 内核应用程序。

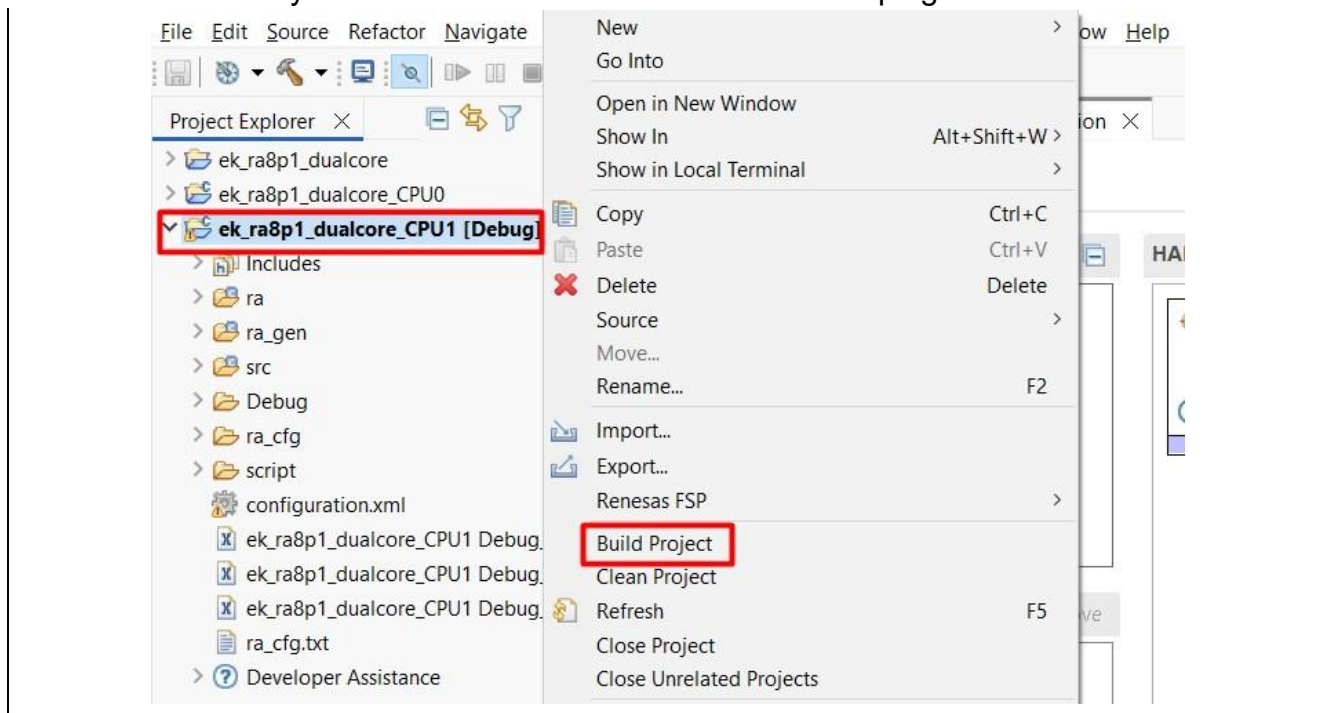


图 56. 编译 CPU1 双核项目的示例

通过检查 Build Log 控制台中的输出来确保编译成功完成。

或者，两个内核的编译过程可以通过 Solution 项目执行。右键单击 **ek\_ra8p1\_dualcore** Solution 项目并选择 **Build Project**，如图 57 所示。此命令将按顺序编译 Solution 中的所有项目，遵循 CPU0 → CPU1 的顺序。

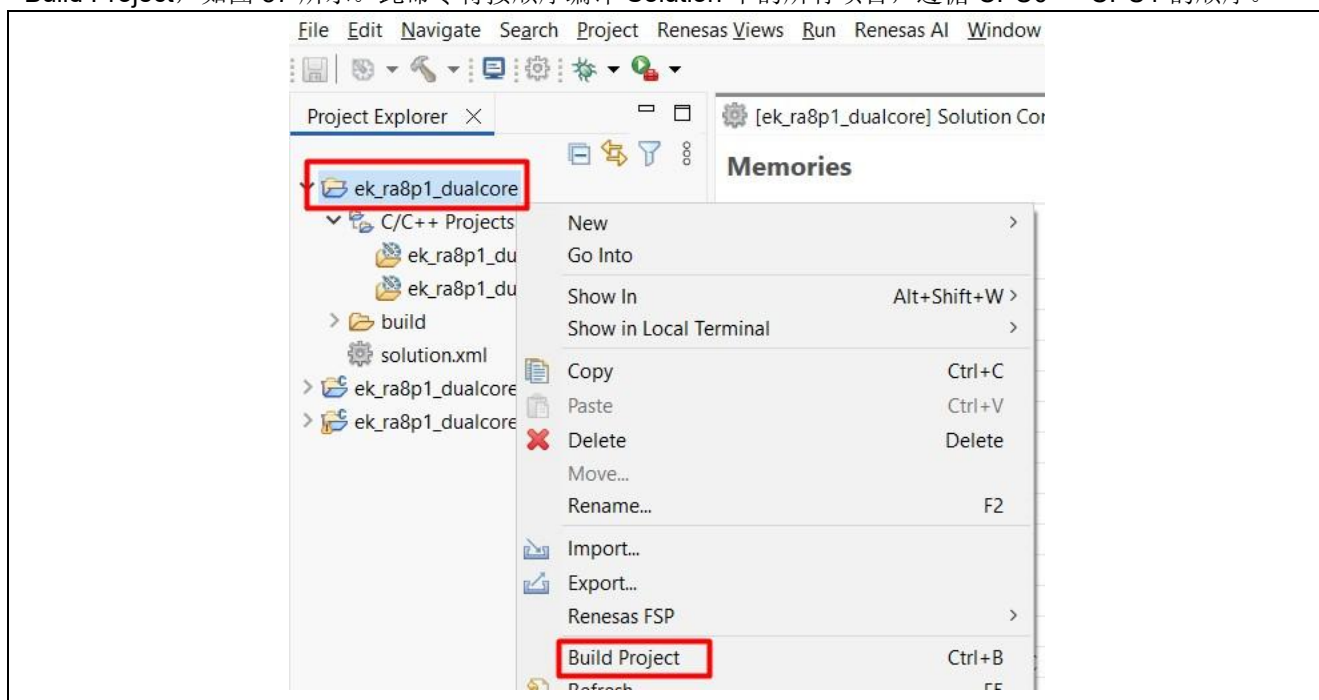


图 57. 编译 RA Solution 双核项目的示例

### 6.3 下载和运行项目

如第 3.2 节中的调试设置所述，器件必须在 **OEM\_PL2** 状态下初始化，并且此配置不需要 **TrustZone** 边界设置。

要启动双核调试会话，请打开 Debug Configurations 对话框，如图 58 所示。

选择"ek\_ra8p1\_dualcore\_CPU1 Debug\_Multicore Launch Group"，然后如图 59 所示点击 Debug 以同时启动两个内核的调试会话。

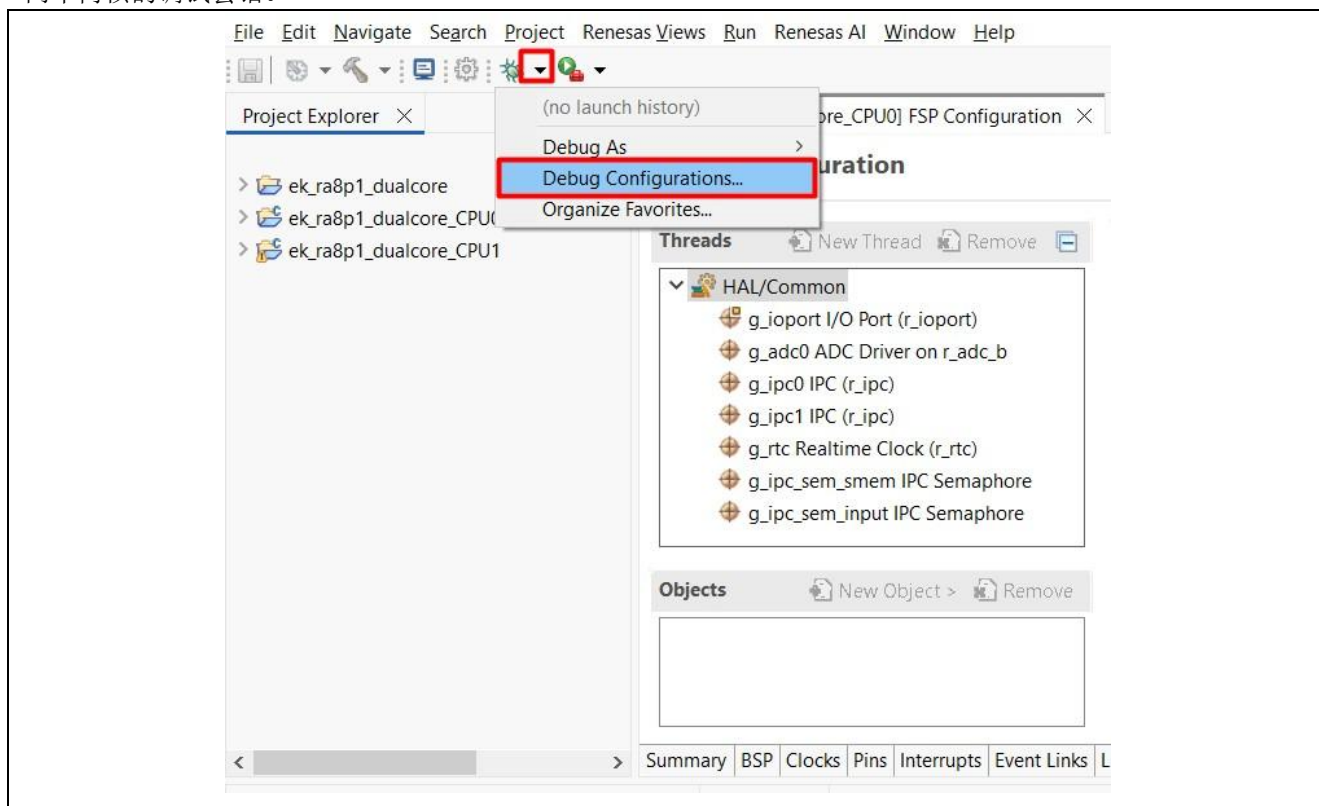


图 58. 在工具栏上选择 Debug 配置的示例

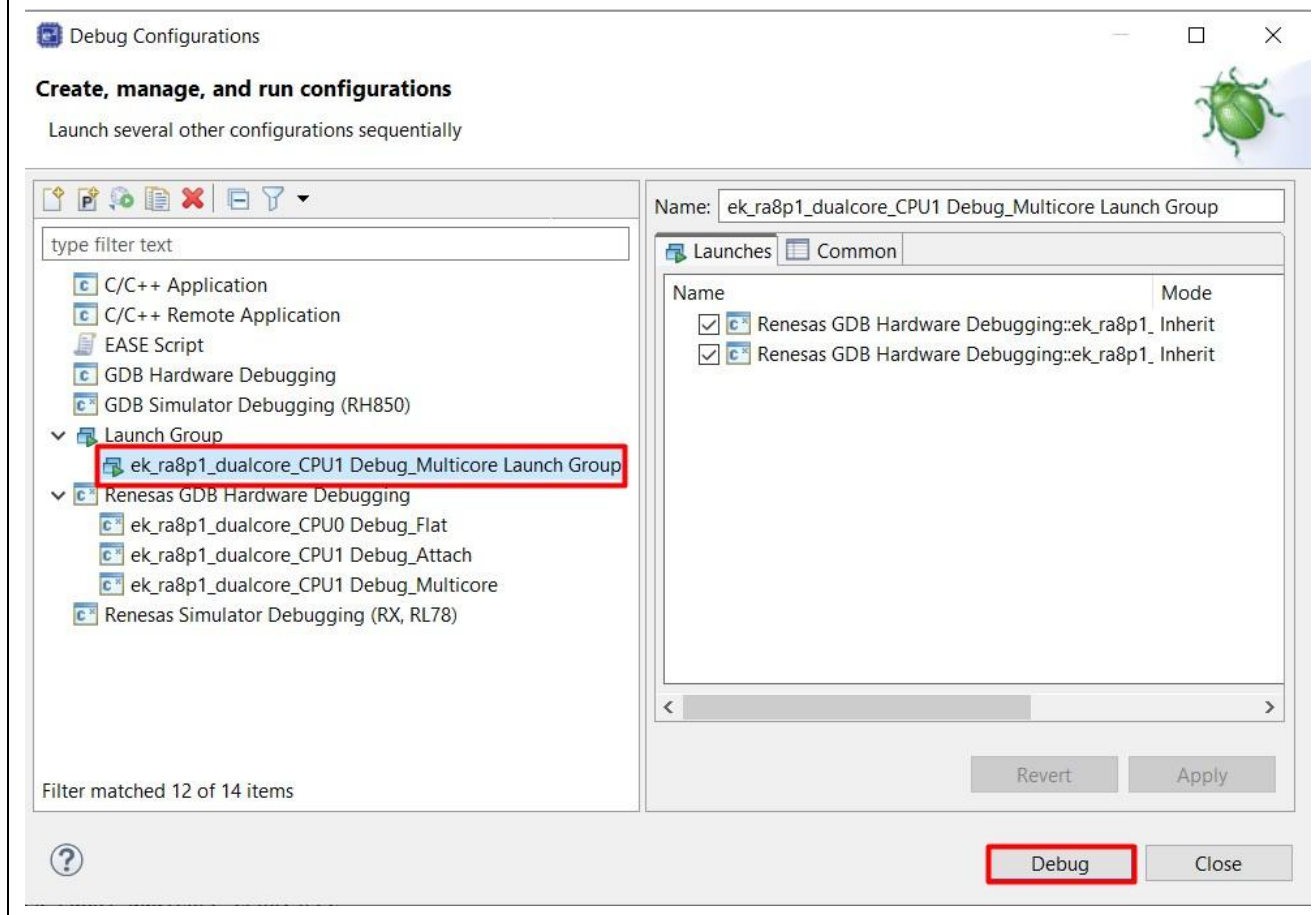


图 59. 使用 Multicore Launch Group 调试的示例

调试信息将显示在调试控制台上。在点击 **Resume** 开始应用程序执行之前，需要配置 Tera Term 终端以监控输出并验证应用行为。

要设置终端，请按照以下步骤操作：

1. 打开 Tera Term（图 60）。
2. 从串行连接选项中选择适当的 J-Link CDC UART Port，以建立与目标设备的通信（图 61）。
3. 通过导航到 **Setup > Terminal** 并勾选 **Local Echo** 选项来启用本地回显（图 61）。
4. 通过选择 **Setup > Serial Port** 来配置终端设置，然后根据项目需求调整波特率和其他串口参数（图 62）。

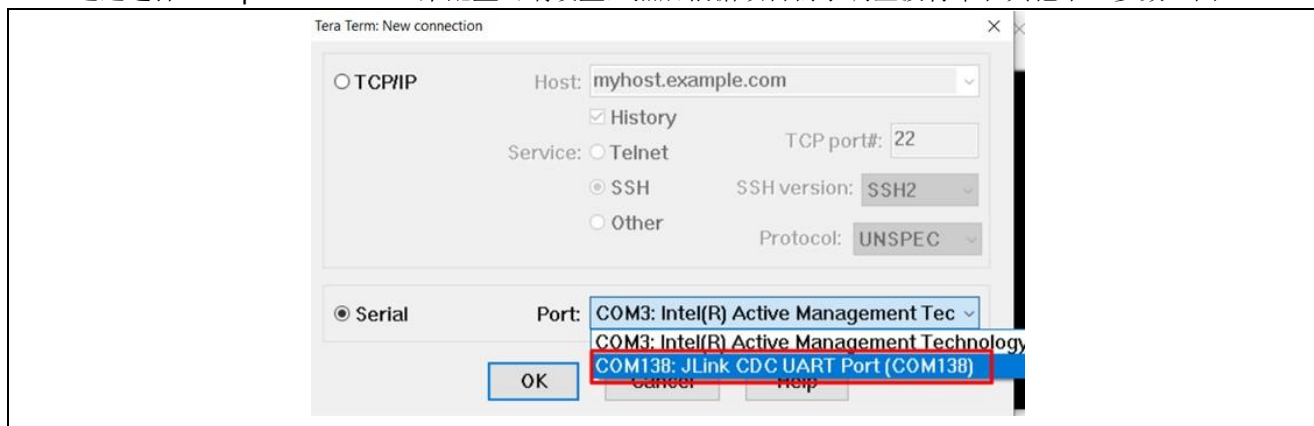


图 60. 连接到 Tera Term 终端的示例

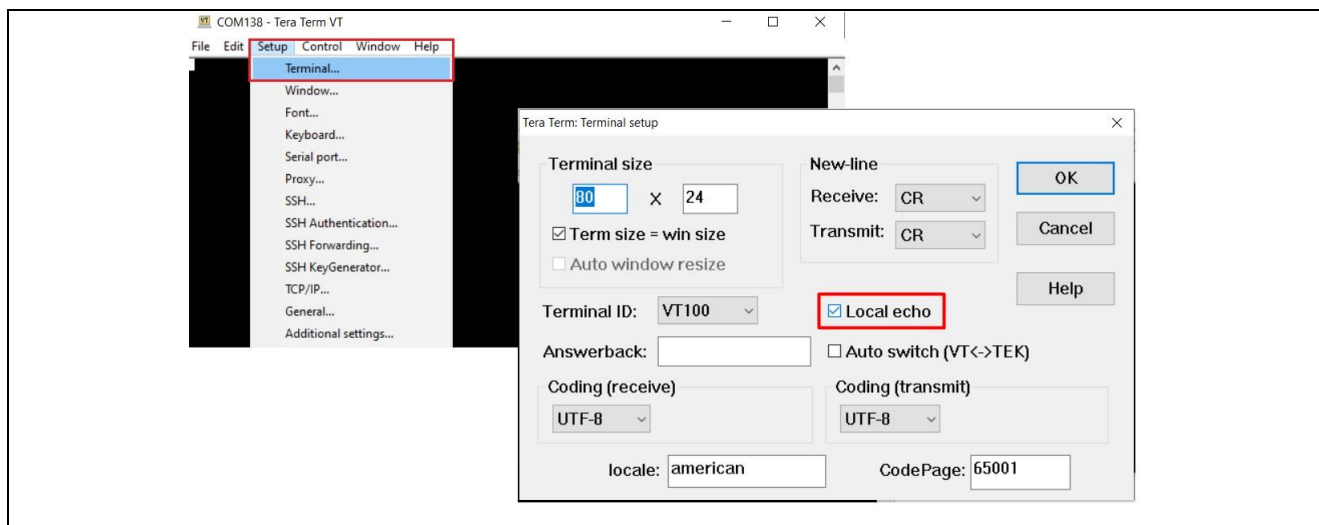


Figure 61. Example of Enable Local echo Tera Term Terminal

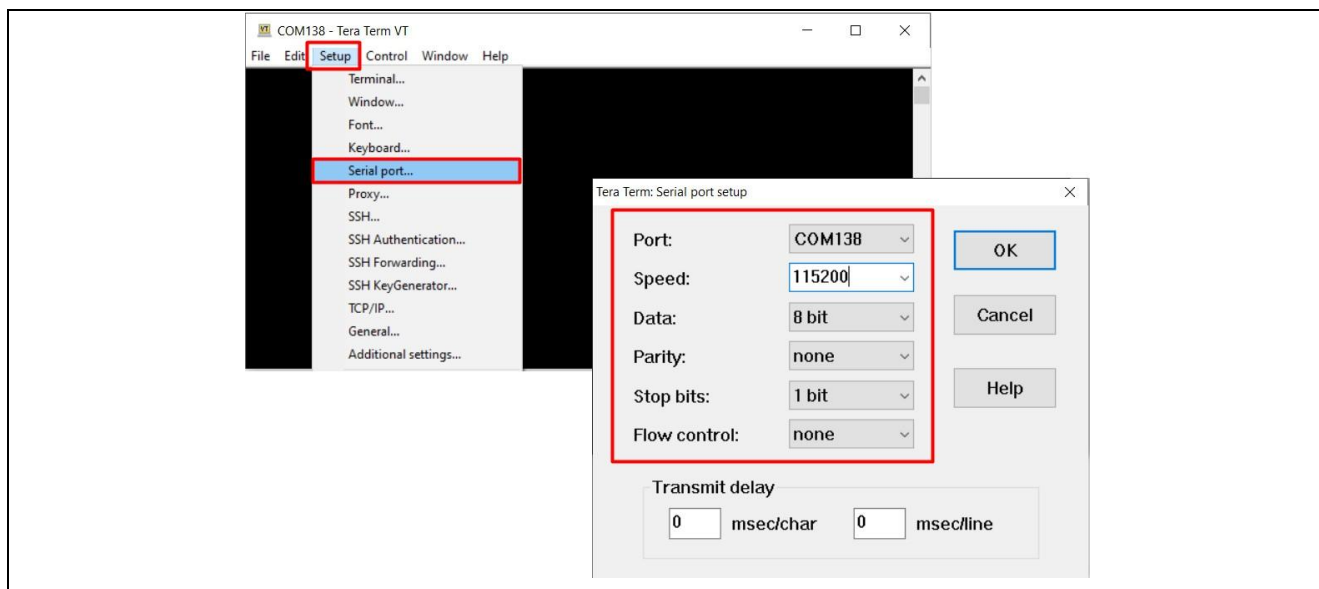


图 62. Tera Term 终端中串口设置的示例

完成 Tera Term 终端设置后，返回 e<sup>2</sup> studio 并点击 Resume 三次以启动应用程序执行。

初始屏幕布局应如图 63 所示。

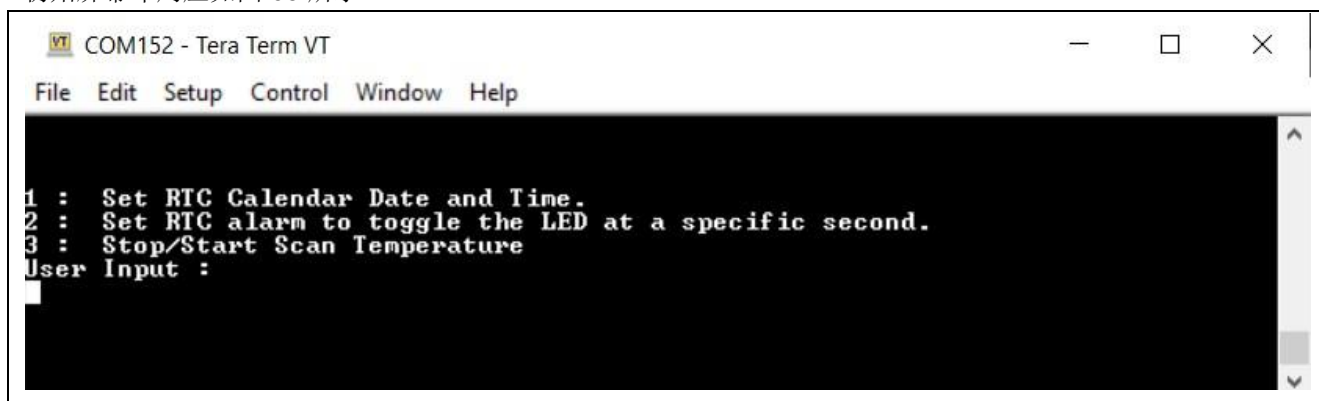


图 63. 应用程序的初始终端布局

根据选择的用户输入选项，终端布局将动态变化。图 64 展示了不同用户输入对应的各种终端布局。

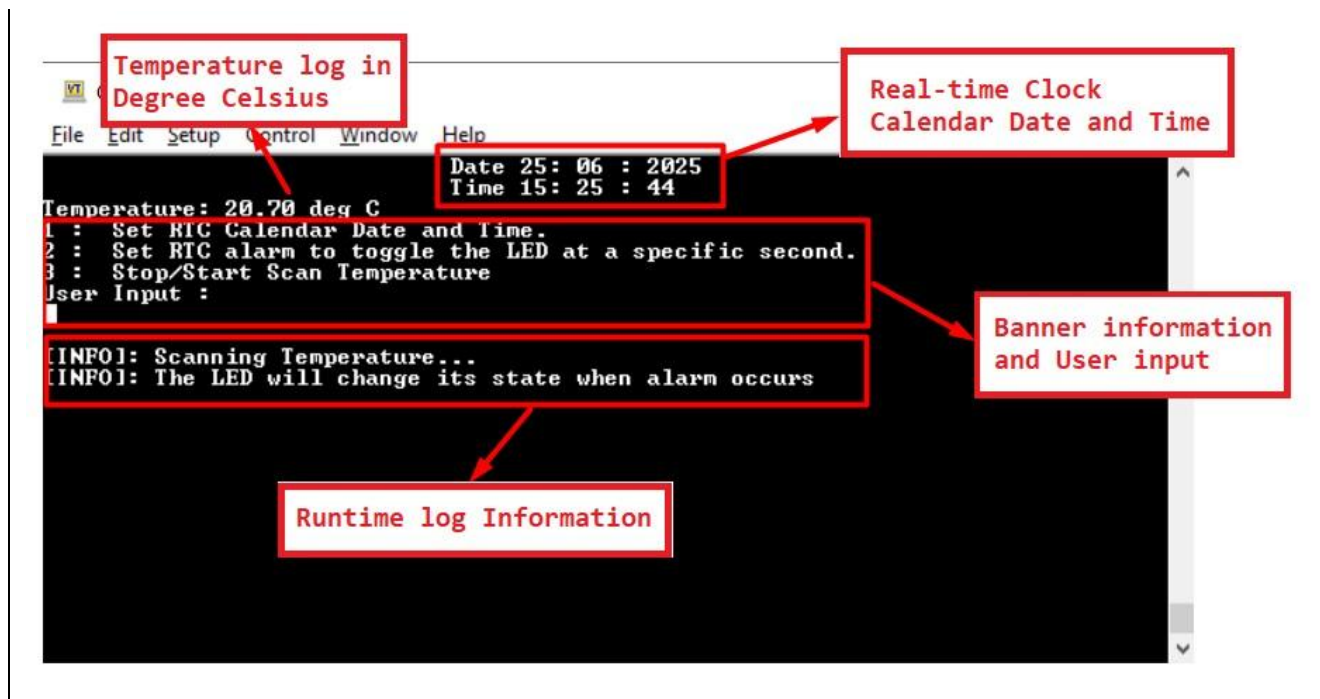


图 64. 应用终端布局概览

注意：当闹钟设置的秒数与 RTC 中的秒计数器匹配时，RTC 闹钟将激活。

要操作应用项目，请按照以下顺序进行：选项 1 → 选项 2 → 选项 3。此顺序是必需的，因为温度打印和 RTC 闹钟都由 RTC 定时器触发。

## 7. 验证基于 FreeRTOS 的项目

### 7.1 导入项目

1. 启动 e<sup>2</sup> studio IDE。
2. 在 Workspace launcher 中选择任意工作空间。
3. 关闭 Welcome 窗口。
4. 选择 File > Import。
5. 从 Import 对话框中选择 Existing Projects into Workspace。
6. 选择名为"Developing\_with\_RA8\_Dual\_Core\_MCU.zip"的文件中的压缩文件"dsp\_example\_dual\_core.zip"。
7. 选择 Solution 项目和每个内核上的已开发项目示例，如下所示，点击 Finish。

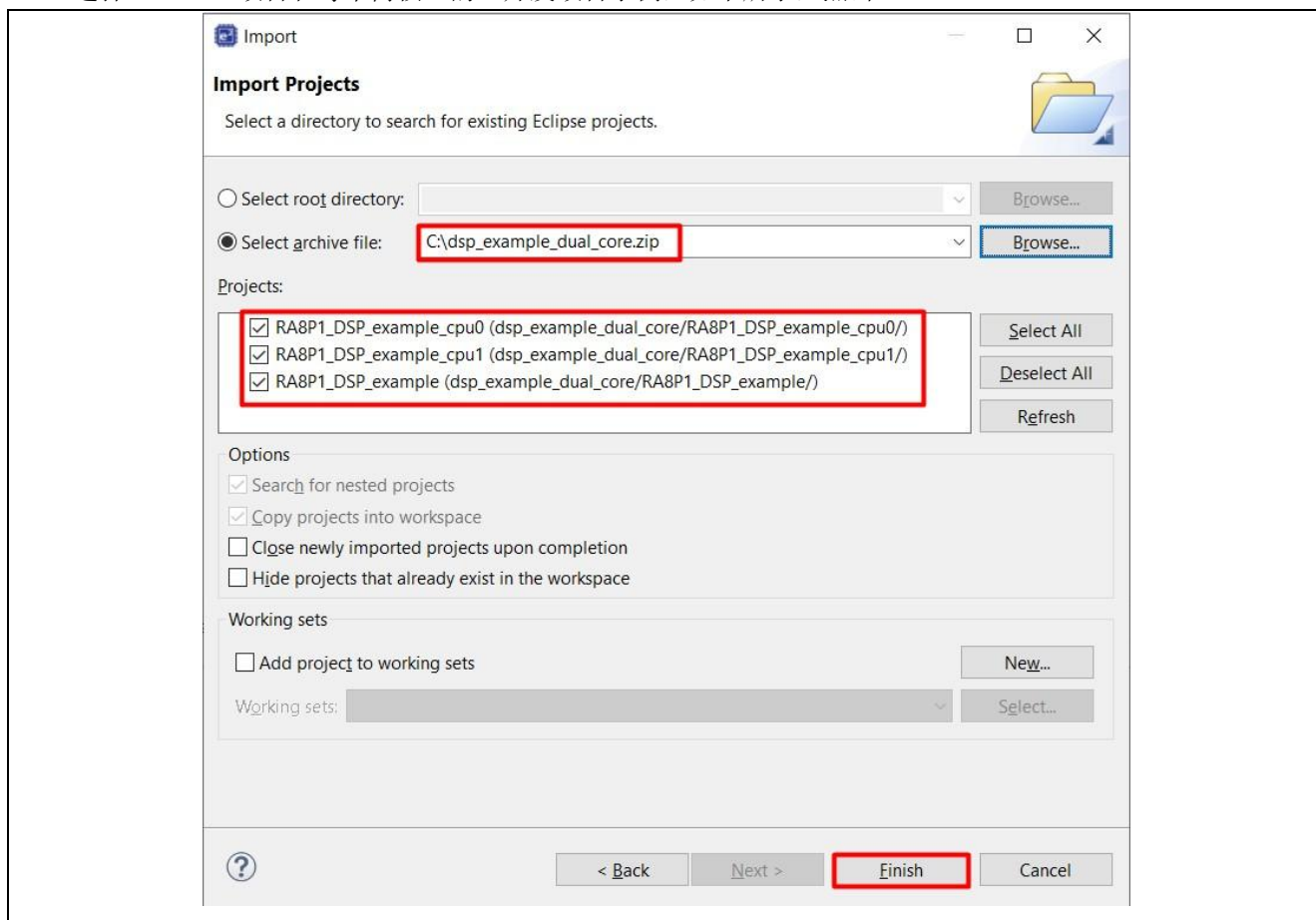


Figure 65. Example of Build DSP Example Project.

### 7.2 编译项目

按照 CPU0 → CPU1 的顺序依次编译项目，详见第 6.2 节。或者，右键单击 Solution 项目并选择 Build Project 以一次性编译所有包含的项目，如图 66 所示。

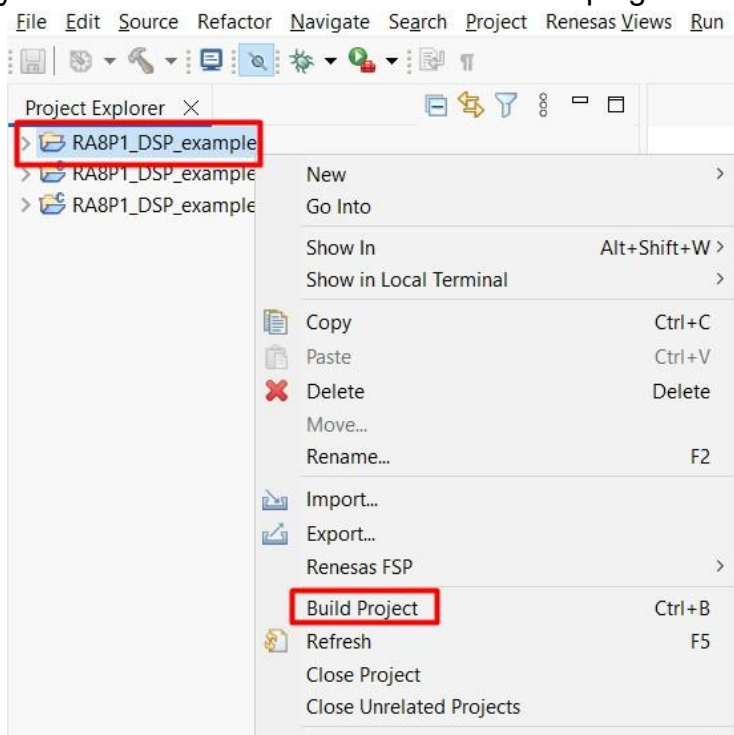


图 66. 编译 DSP 示例项目的示例。

通过验证 Build Log 控制台中的编译状态，确保 CPU0 和 CPU1 项目都编译成功。

### 7.3 下载和运行项目

如第 3.2 节所述，器件必须在 OEM\_PL2 状态下初始化，不需要 TrustZone 边界设置。

要同时开始调试两个内核：

1. 如图 67 所示打开 Debug Configurations。
2. 如图 68 所示选择"RA8P1\_DSP\_example\_cpu1 Debug\_Multicore Launch Group"。
3. 点击 Debug 启动双核调试会话。

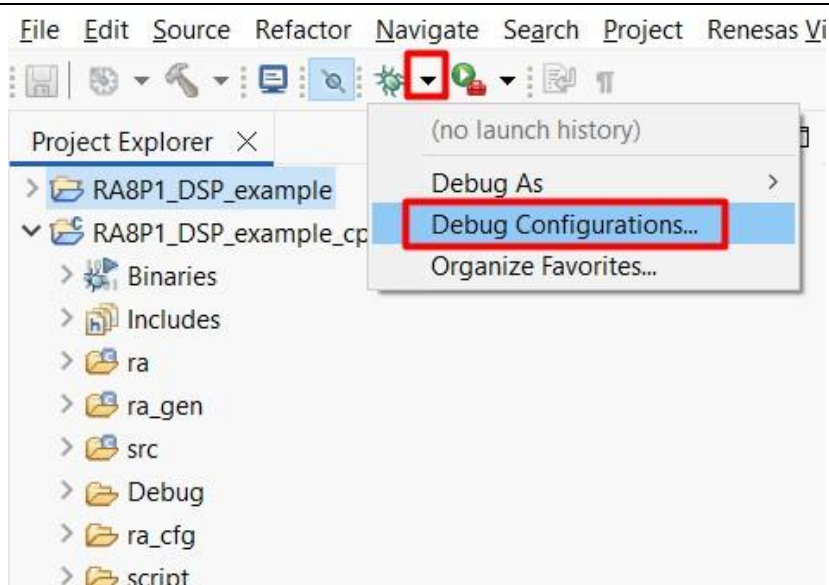


图 67. 在 DSP 示例中打开 **Debug Configuration** 的示例

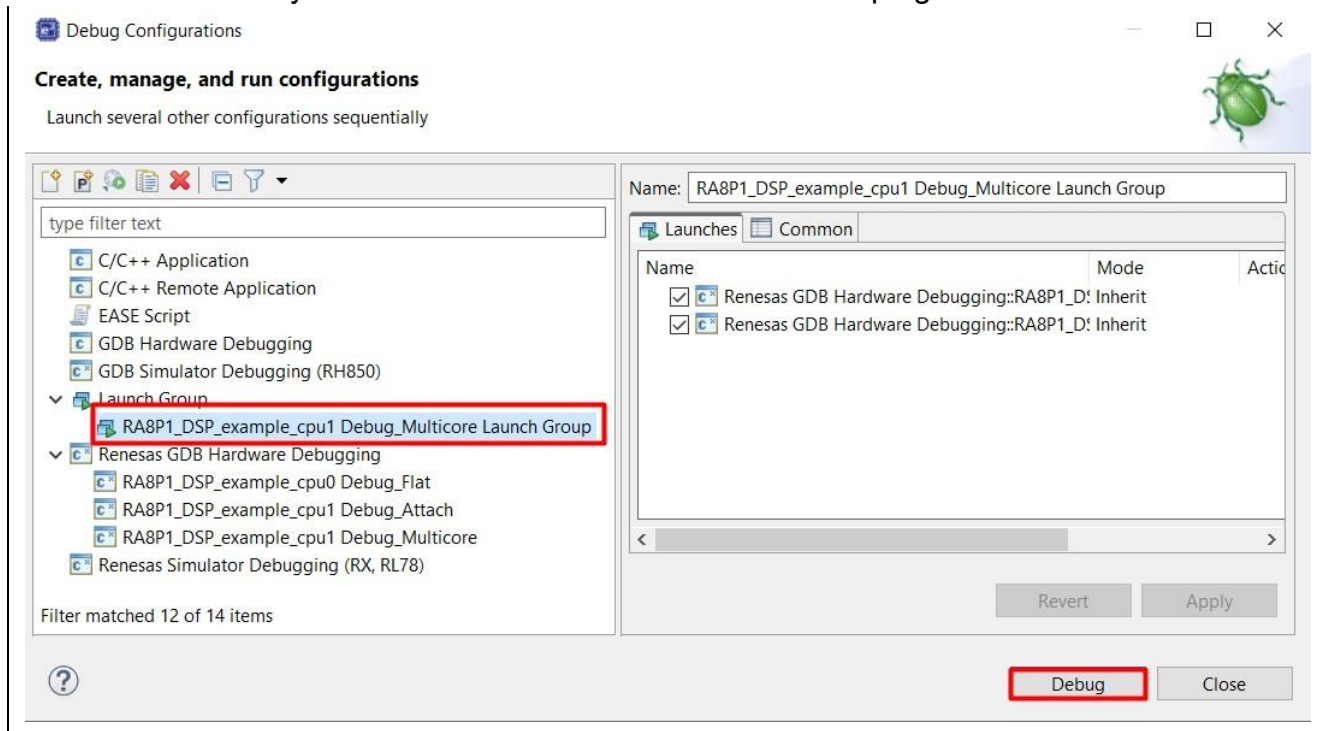


图 68. 调试 DSP 双核示例

打开串口终端并连接到 J-Link CDC UART Port，设置如下：波特率 115200 bps，8 位数据位，无校验，1 个停止位，无流控制。图 69 和图 70 展示了使用 Tera Term 终端的连接过程和配置步骤。

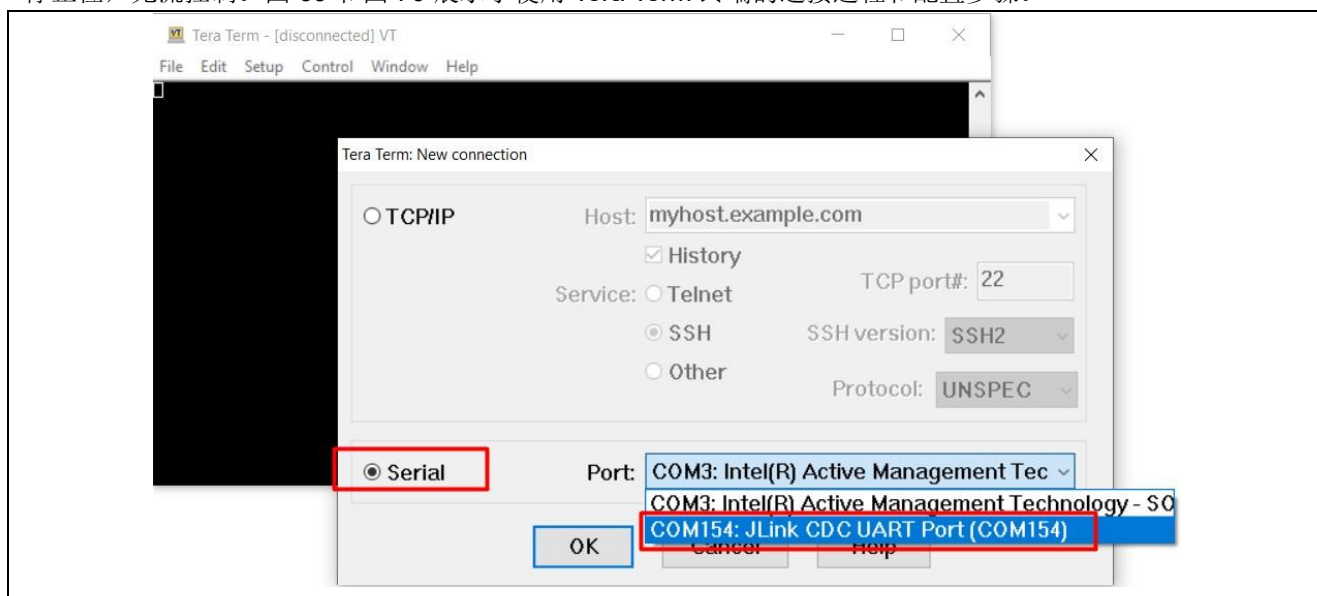


图 69. 连接到 J-Link CDC UART Port 的示例

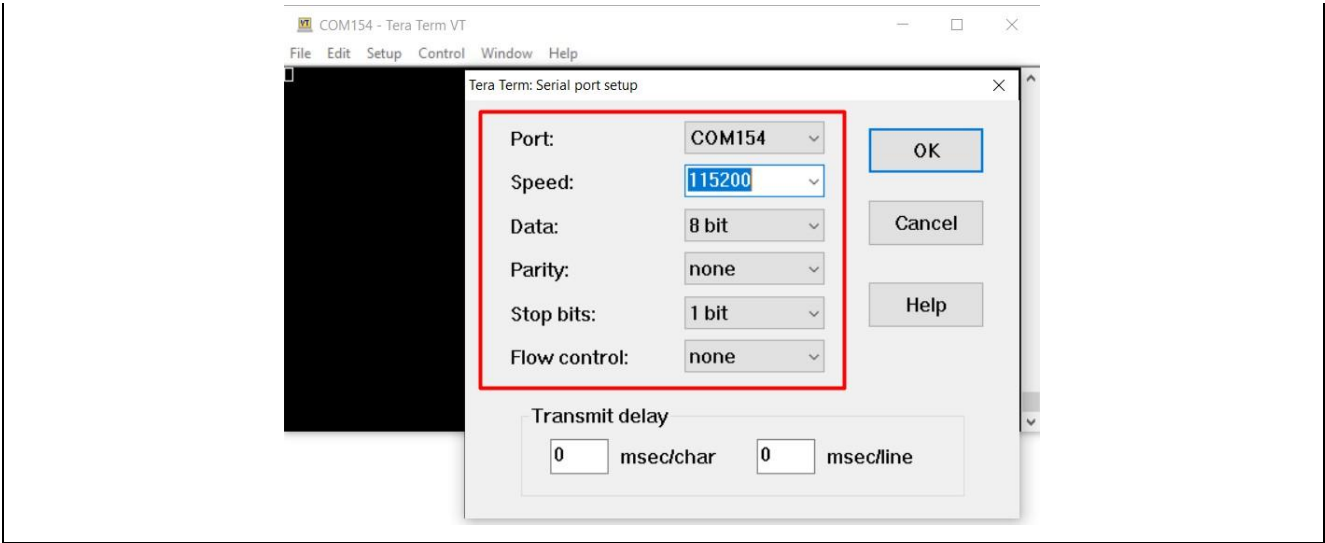


Figure 70. Example of Set Up Tera Term Serial Terminal

Return to e<sup>2</sup> studio and press Resume  three times to execute the application across both cores. Upon completion, the CMSIS-DSP FFT example will output its status to the terminal, as illustrated in Figure 71.

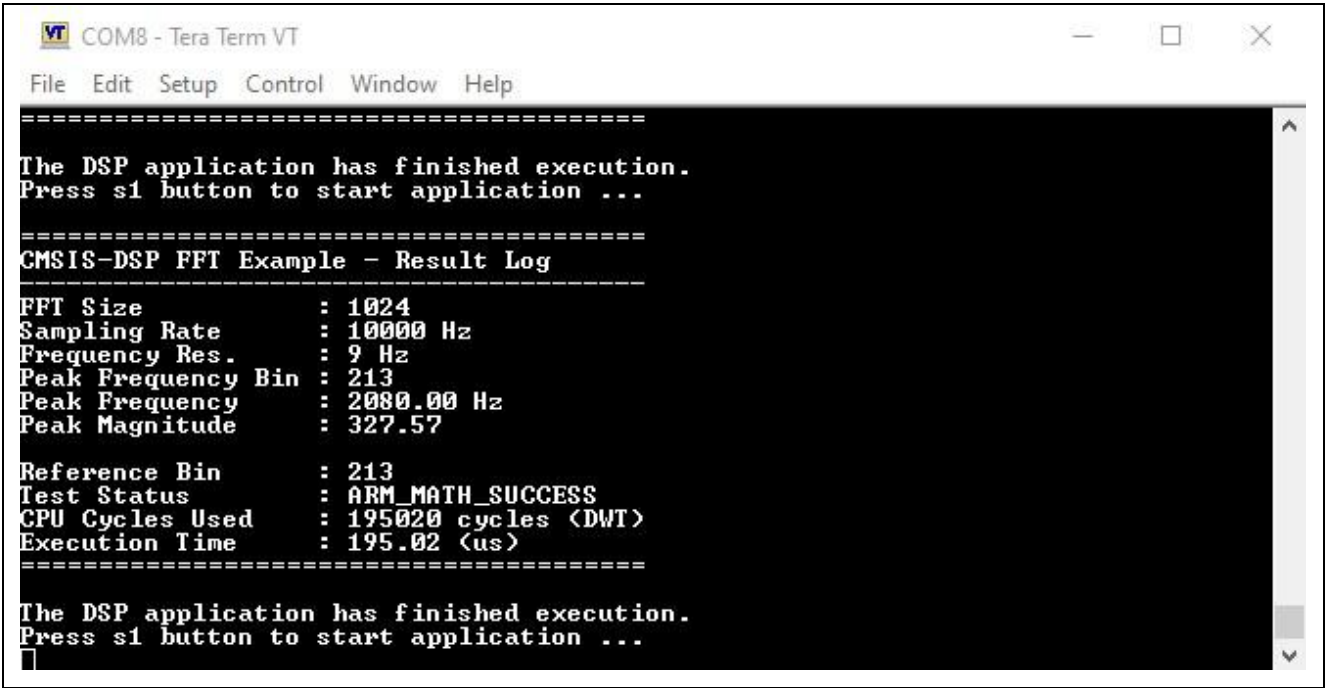


图 71. CMSIS-DSP FFT 示例的成功结果日志

## 8. 将双核示例项目迁移到新的双核 MCU

本章描述的步骤展示了将 RA8P1 上的 dsp\_example\_dual\_core 应用程序迁移到另一个 RA8 双核 MCU（如 RA8D2、RA8M2 或 RA8T2）的过程。

本节以 RA8D2 器件作为演示示例。然而，相同的步骤适用于其他双核 MCU，只需按照后续描述中的说明进行少量调整。

相同的迁移步骤也可以应用于本应用笔记中包含的 ek\_ra8p1\_dualcore 项目。

整个迁移过程可以分为三个主要部分：

- 迁移 **Solution** 项目：建议通过选择目标板来创建新的 **Solution** 项目，然后更新 **solution.xml** 文件中的内存分区、引脚配置和时钟设置以匹配目标 **MCU**。
- 迁移 **CPU0** 项目：调整 **BSP** 配置和 **FSP** 模块，并实现必要的源代码修改以满足 **CPU0** 上迁移项目的需求。
- 迁移 **CPU1** 项目：调整 **BSP** 配置和 **FSP** 模块，并实现必要的源代码修改以满足 **CPU1** 上迁移项目的需求。

### 步骤 1：创建新的双核 **Solution** 项目

参考第 3.1 节描述的步骤，使用 Multicore Flat FreeRTOS 模板创建一个新的 **Solution** 项目 **RA8D2\_DSP\_example**。

在 Device Selection 对话框中，选择 EK-RA8D2 作为目标板，并选择 LLVM 作为工具链，如图 72 所示。

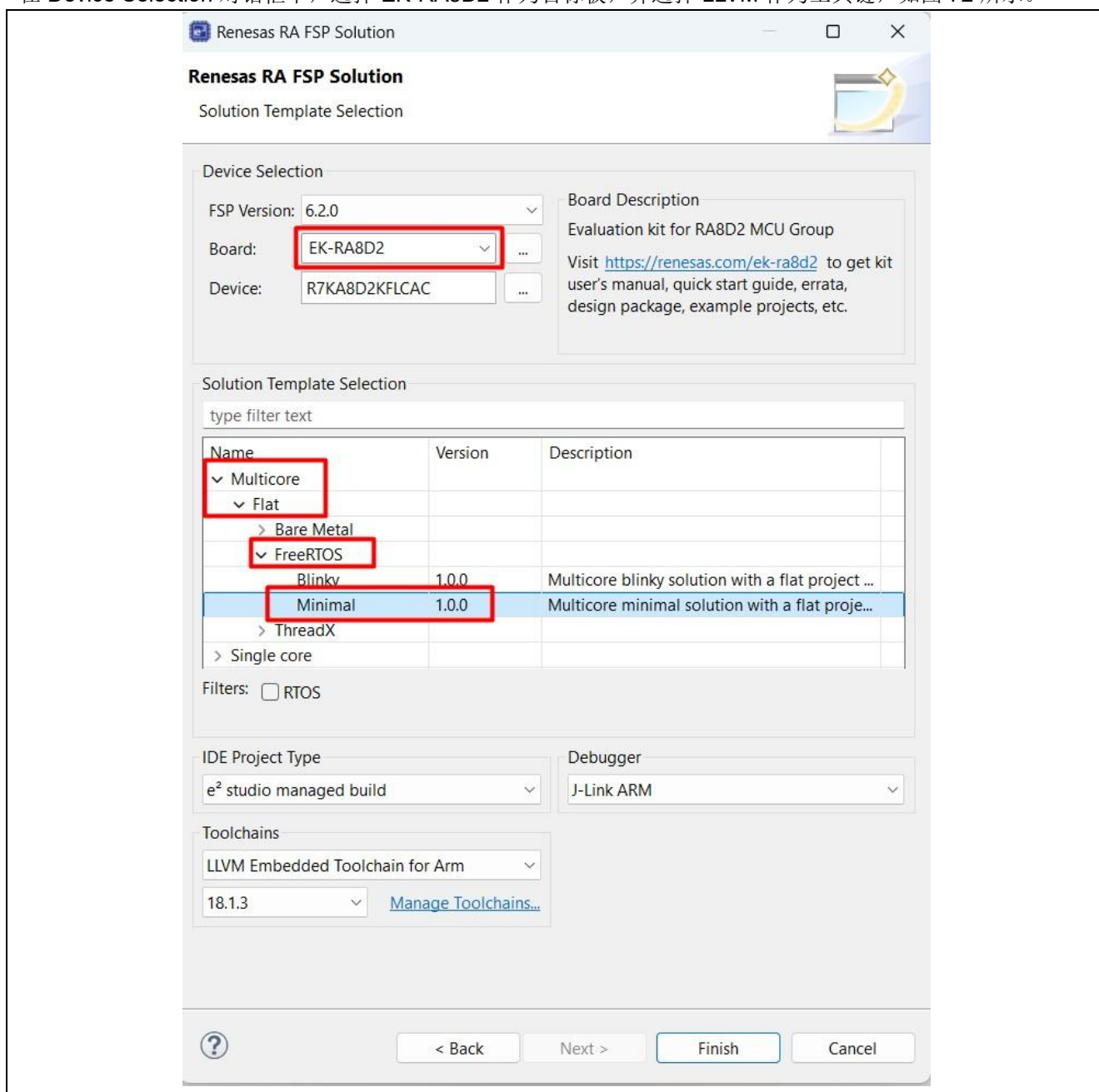


图 72. Solution 模板选择示例。

步骤 2: 在 **Solution** 项目中定义内存分区、配置引脚和时钟。

- 添加分区（例如 SHARED\_MEM）。
- 为 SHARED\_MEM 分区分配 Start/Size/Core/Security，如下所示。

对于基于 `ek_ra8p1_dualcore` 的迁移，这两个配置步骤可以参考第 5.1.2 节的设置。在

`RA8P1_DSP_example` 项目中，内存映射保持默认配置。因此，在迁移后的 `Solution` 项目中不需要修改内存映射设置。

- 整个多核项目的时钟配置。对于 `RA8D2_DSP_example`，启用 `SCICLK` 为 120 MHz 以使用 `VCOM UART` 在 `RA8D2_DSP_example_CPU1` 上进行日志终端输出。导航到 `RA8D2_DSP_example` 项目 `solution.xml` 中的 `Clocks` 选项卡以应用此配置，如图 73 所示。禁用好未使用的时钟以优化系统性能并降低功耗。

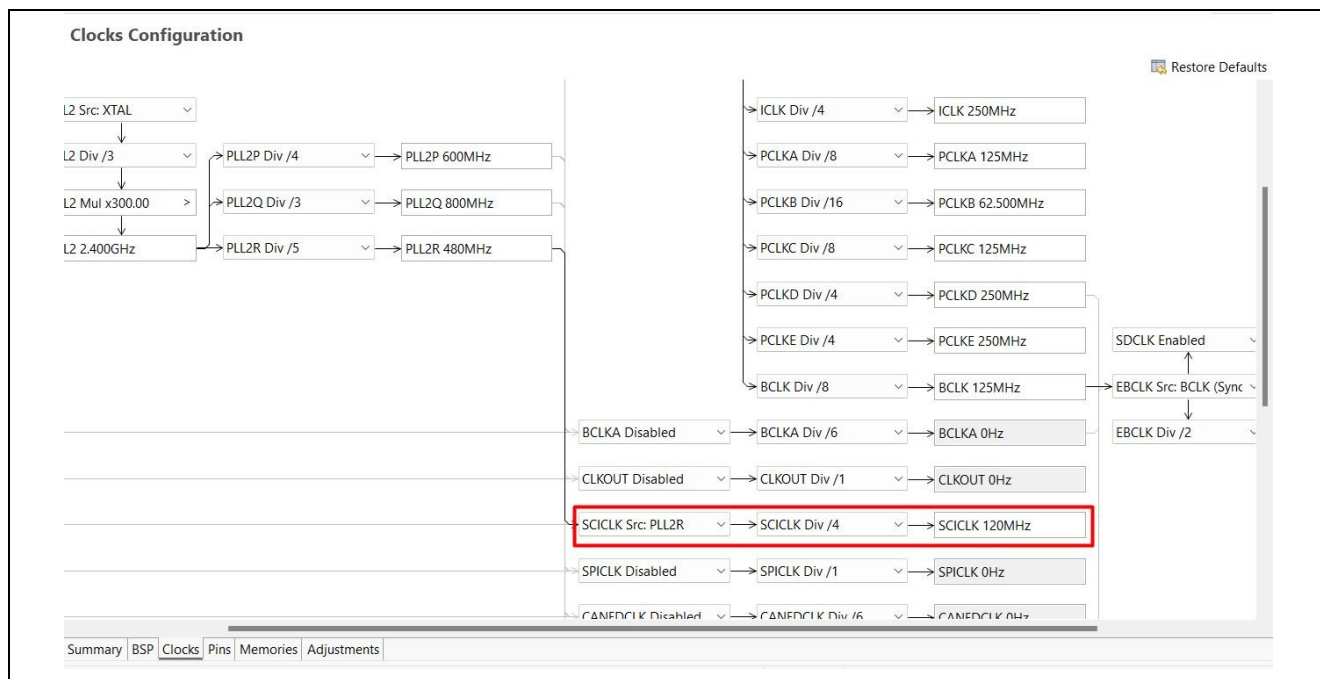


图 73. `Solution` 项目中时钟配置的示例

- 引脚配置：SCI UART 引脚 `PD02 (RXD8)`、`PD03 (TXD8)` 和外部中断引脚 `P009 (IRQ13-DS)` 分配给 `CPU1` 项目，用于与终端进行 `VCOM` 通信和接收按钮中断信号。因此，这些引脚稍后将在 `CPU1` 项目中进行配置。在 `CPU0` 项目中禁用这些 Pin。

导航到 Pins 选项卡，展开 Connectivity: SCI 类别，选择 `SCI8`。然后将 Operation Mode 设置为 Disable。

接下来，在 IRQ 中，向下滚动并将 `IRQ13-DS` 设置为 None。

**注意：**确保在 `Solution` 项目中所做的任何配置更改都通过右键单击 `Solution` 项目 `RA8D2_DSP_example` 并选择 **Build Project** 应用到链中的所有项目。

### 步骤 3：为 `CPU0` 项目添加和配置 FSP Stack

在 `RA8D2_DSP_example_CPU0` 中配置 IPC 并设置 IPC 可屏蔽中断。核间通信 (IPC) 在专门的 IPC 线程中处理，该线程以最高优先级运行，以确保两个内核之间及时的消息交换。在此应用的 IPC 操作期间还使用了互斥锁 (Mutex)，以安全地管理对共享资源的访问。

通过在 `RA8D2_DSP_example_CPU0/configuration.xml` 的 Stacks 选项卡中点击 **New Thread** 来创建一个名为 `Ipc Thread` 的新线程。此线程的配置如图 74 所示。

| Property                          | Value           | Property                | Value      |
|-----------------------------------|-----------------|-------------------------|------------|
| General                           |                 | Common                  |            |
| Custom FreeRTOSConfig.h           |                 | Thread                  |            |
| Use Preemption                    | Enabled         | Symbol                  | ipc_thread |
| Use Port Optimised Task Selection | Disabled        | Name                    | Ipc Thread |
| Use Tickless Idle                 | Disabled        | Stack size (bytes)      | 1024       |
| Cpu Clock Hz                      | SystemCoreClock | Priority                | 11         |
| Tick Rate Hz                      | 1000            | Thread Context          | NULL       |
| Max Priorities                    | 12              | Memory Allocation       | Static     |
| Minimal Stack Size                | 128             | Allocate Secure Context | Enable     |
| Max Task Name Len                 | 16              |                         |            |
| Use 16-bit Ticks                  | Disabled        |                         |            |
| Idle Should Yield                 | Enabled         |                         |            |
| Use Task Notifications            | Enabled         |                         |            |
| Task Notification Array Entries   | 1               |                         |            |
| Use Mutexes                       | Enabled         |                         |            |
| Use Recursive Mutexes             | Disabled        |                         |            |
| Use Counting Semaphores           | Enabled         |                         |            |

图 74. RA8D2\_DSP\_example\_CPU0 中 Ipc Thread 设置的示例

通过选择 New Stack > System > IPC (r\_ipc) 在 Ipc Thread 下添加两个 IPC Stack。IPC 配置如图 75 所示。

| Stacks Configuration                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Stacks Configuration                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>Threads</div> <div> <div>HAL/Common</div> <div>g_ioport I/O Port (r_ioport)</div> <div>FreeRTOS Port (rm_freertos_port)</div> <div>Ipc Thread</div> <div>g_ipc0 IPC (r_ipc)</div> </div> <div>Objects</div> <div>Summary</div> <div>BSP</div> <div>Clocks</div> <div>Pins</div> <div>Interrupts</div> <div>Event Links</div> <div>Linker Sections</div> <div>Stacks</div> <div>Components</div> <div>Problems</div> <div>Console</div> <div>Properties</div> <div>Smart Browser</div> <div>Smart Manual</div> <div>Debug</div> <div>g_ipc0 IPC (r_ipc)</div> <div>Settings</div> <div>Property</div> <div>Value</div> <div>Common</div> <div>Module g_ipc0 IPC (r_ipc)</div> <div>Name</div> <div>g_ipc0</div> <div>Channel</div> <div>0</div> <div>Callback</div> <div>g_ipc0_callback</div> <div>Interrupt Priority</div> <div>Priority 5</div> | <div>Threads</div> <div> <div>HAL/Common</div> <div>g_ioport I/O Port (r_ioport)</div> <div>FreeRTOS Port (rm_freertos_port)</div> <div>Ipc Thread</div> <div>g_ipc0 IPC (r_ipc)</div> <div>g_ipc1 IPC (r_ipc)</div> </div> <div>Objects</div> <div>Summary</div> <div>BSP</div> <div>Clocks</div> <div>Pins</div> <div>Interrupts</div> <div>Event Links</div> <div>Linker Sections</div> <div>Stacks</div> <div>Components</div> <div>Problems</div> <div>Console</div> <div>Properties</div> <div>Smart Browser</div> <div>Smart Manual</div> <div>Debug</div> <div>Progress</div> <div>g_ipc1 IPC (r_ipc)</div> <div>Settings</div> <div>Property</div> <div>Value</div> <div>Common</div> <div>Module g_ipc1 IPC (r_ipc)</div> <div>Name</div> <div>g_ipc1</div> <div>Channel</div> <div>1</div> <div>Callback</div> <div>NULL</div> <div>Interrupt Priority</div> <div>Disabled</div> |

图 75. RA8D2\_DSP\_example\_CPU0 中 IPC 配置的示例

通过点击 New Stack > System > IPC Semaphore 在 Ipc Thread 下于 CPU0 项目中添加两个硬件信号量 0 和 1。IPC 信号量配置如图 76 所示。

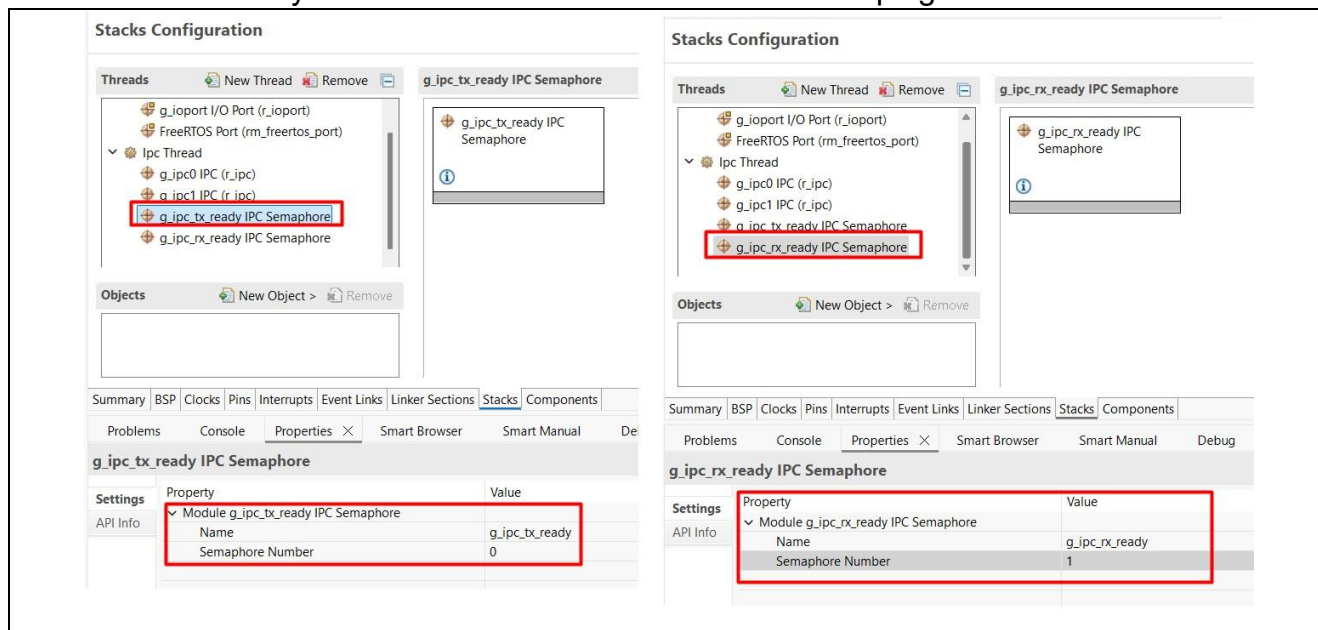


图 76. RA8D2\_DSP\_example\_CPU0 项目中硬件信号量配置的示例

在 RA8D2\_DSP\_example\_CPU0 中，Arm DSP 示例在一个单独的线程中运行。打开 RA8D2\_DSP\_example\_CPU0/configuration.xml，然后点击 Add New Thread 并创建一个名为 DSP Thread 的线程。此线程的配置如图 77 所示：

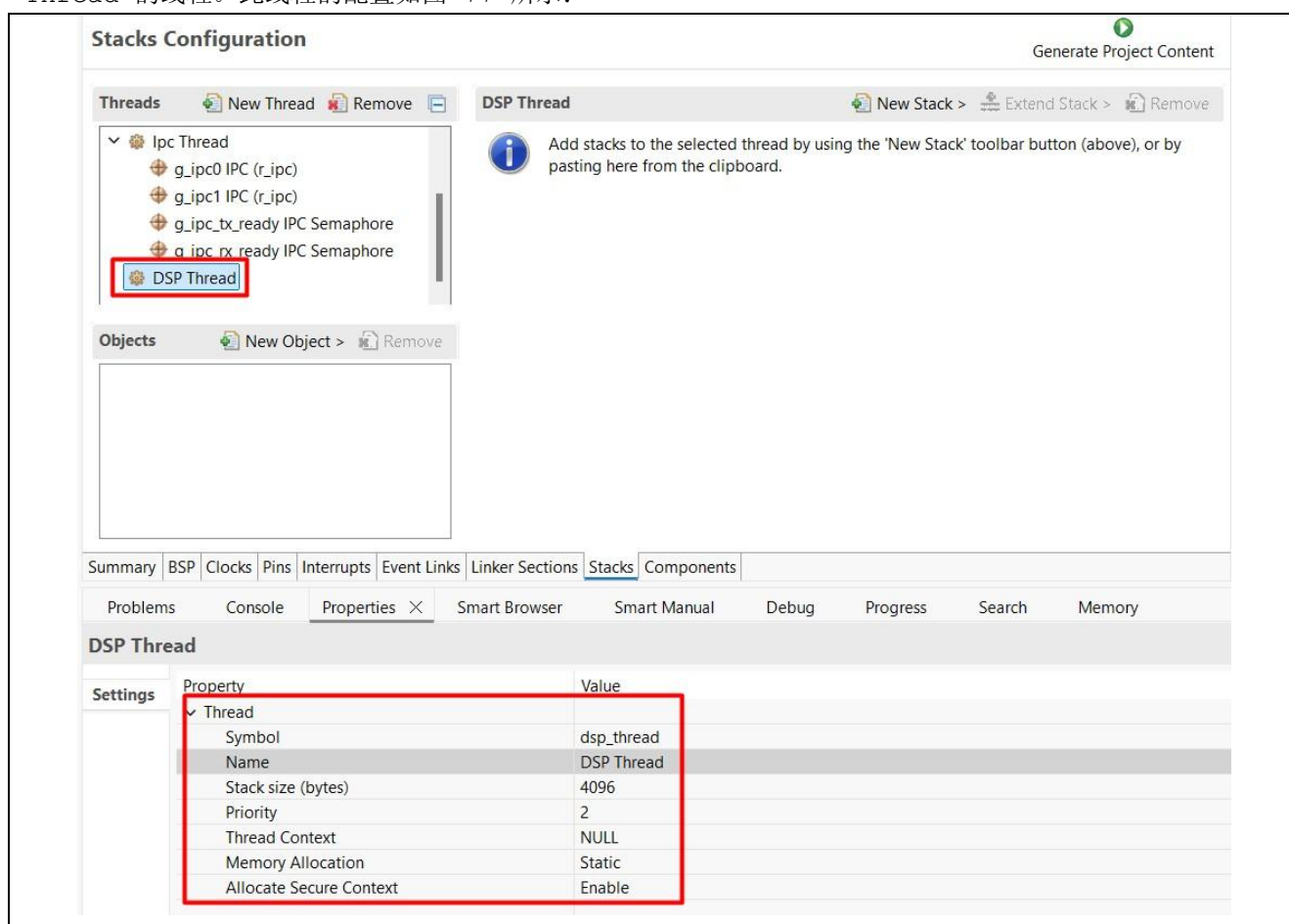


图 77. 在 RA8D2\_DSP\_example\_CPU0 项目中添加 DSP Thread 的示例

然后按以下方式在 DSP Thread 下将 Arm DSP 库添加到 RA8D2\_DSP\_example\_CPU0 项目中：  
 点击 New Stack > DSP > Arm CMSIS DSP Library Source。完整的 FSP Stack 配置如图 78 所示。  
 RA8D2\_DSP\_example\_CPU0 as shown in Figure 78.

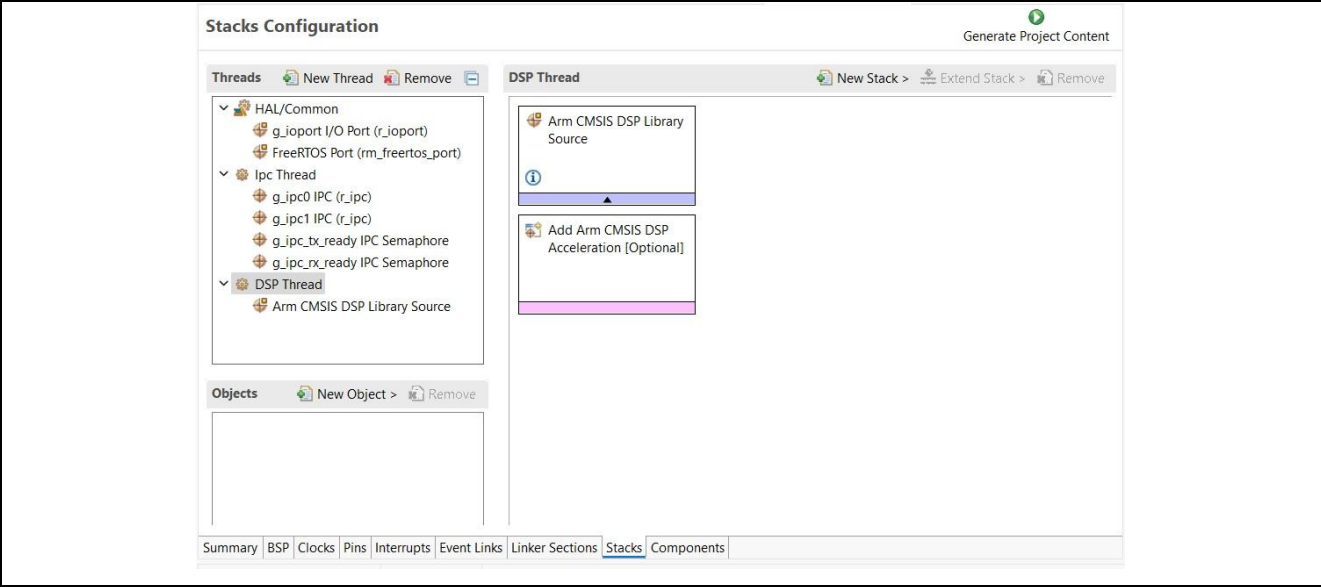


图 78. RA8D2\_DSP\_example\_CPU0 项目中的完整 FSP Stack 配置。步骤 4:

RA8D2\_DSP\_example\_CPU0 项目中的引脚配置。

如步骤 1 所述，SCI UART 通道 8 引脚（PD02、PD03）和外部中断引脚（P009）也必须在 CPU0 项目中禁用，如下所示。

1. 导航到 Pins 选项卡，展开 Connectivity: SCI 类别，选择 SCI8。然后将 Operation Mode 设置为 Disable。
2. 在 IRQ 中，向下滚动并将 IRQ13-DS 设置为 None。

步骤 5: 将 arm\_cmplx\_mag\_f32 函数放置在 ITCM 中，将 testInput\_f32\_10khz 缓冲区放置在 DTCM 中。  
 参考第 4.4.1 节将 arm\_cmplx\_mag\_f32 函数分配到 ITCM，参考第 4.4.2 节将 testInput\_f32\_10khz 缓冲区分配到 DTCM。成功配置如图 79 所示。

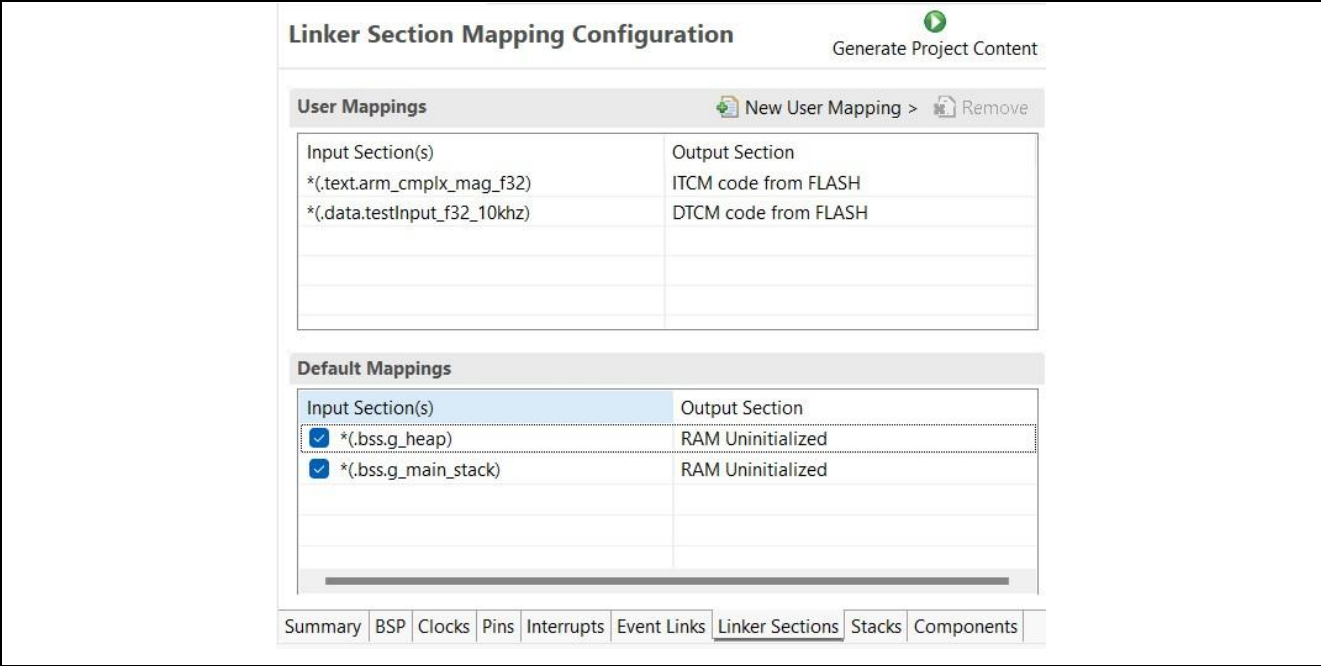


图 79. DTCM 和 ITCM 中数据和函数设置成功的示例

**步骤 6: 在 RA8D2\_DSP\_example\_CPU0 项目中生成项目内容**

在 RA8D2\_DSP\_example\_CPU0/configuration.xml 下，点击 Generate Project Content 以应用 CPU0 项目中的所有 FSP 配置。

**步骤 7: 实现 CPU0 特定的任务。**

RA8P1\_DSP\_example\_cpu0 项目的 /src 文件夹下实现的所有应用源文件与 RA8D2\_DSP\_example\_CPU0 项目兼容。因此，只需将 RA8P1\_DSP\_example\_cpu0/src 中的所有内容复制到 RA8D2\_DSP\_example\_CPU0/src。

**步骤 8: 编译 CPU0 项目。**

右键单击 RA8D2\_DSP\_example\_CPU0 项目并选择 Build Project。

**步骤 9: 在 CPU1 项目中添加 FSP Stack**

此步骤还将 RA8P1\_DSP\_example\_cpu1 项目中的 FSP 配置应用到 RA8D2\_DSP\_example\_CPU1 项目。

与步骤 3 中执行的手动配置不同，此步骤利用 e<sup>2</sup> studio 提供的 Import/Export FSP Stacks 功能，简化了将现有项目移植或迁移到不同目标 MCU 的过程。

1. 从 RA8P1\_DSP\_example\_cpu1 项目导出配置：
2. 打开 RA8P1\_DSP\_example\_cpu1 项目中的 configuration.xml 文件。
3. 在 Ipc Thread 下，右键单击每个模块并选择 Export，如图 80 所示。

将导出的配置保存为新的 .xml 文件（例如 ipc\_thread\_config.xml），确保勾选 Include Common Properties 选项，然后点击 Finish，如图 81 所示。

图 80. 导出 RA8P1\_DSP\_example\_cpu1 中的 FSP Stack

图 81. Stack Export 对话框中的设置示例

对 RA8P1\_DSP\_example\_cpu1 项目中使用的 FSP Stack 重复相同的导出过程。建议为每个线程分别导出每个 FSP Stack，以便在项目移植或迁移到另一个 MCU 时使配置管理和重新导入更加容易。在此示例中，FSP Stack 分为两类分别导出：ipc\_thread 和 ui\_thread。

成功从 RA8P1\_DSP\_example\_cpu1 项目导出所有 FSP Stack 后，继续下一步将 FSP Stack 导入到 RA8D2\_DSP\_example\_CPU1 项目。

**步骤 10: 将 FSP Stack 导入到 RA8D2\_DSP\_example\_CPU1 项目**

双击 RA8D2\_DSP\_example\_CPU1 项目的 configuration.xml，然后导航到 Stacks 选项卡，手动创建两个线程 Ipc Thread 和 UI Thread，其常规设置如图 82 所示，每个线程的详细配置如图 83 所示。

图 82. RA8D2\_DSP\_example\_CPU1 项目中 FreeRTOS 线程设置的示例

1. 图 83. RA8D2\_DSP\_example\_CPU1 项目中线程配置的示例。从 RA8P1\_DSP\_example\_cpu1 项目导入之前保存的 FSP Stack：
2. 在 Ipc Thread Stacks 中，右键单击并选择 Import。

在 Stack Import 对话框中，导航到步骤 9 中导出的 ipc\_thread\_config.xml 文件，然后点击 Select All 并启用 Include Common Properties 选项。最后，点击 Finish 完成导入过程。

1. 图 84. RA8D2\_DSP\_example\_CPU1 项目中 Ipc Thread 的 Stack Import 对话框示例。对 UI Thread 下导入 FSP 配置执行相同的过程：

2. 在 UI Thread Stacks 中，右键单击并选择 Import。

在 Stack Import 对话框中，导航到步骤 9 中导出的 ui\_thread\_config.xml 文件，然后点击 Select All 并启用 Include Common Properties 选项。最后，点击 Finish 完成导入过程。

#### 图 85. RA8D2\_DSP\_example\_CPU1 项目中 UI Thread 的 Stack Import 对话框示例

步骤 11: 在 RA8D2\_DSP\_example\_CPU1 项目的 FSP Configurator 中，导航到 Pins 选项卡，确保 SCI UART 通道 8 引脚（PD02、PD03）和外部中断引脚（P009）都已启用，如图 86 和图 87 所示。

图 86. RA8D2\_DSP\_example\_CPU1 项目中 SCI UART 引脚配置的示例

图 87. RA8D2\_DSP\_example\_CPU1 项目中外部中断引脚配置的示例。步骤 12: 将 ipc\_callback 放置到 CTCM。

关于将 ipc\_callback 函数分配到 CTCM 的详细信息，请参考第 4.4.3 节。

步骤 13: 点击 **Generate Project Content** 以应用所有配置设置。

步骤 14: 实现 CPU1 特定的任务。

将 RA8P1\_DSP\_example\_cpu1/src 下的文件复制到 RA8D2\_DSP\_example\_CPU1/src。

步骤 16: 按照第 7.3 节的描述调试和验证项目。

## 9. 参考资料

以下文档用于创建本快速设计指南：

- 瑞萨 RA8P1 群组用户手册 硬件篇，文档编号 R01UH1064
- 瑞萨 RA8D2 群组用户手册 硬件篇，文档编号 R01UH1065
- 瑞萨 RA8M2 群组用户手册 硬件篇，文档编号 R01UH1066。瑞萨 RA8T2 群组用户手册 硬件篇，文档编号 R01UH1067
- RA Flexible Software Package 文档 发布版 v6.2.0。
- RA8P1 内存架构应用笔记，文档编号 R01AN7880
- Reference System Design for Vision AI Design using Ethos-U NPU，文档编号 R11AN0995
- Using the Ethos-U NPU with RA8 MCUs，文档编号 R01AN7712
- Arm® Cortex®-M85 处理器技术参考手册，文档编号 101924，可从 Arm 获取
- Arm® Cortex®-M85 处理器技术参考手册，文档编号 101924，可从 Arm 获取

**网站与支持****RA 产品信息**[www.renesas.com/ra](http://www.renesas.com/ra)**RA 产品支持论坛**[www.renesas.com/ra/forum](http://www.renesas.com/ra/forum)**RA Flexible Software Package**[www.renesas.com/FSP](http://www.renesas.com/FSP)**瑞萨支持**[www.renesas.com/support](http://www.renesas.com/support)

访问以下 URL 以了解 RA 系列的关键要素、下载组件及相关文档，并获取支持。

| 版本   | 日期        | 描述 |                |
|------|-----------|----|----------------|
|      |           | 页码 | 摘要             |
| 1.00 | 2025.6.30 | -  | 初始版本           |
| 1.10 | 2026.1.30 | -  | 更新至 FSP v6.2.0 |

## 微处理器单元和微控制器单元产品使用中的一般注意事项

1. 以下使用注意事项适用于瑞萨所有的微处理器单元和微控制器单元产品。关于本文档所涵盖产品的详细使用注意事项，请参考文档的相关章节以及已为该产品发布的任何技术更新。  
静电放电（ESD）预防措施
2. 强电场暴露在 CMOS 器件上时，可能导致栅极氧化层损坏并最终降低器件性能。必须采取措施尽量阻止静电的产生，并在产生静电时迅速将其消散。环境控制必须包括适当的湿度水平。  
上电时的处理
3. 上电时产品的状态是不确定的。LSI 内部电路的状态不确定，上电时寄存器设置和引脚的状态也是不确定的。在复位信号施加到外部复位引脚并已识别到系统时钟的最终产品中，从上电到复位完成期间，引脚的状态无法保证。  
断电状态下输入信号
4. 不要在器件断电时输入信号或 I/O 上拉电源。由此产生的电流注入可能导致故障，此时通过器件的异常电流可能导致内部元件的退化。  
未使用引脚的处理
5. 按照手册中关于未使用引脚处理的指示处理未使用的引脚。CMOS 产品的输入引脚通常处于高阻抗状态。未使用的引脚处于开路状态时，会在引脚附近感应出额外的电磁噪声，导致器件发生故障。  
时钟信号
6. 施加复位后，只有在工作时钟信号稳定后才释放复位线。在程序执行期间切换时钟信号时，需等待目标时钟信号稳定。当时钟信号由外部谐振器或外部振荡器产生时，需充分考虑振荡稳定时间。  
输入引脚的电压施加波形
7. 由输入噪声或反射波引起的波形失真可能导致故障。如果 CMOS 器件的输入由于噪声等原因停留在 VIL（最大值）和 VIH（最小值）之间的区域，器件可能会发生故障。当输入电平固定以及输入电平在翻转过程中通过 VIL（最大值）和 VIH（最小值）之间的区域时，请注意防止抖动噪声进入器件。  
禁止访问保留地址
- 禁止访问保留地址。保留地址是为将来可能的功能扩展而提供的。不要访问这些地址，因为 LSI 的正确操作无法得到保证。

## 产品之间的差异

在从一种产品更换为另一种产品（例如更换为不同型号的产品）之前，请确认该更换不会导致问题。同一群组但型号不同的微处理器单元或微控制器单元产品的特性可能在内存容量、布局等方面有所不同。

- 1.
2. 声明
3. 本文档中关于电路、软件和其他相关信息的描述仅用于说明半导体产品的操作和应用示例。您对将这些电路、软件和信息纳入您的设备或系统的设计以及对其使用承担全部责任。瑞萨电子对因使用这些电路、软件或信息而可能导致的任何损失或损害不承担任何责任。
4. 瑞萨电子特此明确否认因使用本文档中描述的瑞萨电子产品或技术信息而产生的或与之相关的、针对第三方专利、版权或其他知识产权侵权的任何保证和责任。
5. 本文档不以明示、暗示或以其他方式授予瑞萨电子或其他方的任何专利、版权或其他知识产权项下的任何许可。
6. 您应负责确定需要从任何第三方获得哪些许可，并为任何包含瑞萨电子产品的产品的合法进口、出口、制造、销售、使用、分销或其他处置获取此类许可（如果需要）。  
您不得更改、修改、复制或反向工程任何瑞萨电子产品，无论是全部还是部分。瑞萨电子对因您或第三方进行此类更改、修改、复制或反向工程而产生的任何损失或损害不承担任何责任。  
瑞萨电子产品按以下两个质量等级分类："Standard"和"High Quality"。每个瑞萨电子产品的预期应用取决于该产品的质量等级，如下所示。  
"Standard": 计算机；办公设备；通信设备；测试和测量设备；音频和视频设备；家用电器；机床；个人电子设备；工业机器人等。
7. "High Quality": 运输设备（汽车、火车、船舶等）；交通控制（交通信号灯）；大规模通信设备；关键金融终端系统；安全控制设备等。

除非在瑞萨电子产品数据手册或其他瑞萨电子文档中明确指定为高可靠性产品或适用于恶劣环境的产品，否则瑞萨电子产品不拟用于或授权用于可能对人体生命构成直接威胁的产品或系统：

没有任何半导体产品是绝对安全的。

尽管瑞萨电子硬件或软件产品中可能实现了安全措施或功能，瑞萨电子对因任何漏洞或安全漏洞（包括但不限于未经授权访问或使用瑞萨电子产品产生的任何知识产权损失或损害）而产生的任何责任概不承担。

瑞萨电子不保证或担保瑞萨电子产品、或使用瑞萨电子产品创建的任何系统将是坚不可摧的或免受损坏、攻击、病毒、干扰、黑客攻击、数据丢失或盗窃或其他安全入侵（“漏洞问题”）。

瑞萨电子对因任何漏洞问题引起或与之相关的任何及所有责任或义务概不承担。

- 此外，在适用法律允许的范围内，瑞萨电子对本文档及任何相关或随附的软件或硬件不提供任何明示或暗示的保证，包括但不限于适销性或适用于特定目的的暗示保证。

使用瑞萨电子产品时，请参考最新的产品信息（数据手册、用户手册、应用笔记、可靠性手册中的“半导体器件处理和使用的注意事项”等），并确保使用条件在瑞萨电子规定的最大额定值、工作电源电压范围、散热特性、安装条件等范围内。

瑞萨电子对因在超出规定范围使用瑞萨电子产品而引起的任何故障、失效或事故概不承担任何责任。

- 尽管瑞萨电子努力改进瑞萨电子产品的质量和可靠性，半导体产品具有特定的特性，例如在一定条件下会发生一定概率的故障和误动作。除非指定为高可靠性产品或适用于恶劣环境的产品，否则瑞萨电子产品不进行抗辐射设计。
- 有关每个瑞萨电子产品环境兼容性等环保事项的详细信息，请联系瑞萨电子销售办事处。您有责任仔细并充分调查规范包含或使用受控物质的法律法规。
- 瑞萨电子产品和技术不得用于或被纳入任何根据适用国内或外国法律法规禁止制造、使用或销售的产品或系统中。您应遵守任何适用出口管制法律法规的规定。
- 瑞萨电子产品的购买方或分销商，或任何分销、处置或以其他方式销售或转让产品给第三方的其他方，有责任提前将该方告知本文档中规定的内容和条件。
- 未经瑞萨电子事先书面同意，不得以任何形式转载、复制或复印本文档的全部或部分内容。

如果您对本文档中包含的信息或瑞萨电子产品有任何疑问，请联系瑞萨电子销售办事处。

（注 1）本文档中使用的“瑞萨电子”指瑞萨电子株式会社及其直接或间接控制的子公司。

## 公司总部

TOYOSU FORESIA, 3-2-24 Toyosu,  
Koto-ku, Tokyo 135-0061, Japan  
[www.renesas.com](http://www.renesas.com)

## 联系方式

有关产品、技术、最新版本的文档或 **nearest sales office** 的更多信息，请访问：[www.renesas.com](http://www.renesas.com)

## 商标

**Renesas 和 Renesas 标志是 Renesas Electronics Corporation 的商标。所有商标和注册商标均为其各自所有者的财产。**

Renesas and the Renesas logo are trademarks of Renesas Electronics